

Kursus TSIX

Edisi Indonesia

25 modul

Kernel, VFS, Jaringan, GUI & Desktop, Pengembangan

System design & architecture: Andriansah

Dibuat: 2026-08-12

Platform: TSIX (educational OS-like runtime on Node.js/TypeScript)

Daftar Isi

00	Overview & Peta Mental	Fondasi
01	Filosofi & Gambaran Besar	Fondasi
02	Model Ring & Batas Privilege	Fondasi
03	Boot Sequence	Boot & Kernel Runtime
04	Proses & Scheduler	Boot & Kernel Runtime
05	Syscall & IPC	Boot & Kernel Runtime
06	Permission & Security	Boot & Kernel Runtime
07	Mount & Path Resolution	Boot & Kernel Runtime
08	VFS	Storage & I/O
09	FD Table & File Syscalls	Storage & I/O
10	Device Drivers (HAL)	Storage & I/O
11	Worker Thread & Sandbox	Isolasi Proses
12	Module Resolution & Direct Memory Execution	Isolasi Proses
13	TTY & Virtual Console	Interaksi Manusia
14	Userland: init, login, shell, app	Interaksi Manusia
15	Networking MQTNL	Jaringan
16	Wire Protocol MQTNL	Jaringan
17	PixelSpace Protocol	GUI & Desktop
18	DOM Engine (Display Server)	GUI & Desktop
19	Emerald Widget Toolkit	GUI & Desktop
20	Cashew Component Framework	GUI & Desktop
21	Asteracea & TDE	GUI & Desktop
22	State Replay & Persistence	GUI & Desktop
23	Development Workflow	Pengembangan
24	Best Practices & Penulisan App	Pengembangan

RFC-TSIX-000 | Peta mental arsitektur TSIX — untuk kontributor (manusia & AI), hobbyst, penggiat edukasi, dan profesional.

Dokumen ini adalah **peta mental sistem**. Ia menjelaskan *apa, di mana, dan kenapa* — bukan tutorial per-file. Gunakan sebagai titik masuk sebelum membaca kode, dan sebagai rujukan saat menjelajah subsistem.

Ini adalah **Modul 00** dari kurikulum TSIX. Untuk roadmap lengkap, lihat [toc.md](#).

Daftar Isi

1. [Filosofi & Gambaran Besar](#)
2. [Model Ring & Batas Privilege](#)
3. [Boot Sequence \(Host → Userland\)](#)
4. [Kernel Core: Proses, Scheduler, Syscall, IPC](#)
5. [VFS & Device Drivers \(HAL\)](#)
6. [Worker Thread & Sandboxing](#)
7. [Networking MQTNL](#)
8. [TTY & Virtual Console](#)
9. [Userland: Init, Shell, Aplikasi](#)
10. [PixelSpace & TDE](#)
11. [Alur Pengembangan \(Sync VFS\)](#)
12. [Insight Desain & Gotchas](#)
13. [Peta Membaca Kode](#)

1. Filosofi & Gambaran Besar

TSIX adalah **sistem operasi simulasi berbasis Node.js + TypeScript**. Ia bukan VM yang mengemulasi CPU — ia membangun **abstraksi OS di atas runtime Node.js yang sudah ada**.



Tiga prinsip inti:

1. **"Syscall = satu-satunya pintu"** — Worker thread tidak pernah menyentuh resource langsung. Bahkan `print` pun `syscall`. Ini menjaga batas ring benar-benar berarti.
2. **"Everything is a File"** — file dan device sama-sama `IDevice`; `read/write` polimorfik. Buka file biasa → `FileSystemDevice`; buka `/dev/tty1` → `TTYDevice`.
3. **"Direct Memory Execution"** — framework (`/lib`) di-pre-compile ke memori saat boot, dikirim ke worker via `workerData`, dieksekusi tanpa hit filesystem. `@tsix/*` terasa instan.

Kunci: kernel tidak pernah menjalankan app. Semua userland (termasuk PID 1 / init) berjalan di Worker Thread. Batas Ring 1/2 vs Ring 4 adalah batas *thread + IPC*, diperkuat `PermissionManager` berbasis uid/gid/mode.

2. Model Ring & Batas Privilege

Ring adalah **konsep** (dokumentasi), bukan mekanisme hardware/isolasi V8:

Ring	Isi	File utama
0	Host: Linux + Node/V8	— (reserved)
1	Kernel core: Scheduler, Syscall, Permission	<code>src/kernel/*</code> , <code>src/common/*</code>
2	Driver & FS: HAL devices, VFS backends, MountManager	<code>src/kernel/devices/*</code> , <code>src/vfs/*</code>
3	Library framework: UserLib, Application	<code>src/mirror/lib/*</code>
4	Aplikasi: <code>/bin/*</code> , <code>init</code> , <code>tsh</code> , <code>daemon</code>	<code>src/mirror/bin/*</code>

Batas privilege nyata ada di dua lapisan:

- WorkerEntry sandbox** — app hanya boleh `require framework @tsix/* / @common/*`;
`process.exit / process.kill` disabotase. App privileged (nama mengandung `server / daemon / dome / tbuild / vfs`) dapat akses allow-list modul host (`http`, `ws`, `fs`, `crypto`, `esbuild`, `dll`).
- PermissionManager (kernel)** — cek `rwX`: root (uid 0) bypass → owner → group → others. SetUID bit didukung (mis. `/bin/login 0o4755`).

⚠ Catatan kontributor: status privileged berbasis **substring nama app** — heuristik rapuh. Ini gap arsitektural yang layak diperbaiki (ideally capability-based).

3. Boot Sequence

```
main.ts
Config.load()           → baca sysconfig.json
new Kernel()            → Logger, PermissionManager, MountManager, GUIRegistry
kernel.boot()
  initializeSubsystems()
    mount BKFS("/")      → SQLite system.db (root filesystem)
    processFstab()       → /tmp (RamFS), /mnt/* (HostVFS), /mnt/sbak (BKFS)
    new Scheduler()
    new PermissionManager()
    new PortManager()
    new SyscallDispatcher() → scheduler.setSyscallHandler(...)
    rebuildVFSCache()    → pre-compile /lib ke memori
  new TTYManager(32) + TTYDevice tty1..32
  devices{}              → stdin, fb0, stdout, stderr, null
  init network interfaces → SimpleMQTNLDriver per config
  loadAuxDevices()        → plugin driver (random, mysql, mcp23017)
  SerialDeviceManager     → auto-detect ttyUSB*
  applyDeviceConfigs()    → "udev": mode/uid/gid dari sysconfig
  pastikan /dev ada di VFS
  load RSA identity        → fingerprint visual di TTY
  ensureDefaultAuth()     → seed /etc/passwd + /etc/shadow (root)
  wire keyboard + TTY callbacks → Ctrl+C → SIGINT fg; Alt+F1-6 → switch
kernel.runInit()
  resolve /bin/init.js
  createProcess("init", fds:[tty1x3]) → PID 1
  setForegroundProcess(1, 1)
[init.ts di Worker]
  enforce setuid (passwd, sudo)
  generate/verify RSA identity
  exec /etc/rc.local.js + waitpid
  spawn login TTY2..6 (monitorProcess → respawn)
```

```
spawn login TTY1 (foreground) → loop forever
keepAlive (100ms di main.ts)
jika PID 1 EXITED → process.exit(exitCode) (1 = reboot)
```

Urutan rc.local (daemon): airtermd (remote access) → tpkgd (package server) → scpd (SCP) → otad (OTA ESP) → iot-listener (MQTNL sensor) → dome (display server, harus setelah listener) → Asteracea WM (delay 1s, menunggu DOME) → crond.

4. Kernel Core

4.1 Proses (PCB)

interface PCB di Scheduler.ts: pid, ppid, name, state, pc, owner/uid/gid/ruid/groups, cwd, worker?, fdTable, env, exitCode?, ttyId?, uuid?.

Lifecycle: READY → RUNNING → BLOCKED → EXITED

- **Spawn:** createProcess() → alokasi pid → PCB → spawnWorker() (Worker Thread).
- **Exit:** EXIT(code) → cleanup FD → worker.terminate(). Event worker.on("exit") → reparent orphan ke PID 1 → resolve waitQueue → reap() (zombie jika tanpa waiter).
- **waitpid:** jika EXITED → reap + return exitCode; jika belum → daftar di waitQueue.
- **daemonize:** DETACH → resolve waiters, lepas foreground TTY, kosongkan ttyld.
- **REEXEC:** terminate worker lama, spawn baru dengan **PID/PCB sama** (state REEXECING mencegah cleanup hook).

4.2 Syscall Dispatch

```
App → UserLib.dispatch(code, args)
→ postMessage({ requestId: uuid, pid, code, args })
→ Kernel: scheduler → syscallHandler.handleRequest(req)
  → validateArgs(code, args) // kontrak argumen
  → dispatch(pid, code, args) // switch-case ~65 syscall
  → PermissionManager.check() // satpam
  → MountManager.resolve() // path → backend
  → VFS / device / scheduler
→ postMessage({ requestId, success, data|error })
→ UserLib: cocokkan requestId di responseMap → resolve/reject
```

Korelasi request-response via requestId (UUID) + responseMap — murni RPC asinkron, tidak ada urutan dijamin. Banyak syscall bisa in-flight sekaligus.

Push event (kernel → worker, tanpa requestId): { type, data }. Tipe: signal, ipc_message (SEND_MSG), gui_request (forwarding ke gued), resize.

4.3 Sinyal

Sinyal	Efek
SIGKILL (9)	Hard kill: worker.terminate() langsung
SIGINT (2)	Graceful: push event, grace 100ms → terminate jika tak ditangani. Default exit 130
SIGTERM (15)	Graceful: grace 300ms. Default exit 143. Dipakai SHUTDOWN
SIGSTOP/SIGCONT	Soft: ubah pcb.state BLOCKED/RUNNING + event
SIGSEGV	Dikirim ke PID yang melanggar kepemilikan window GUI
SIGHUP/USR1/USR2/WINCH	Push event generik

PID 1 diproteksi dari kill — hanya SHUTDOWN/REBOOT (root) yang sah. SHUTDOWN bertingkat: SIGTERM broadcast → 5s → SIGKILL survivor → flush network 1s → kill PID 1.

4.4 MountManager & PortManager

- **MountManager**: array `MountPoint {vfsPath, vfs, type, source, readOnly, uid, gid}`. `mount()` sortir descending by path length; `resolve()` → prefix terpanjang menang → `{vfs, relativePath, mountPoint}`.
- **PortManager**: port virtual 0-65535 untuk MQTNL; `allocatePort`, `allocateRandomPort(10000-20000)` untuk `bind(0)`, `releasePortsByPid()` saat proses exit (jaring pengamanan socket bocor).

5. VFS & Device Drivers (HAL)

5.1 Lapisan VFS

```
Aplikasi → syscall OPEN/READ/WRITE
→ MountManager.resolve(path) → backend IVFS
  "/"      → BKFS    (SQLite system.db, persisten)
  "/tmp"   → RamFS   (volatile, in-memory)
  "/mnt/*" → HostVFS (bridge folder host, anti-escape)
→ path /dev/xxx → kernel.devices[xxx] (HAL, bypass MountManager)
```

- **IVFS** = satu kontrak (`ls/mkdir/read/touch/stat/chmod/chown/.../readChunk/writeChunk/getSize`) untuk semua backend — memungkinkan "swap backend" tanpa ubah syscall.
- **BKFS**: file = baris di tabel `vnodes` (parent_id tree). Chunked I/O dijalankan **dalam SQL** (`SUBSTR/CONCAT`) tanpa menarik konten penuh — untuk file besar/OTA.
- **HostVFS**: `toHostPath()` + cek anti-escape (Security Violation) — view read-only folder host.

5.2 Device Model (HAL)

`IDevice`: `read/write/ioctl` + optional `init/open/close` + metadata `uid/gid/mode`.

/dev/	Class	Fungsi
<code>stdin</code>	KeyboardDevice	Input keyboard (cooked/raw)
<code>fb0/stdout/stderr</code>	TTYDevice	Output standar → TTY aktif
<code>tty1..tty32</code>	TTYDevice	Virtual console
<code>null</code>	NullDevice	Lubang hitam
<code>smqtnl0/1</code>	SimpleMQTNLDriver	Network interface MQTNL
<code>randomdevice</code>	RandomDevice	Angka acak
<code>mysql (eksperimental)</code>	MySQLDevice	Koneksi DB eksternal — POC integrasi, lihat catatan di bawah
<code>mcp23017</code>	MCP23017Device	GPIO I2C
<code>ttyUSB*</code>	SerialDevice	Auto-detect
<i>(virtual)</i>	PipeDevice / SocketDevice	Instance runtime (syscall PIPE/SOCKET), bukan path

Kunci "everything is a file": FD table berisi `FDEntry {device, context, flags}` → semua objek `IDevice`. File biasa dibungkus `FileSystemDevice`. Pipe refcount via `ioctl (INC_REF/DEC_REF)`, EOF saat `writeRefs==0`.

Plugin driver: folder `aux-devices/` di-scan saat boot; `new DeviceClass()` + konvensi `static autoRegister(kernel)` untuk hardware (MCP23017). `applyDeviceConfigs()` = "udev" dari `sysconfig.json`.

[!NOTE] **Soal MySQLDevice (/dev/mysql)** — **transport pertama, bukan satu-satunya** `MySQLDevice` adalah integrasi database eksternal lewat model device — ia **bukan** driver hardware sungguhan, melainkan **transport pertama** untuk akses DB (contoh perluasan HAL). **DbLib sudah terimplementasi** (sub-library `UserLib`, pola `lib.fs/lib.net`) dengan **dual transport pluggable**:

- **Device** (`/dev/mysql`): syscall `DB_*` (67-69) → kernel → device → `mysql2`
- **Service daemon** (`mysqld`): syscall `DB_*` → kernel → daemon Ring 4 → `mysql2`

Kernel me-route secara **dinamis**: jika `mysqld` terdaftar (`syscall DB_SERVICE_REGISTER=70`) → ke daemon; jika tidak → ke device (fallback). Daemon membalas via `DB_SERVICE_REPLY=71`. Aplikasi cuma lihat `db.connect()/query()/disconnect()` — medium tak terlihat. Sandbox aman karena `mysql2` hanya disentuh kernel atau daemon privileged.

6. Worker Thread & Sandboxing

6.1 Bootstrap Worker

```
Kernel.spawnWorker()
  workerData = { pid, appName, args, appPath, appContent, env, vfsCache }
  execArgv:
    *.js → JS-Direct (FAST)    : --enable-source-maps
    *.ts → TS-Transpile        : -r esbuild-register -r tsconfig-paths/register

WorkerEntry.ts (bootloader)
  1. realExit = process.exit (sebelum sandbox)
  2. hijack Module._load → resolve @tsix/* & @common/* dari vfsCache (DME)
  3. pasang unhandledRejection/uncaughtException → kirim error ke parent
  4. new UserLibClass(pid) → global._tsixLib
  5. muat app:
      DME : appContent di-transpile, __filename = BKFS path
      fisik: hostRequire(finalAppPath)
  6. cari export main/Main/default
  7. restrictHostAPI(appName) ← kunci pintu
  8. new AppClass() → await execute(lib, args)
  9. return string → std.print; lalu shell.exit(0)
```

6.2 Direct Memory Execution

Kernel pre-compile `/lib` → `vfsCache` → dikirim via `workerData` → worker `_compile` dari memori. Trick penting: modul framework diberi nama `@tsix_Application.js` supaya import relatif `./x` bisa di-rewrite jadi `@tsix/x` — siklus alias konsisten. App punya **dua identitas filename**: fisik (biar `require` nemu `node_modules`) vs BKFS (biar stack trace benar).

6.3 Sandbox Table

Lapisan	Batasan
Semua app	<code>process.exit</code> / <code>process.kill</code> → throw; <code>require</code> non-framework diblokir
App non-privileged	<code>http/fs/crypto</code> dll diblokir total
App privileged	allow-list: <code>http</code> , <code>ws</code> , <code>path</code> , <code>fs</code> , <code>url</code> , <code>esbuild</code> , <code>crypto</code> , <code>os</code> , <code>bcryptjs</code>
Kernel	<code>PermissionManager</code> + <code>validateArgs</code> + <code>SETUID</code> root-only

7. Networking MQTNL

MQTNL (MQTT Network Layer) = protokol networking TSIX. Alih-alih IP/routing TCP/IP, ia memakai **MQTT pub/sub** sebagai wire, plus:

- **Alamat** = nama node (string: `"tsix"`, `"esp32S3"`) — bukan IP.
- **Port** = endpoint aplikasi (PortManager 0-65535).
- **Topic** = `mqtnl@1.0/<dstAddress>` (JSON) / `mqtnl@1.1/<dstAddress>` (Binary).
- **Packet**: header 9 field + payload; fragmentasi 32KB; reassembly TTL 30s; enkripsi RSA + ChaCha20-Poly1305.

```
App → socket() → bind(port) → driver.registerHandler(port)
→ sendto(addr, port, data) → driver.send → publish topic
→ MQTT broker → semua node subscribe 'mqtnl@1.x/#'
```

```
→ filter dstAddress → reassembly → decrypt → socket.push()
→ recvfrom() → buffer.shift()
```

Dual protocol: MQTTNLProtocolJSON (v1.0, legacy, readable) vs MQTTNLProtocolBinary (v1.1, kompak, tanpa enkripsi — untuk OTA byte-exact). Driver deteksi dari **magic byte pertama** per srcAddress. `src/common/protocols/` berisi kontrak IMQTNLProtocol — userland **tidak pernah menyentuh ini langsung**.

Syscall: SOCKET=30, BIND=31, SENDTO=32, RECVFROM=33, NETSTAT=34. Tidak ada `listen/accept` — UserLib emulasi: `listen()` = `socket+bind`, `accept()` = `polling recv()`.

8. TTY & Virtual Console

- **TTYManager** mengelola 1–32 konsol virtual; switch hanya 1–12, login di 1–6.
- **TTY** (`src/kernel/tty/TTY.ts`): `buffer char[x][y]`, cursor, parser ANSI subset, render ulang penuh.
- **TTYDevice** (`/dev/ttyN`): `ioctl clear(1)`, `raw(10)`, inject input(0x2001), read output(0x2002), window size(4) — memungkinkan aplikasi remote (pixelterm) mengontrol TTY.
- **Alokasi TTY ke proses:** via `ttyId` di syscall EXEC — kernel override FD 0/1/2 ke `tty{n}`. (Beda dengan PortManager yang khusus jaringan.)
- **Keyboard:** cooked/raw mode, Ctrl+C → SIGINT fg, Alt+F1-6 → TTY switch, resize → SIGWINCH broadcast.

9. Userland

9.1 Init (PID 1)

Tanpa `/etc/inittab` — logika hardcoded di `init.ts`: enforce `setuid` → RSA identity → exec `rc.local` → spawn login per TTY (via `monitorProcess` respawn anti-crash-loop) → loop.

9.2 Login → Shell

`login.ts`: verifikasi `/etc/passwd` + `/etc/shadow` (bcrypt) → `setgroups` → `setgid` → `setuid` (urutan POSIX) → exec `$SHELL` → `tsh.ts`.

9.3 Dua Gaya Aplikasi

1. **Class IProgram** (legacy mayoritas): `export class main implements IProgram { async execute({fs, shell, std}: OSContext, args) {...} }` — `ls`, `cat`, `ps`, `kill`, `tsh`.
2. **Program() wrapper + proxy singletons** (baru): `import { Program, std, fs, shell, net } from "@tsix/Application"` — `esp-send`, `dome`, `iot-dashboard`.

9.4 Error = "Window"

`std.error()` melakukan 4 hal: tulis syslog → broadcast ke parent → print TTY merah → kirim `GUI_WINDOW_ERROR` ke WM (dengan `wid`, `pid`, `fileHint` dari stack trace).

9.5 DbLib — Database Sub-Library

DbLib (`@tsix/DbLib`, `lib.db`) adalah sub-library kelima UserLib (setelah `std/fs/shell/net`). Ia membungkus syscall `DB_*` (67–71) menjadi API `db.connect()/query()/disconnect()`. **Transport pluggable:** kernel me-route ke `/dev/mysql` (device) atau `mysqld` (service daemon Ring 4) secara dinamis. Aplikasi tidak tahu mediumnya — persis pola `lib.fs` yang tidak peduli backend VFS-nya.

```
App → db.query(sql) → dispatch(DB_QUERY=68)
→ Kernel: mysqld terdaftar?
    YA → sendEvent(daemon, "db_request") → mysql2 → dbServiceReply → resolve
    TIDAK → /dev/mysql device → mysql2 → resolve
→ App: rows
```


10. PixelSpace & TDE

Detail lengkap di `PIXELSPACE_DEVELOPER_GUIDE.md`. Ringkasan arsitektural:

```
Worker (app) → GUI_REQ (61) → Kernel (GUIRegistry auth pid→wid)
  → DOME daemon (WS broadcast) → Browser (DOM)
Browser → event → DOME → Kernel (SEND_MSG) → Worker (callback)
```

- **Kontrak:** `GUITypes.ts` (`IDOMNode`, `IGUIPayload`, `GUIAction`, `IBrowserEvent`, `IGUIEventIPC`) — "konstitusi", jangan diubah sembarangan.
- **GUIRegistry (kernel)** = otoritas tunggal kepemilikan window: `CREATE_WINDOW` = registrasi `wid→pid`; akses window orang → `SIGSEGV`; payload rusak → `SIGKILL`; proses mati → window auto-destroy.
- **DOME** = display server (Ring 4 daemon, port 8080): relay WS + primitive DOM producer + **kompositor** (titlebar, drag, resize, focus, replay). Monolitik karena pertimbangan latency drag/resize.
- **Emerald** = widget toolkit (`@tsix/emerald`): `Screen`, `Window`, factory functions, connected widgets.
- **Cashew** = component framework (`@tsix/cashew`): OOP/Delphi-style `TForm`/`TButton`/`TEdit`, auto-bind lifecycle, `TDialogs` & `TTimer` — layer di atas Emerald.
- **Asteracea** = window manager (fullscreen frameless app): taskbar, launcher, login, wallpaper; listen `GUI_WINDOW_*` lifecycle events via `/etc/asteracea/wm-pid`.

11. Alur Pengembangan

Siklus dev (host ↔ VFS):

```
scripts/vfs-bootstrap.ts  host → system.db (transpile TS→JS, chmod, setuid)
scripts/sync-vfs.ts       sync satu file host ↔ VFS
scripts/vfs-pull.ts       system.db → src/root (pull balik)
SYNC_TO_HOST (syscall)   DB → host (app dalam VFS menulis /lib → src/.tsix_sdk)
userlib-update.ts         sinkronkan /lib → src/.tsix_sdk/lib (agar Node.js host bisa require)
```

Konfigurasi: `src/sysconfig.json` (database path, `workerEntryPath`, `bootEntry`, network interfaces). `tsconfig.json` memetakan `@tsix/*` → `src/.tsix_sdk/lib/*`, `src/root/lib/*`, `src/mirror/lib/*`.

12. Insight Desain & Gotchas

Insight (kenapa didesain begini):

1. **Syscall sebagai ABI mini** — `SyscallCode` (1-66) + `validateArgs` + `requestId` correlation. Satu mekanisme untuk semuanya: file, proses, IPC, GUI, jaringan.
2. **RPC asinkron murni** — request/response via UUID; beda dari trap sinkron Linux.
3. **Kill bertingkat** — `SIGINT`/`SIGTERM` = event + grace period, baru terminate. Replikasi default Unix (unhandled signal → exit).
4. **Zombie & reparent setia** — orphan → PID 1; zombie menunggu `waitpid` + `reap`.
5. **UUID identitas** — `SET_IDENTITY` memungkinkan `SEND_MSG` ke nama stabil walau PID berubah (well-known services).
6. **GUI auth di kernel** — `payload.pid` di-override kernel (jangan percaya userland).
7. **MQTNL = no-IP gratis** — MQTT broker jadi backbone; by design untuk IoT (ESP32) tanpa IP publik.
8. **TTY = PTY emulation lengkap** — master-side I/O via `ioctl` memungkinkan terminal remote.

Gotchas (hati-hati saat kontribusi):

⚠ **Artefak compile basi:** `src/mirror/lib/*.js` dan `src/userland/WorkerEntry.js` bisa berbeda dari `.ts`-nya. Runtime memakai `.ts` yang di-recompile kernel, jadi `.js` yang di-commit rawan drift (contoh: daftar privileged di `WorkerEntry.js` tidak memuat `dome`).

⚠ **Dokumentasi vs kode:** `rc.local.ts` (sumber) memuat daftar daemon lengkap, tapi `rc.local.js` (yang dieksekusi) hanya 3 daemon. Wiki `Networking-MQTNL.md` menyebut topic `tsix/net/*` tapi kode memakai `mqtnl@1.0/`. **Kode adalah kebenaran.**

⚠ **Dead code:** `ExecutableRegistry` terdefinisi tapi tidak di-wire di Kernel — resolusi binary via `MountManager.resolve()` + `vfs.read()`.

⚠ **inittab tidak ada** — init meng-hardcode TTY login.

⚠ **Dual NetworkLib** — `UserLib.ts` (inline, `lib.net`) vs `src/mirror/lib/NetworkLib.ts` (legacy, OSContext-based) menuju syscall yang sama.

⚠ **Privilege berbasis nama** — heuristik substring app (rapuh; idealnya capability-based).

13. Peta Membaca Kode

Mulai dari sini (wajib)

File	Peran
<code>src/main.ts</code>	Entry point host + keep-alive
<code>src/kernel/Kernel.ts</code>	Orkestrator boot semua subsistem
<code>src/common/IPCTypes.ts</code>	Kontrak IPC (request/response/event)
<code>src/common/SyscallCode.ts</code>	ABI syscall (1-66)
<code>src/common/GUITypes.ts</code>	Kontrak PixelSpace

Kernel Core

File	Peran
<code>src/kernel/Scheduler.ts</code>	PCB, proses, sinyal, reexec
<code>src/kernel/Syscalls.ts</code>	Dispatcher ~65 syscall + permission
<code>src/kernel/PermissionManager.ts</code>	Cek rwx
<code>src/kernel/MountManager.ts</code>	Routing path → backend
<code>src/kernel/PortManager.ts</code>	Port network
<code>src/kernel/GUIRegistry.ts</code>	Otoritas window

VFS & Devices

File	Peran
<code>src/vfs/IVFS.ts</code>	Kontrak filesystem
<code>src/vfs/BKFS.ts</code>	Root FS (SQLite)
<code>src/vfs/VFS.ts</code> / <code>RamFS.ts</code> / <code>HostVFS.ts</code>	Backend lain
<code>src/kernel/devices/IDevice.ts</code>	Kontrak HAL
<code>src/kernel/devices/FileSystemDevice.ts</code>	Jembatan file ↔ IVFS
<code>src/kernel/devices/*.ts</code>	Driver: TTY, Keyboard, Pipe, Socket, MQTNL
<code>src/kernel/devices/aux-devices/</code>	Plugin driver

Worker & Userland

File	Peran
src/userland/WorkerEntry.ts	Bootloader worker + sandbox + DME
src/mirror/lib/UserLib.ts	"libc" sisi worker
src/mirror/lib/Application.ts	Framework Program() v2.1
src/mirror/lib/emerald.ts	Widget toolkit
src/mirror/bin/init.ts	PID 1

Networking & Protokol

File	Peran
src/kernel/devices/SimpleMQTNLDriver.ts	Driver MQTNL
src/common/protocols/IMQTNLProtocol.ts	Kontrak protokol
src/common/protocols/MQTNLProtocolJSON.ts	v1.0
src/common/protocols/MQTNLProtocolBinary.ts	v1.1

GUI/TDE

File	Peran
src/mirror/bin/dome.ts	DOME server
src/mirror/bin/dome-client.html	DOME browser
src/mirror/bin/asteracea.ts	WM
src/mirror/lib/theme.ts	Tema

Lampiran: Ringkasan alur data kunci

Baca file:

```
fs.readFile("/etc/passwd")
→ OPEN: resolve → MountManager → BKFS → FileSystemDevice → fd
→ READ: fdTable[fd].device.read() → vfs.read → SQL SELECT
→ CLOSE: ioctl DEC_REF → fdTable[fd] = null
```

Spawn app:

```
shell.exec("/bin/ls")
→ EXEC: resolve → vfs.read(appContent) → permission EXECUTE
→ createProcess → spawnWorker → WorkerEntry → DME app → execute()
```

Kirim pesan antar app:

```
shell.send(pidOrUuid, {type, ...})
→ SEND_MSG → kernel resolve pid (atau uuidMap) → sendEvent(pid, "ipc_message", data)
```

Buka window:

```
Screen → CREATE_WINDOW → kernel (GUIRegistry.auth) → DOME → WS → browser
```

RFC-TSIX-EDU-002 | Modul pertama kurikulum TSIX. Membangun peta mental: apa itu TSIX, mengapa dirancang begini, dan tiga prinsip inti yang menjadi fondasi seluruh sistem.

Sebelum menulis kode, pahami dulu **filosofinya**. TSIX bukan sekadar "emulator OS" — ia adalah **abstraksi OS yang dibangun di atas runtime Node.js yang sudah ada**. Modul ini menjelaskan *kenapa* ia dirancang demikian, dan empat prinsip yang konsisten di semua subsistem.

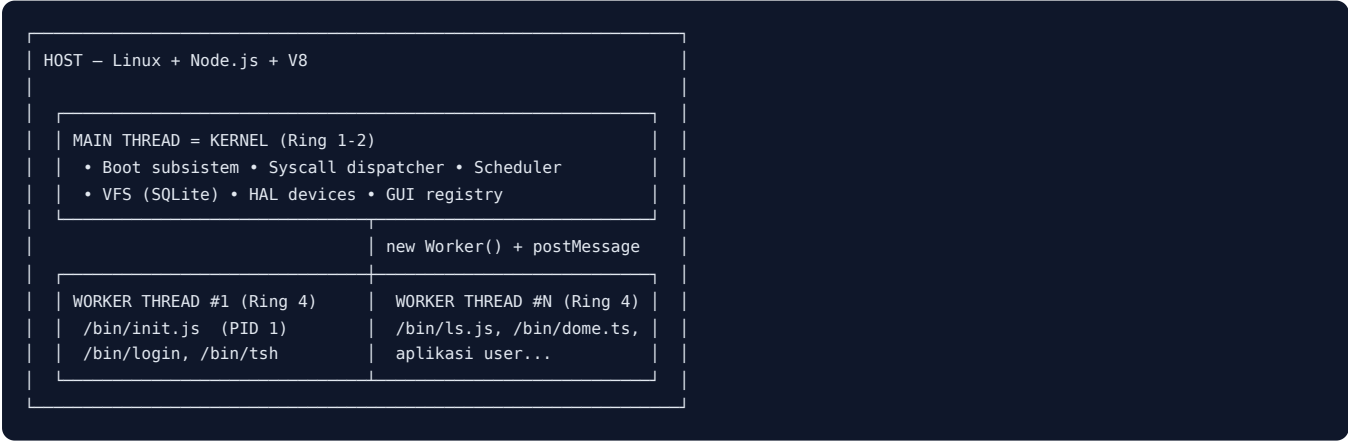
Tujuan Pembelajaran

- ☐ Menjelaskan apa itu TSIX dan apa bedanya dengan VM/emulator
- ☐ Menyebutkan tiga prinsip inti arsitektur TSIX
- ☐ Menjelaskan alasan kernel, driver, dan FS disatukan dalam satu thread
- ☐ Mengenali lapisan Ring 0-4 dan isinya secara garis besar
- ☐ Menjelaskan mengapa kernel tidak pernah menjalankan aplikasi

Konsep Inti

Apa itu TSIX?

TSIX adalah **sistem operasi simulasi berbasis Node.js + TypeScript**. Ia bukan VM yang mengemulasi CPU. Ia membangun **abstraksi OS di atas runtime Node.js yang sudah ada** — memakai `Worker Thread` sebagai batas proses, `postMessage` sebagai IPC, dan `SQLite` sebagai filesystem.



Dengan pendekatan ini, TSIX mengadopsi **konsep arsitektur UNIX** — process isolation, permission UID/GID, filesystem POSIX, signal handling, device abstraction (HAL), syscall communication — diekspresikan dalam TypeScript di atas Node.js.

Kenapa bukan microkernel?

Salah satu keputusan arsitektural paling fundamental: **kernel, driver, dan filesystem disatukan dalam satu thread utama**, bukan dipisah ke thread masing-masing seperti microkernel murni (Minix, QNX, seL4).

Alasan	Penjelasan
Overhead IPC	Microkernel murni = 3-4x <code>postMessage</code> per operasi (Apl → Kernel → FS → Kernel → Apl). Satu thread = 1x (Apl → Kernel). Setiap <code>postMessage</code> melakukan structured clone (serialize → deserialize).
Keterbatasan Worker	Worker Thread Node.js tidak bisa shared memory . Setiap transfer = alokasi baru + copy data. Makin besar data, makin besar overhead.

Alasan	Penjelasan
Target embedded	TSIX untuk IoT dengan CPU terbatas dan RAM kecil. Setiap Worker tambahan = ~4-8MB heap.
Isolasi sudah ada	Aplikasi (Ring 4) sudah di worker terpisah → 90% manfaat isolasi dengan 10% biaya.

Hasilnya: yang paling sering crash adalah aplikasi user — dan itu sudah terisolasi. Kernel + driver + FS jarang crash, dan walaupun crash, sistem mati (tapi jarang).

Empat Prinsip Inti

1. "Syscall = satu-satunya pintu"

Worker thread **tidak pernah menyentuh resource langsung**. Bahkan `print` pun melalui syscall. Ini menjaga batas ring benar-benar berarti — tidak ada jalan pintas.

```
App → UserLib.dispatch(code, args) → postMessage → Kernel → dispatch → kembali
```

2. "Everything is a File"

File dan device sama-sama `IDevice`; `read/write` polimorfik.

- Buka file biasa → `FileSystemDevice`
- Buka `/dev/tty1` → `TTYDevice`
- Buka `/dev/smqtntl0` → `SimpleMQTNLDriver`

Bahkan pipe dan socket adalah device. Satu kontrak, banyak implementasi.

3. "Direct Memory Execution" (DME)

Framework (`/lib`) di-pre-compile ke memori saat boot, dikirim ke worker via `workerData`, dieksekusi **tanpa hit filesystem**. `@tsix/*` terasa instan — tidak ada kompilasi ulang per proses.

4. "Fidelity Unix dulu, pragmatis belakangan"

TSIX meniru perilaku & arsitektur Unix/Linux sedekat mungkin — semantik syscall, permission UID/GID, kredensial (saved UID), format file (`/etc/shadow`, `passwd`), hierarchy filesystem — sebagai **north star** desain. Bukan karena "harus persis", tapi karena perilaku yang **teramati** harus konsisten: non-root tidak bisa baca `/etc/shadow`, proses non-root tidak bisa `setuid` sembarangan, dan seterusnya.

Penyimpangan dari Unix **boleh**, tapi hanya jika runtime Node.js/V8 memang tidak mampu — bukan karena alasan "lebih gampang". Setiap penyimpangan **wajib dicatat** (di changelog dan/atau komentar kode) dengan alasan teknis yang jelas, supaya tidak disalahartikan sebagai bug.

[!NOTE] **Semantik > Mekanisme** — kita tidak perlu mengemulasi bare-metal (hardware interrupt, MMU, dll). Yang penting perilaku yang teramati dari userland sama dengan Unix. Contoh: `setuid` disimulasikan dengan saved UID (`pcb.suid`) di kernel, bukan dengan register CPU — tapi efeknya sama.

Kode Sumber

File	Peran
<code>src/main.ts</code>	Entry point host + keep-alive
<code>src/kernel/Kernel.ts</code>	Orkestrator boot semua subsistem
<code>src/kernel/Syscalls.ts</code>	Dispatcher syscall

File	Peran
<code>src/kernel/Scheduler.ts</code>	Manajemen proses
<code>src/userland/WorkerEntry.ts</code>	Bootloader worker + sandbox
<code>src/common/IPCTypes.ts</code>	Kontrak IPC

Latihan / Praktik

1. Baca `src/main.ts` — temukan di mana keep-alive 100ms berada. Apa yang terjadi jika PID 1 exit dengan code 1?
2. Baca `src/kernel/Kernel.ts` — daftar urutan `initializeSubsystems()`. Cocokkan dengan diagram boot di Modul 03.
3. Jalankan TSIX (`npm start` / sesuai `README.md`) dan amati log boot.

Referensi

- `wiki/Arsitektur-Sistem.md` — diagram lapisan dan analisis microkernel
- `wiki/course/00-overview.md §1` — peta mental global
- `src/main.ts`, `src/kernel/Kernel.ts`

Modul 01 — selesai. Lanjut ke [Modul 02 — Model Ring & Batas Privilege](#).

RFC-TSIX-EDU-002 | Modul kedua kurikulum TSIX. Memahami konsep "Ring" sebagai batas privilege, dan di mana batas keamanan *sebenarnya* berada di TSIX.

Penting: **Ring di TSIX adalah konsep (dokumentasi), bukan mekanisme isolasi hardware.** TSIX berjalan di atas V8, bukan bare metal. Batas nyata ada di dua lapisan: **sandbox WorkerEntry** dan **PermissionManager (kernel)**.

Tujuan Pembelajaran

- ☐ Menjelaskan isi tiap Ring 0–4 dan file utamanya
- ☐ Membedakan Ring sebagai konsep vs mekanisme isolasi nyata
- ☐ Menjelaskan dua lapisan batas privilege nyata: WorkerEntry sandbox & PermissionManager
- ☐ Menjelaskan apa itu setuid bit dan contoh penggunaannya
- ☐ Mengetahui kelemahan privilege berbasis nama app

Konsep Inti

Peta Ring

Ring	Isi	File utama
0	Host: Linux + Node/V8	— (reserved)
1	Kernel core: Scheduler, Syscall, Permission	<code>src/kernel/*</code> , <code>src/common/*</code>
2	Driver & FS: HAL devices, VFS backends, MountManager	<code>src/kernel/devices/*</code> , <code>src/vfs/*</code>
3	Library framework: UserLib, Application	<code>src/mirror/lib/*</code>
4	Aplikasi: <code>/bin/*</code> , <code>init</code> , <code>tsh</code> , <code>daemon</code>	<code>src/mirror/bin/*</code>

Ring 1 & 2 — kernel internals (dispatcher, scheduler, stack FS/Net/HAL) Sumber: [wiki/diagram/Arsitektur-Sistem-2.mmd](#)

Ring adalah **peta tanggung jawab** — bukan isolasi yang ditegakkan oleh V8. Dua file yang berbeda ring boleh saja saling memanggil; yang menjaga keamanan adalah aturan di bawah.

Perbandingan dengan Linux

Fitur	Linux Equivalent	TSIX Ring
Core OS Logic	Ring 0 (Kernel Mode)	Ring 1
Drivers & FS	Ring 0 (Kernel Mode)	Ring 2
Standard Library	Ring 3 (glibc)	Ring 3
User Apps	Ring 3 (User Mode)	Ring 4

[!NOTE] **Ring 0 adalah domain host.** Karena TSIX bukan bare metal, "Ring 0" berarti Linux/Windows + V8. TSIX memulai penomoran dari Ring 1.

Dua Lapisan Batas Privilege Nyata

Lapisan 1 — WorkerEntry Sandbox (sisi worker)

`src/userland/WorkerEntry.ts` adalah **bootloader** setiap worker. Ia mengunci pintu sebelum aplikasi berjalan:

- App hanya boleh `require` framework `@tsix/*` / `@common/*` — sisanya diblokir.
- `process.exit` / `process.kill` disabotase (di-throw).
- App **privileged** (nama mengandung `server`, `daemon`, `dome`, `tbuild`, `vfs`) mendapat akses **allow-list** modul host: `http`, `ws`, `path`, `fs`, `url`, `esbuild`, `crypto`, `os`, `bcryptjs`.

Lapisan 2 — PermissionManager (sisi kernel)

`src/kernel/PermissionManager.ts` adalah "satpam" di kernel. Ia melakukan cek rwx berlapis:

```
check(pid, path, mode):
  root (uid 0) → bypass
  owner       → cek bit owner
  group       → cek bit group
  others      → cek bit others
```

SetUID bit (`0o4000`) didukung: proses dieksekusi dengan hak milik file. Contoh: `/bin/login` memiliki mode `0o4755` — siapa pun yang menjalankan login, prosesnya berjalan sebagai root (untuk melakukan `setuid/setgid` ke user target).

Kode Sumber

File	Peran
<code>src/userland/WorkerEntry.ts</code>	Sandbox sisi worker (restrictHostAPI)
<code>src/kernel/PermissionManager.ts</code>	Cek rwx + setuid
<code>src/kernel/Syscalls.ts</code>	Panggil permission check sebelum aksi
<code>src/mirror/bin/login.ts</code>	Contoh penggunaan setuid (<code>0o4755</code>)

Snippet (level kode)

```
// src/kernel/PermissionManager.ts — inti cek izin (ringkas)
check(pid: number, path: string, mode: number): boolean {
  const { uid, gid } = this.scheduler.getProcess(pid);
  const stat = this.getStat(path);
  if (uid === 0) return true; // root bypass
  if (uid === stat.uid) return !(stat.mode & mode); // owner
  if (gid === stat.gid) return !(stat.mode & (mode >> 3)); // group
  return !(stat.mode & (mode >> 6)); // others
}
```

[!WARNING] **Kelemahan yang diketahui.** Status privileged berbasis **substring nama app** — heuristik rapuh. Siapa pun yang menamai app-nya `my-daemon` otomatis privileged. Idealnya diganti dengan *capability-based*.

Latihan / Praktik

1. Baca `src/userland/WorkerEntry.ts` — temukan daftar allow-list modul host.
2. Baca `src/kernel/PermissionManager.ts` — uji logika cek rwx dengan beberapa kombinasi uid/gid/mode.
3. Jalankan app non-privileged yang mencoba `require("fs")` — amati error yang muncul.

Referensi

- [wiki/ARCHITECTURE_RINGS.md](#) — definisi resmi ring
 - [wiki/Keamanan-dan-Sandboxing.md](#) — detail sandbox
 - [wiki/course/00-overview.md](#) §2
-

Modul 02 — selesai. Lanjut ke [Modul 03 — Boot Sequence](#).

RFC-TSIX-EDU-002 | Modul ketiga kurikulum TSIX. Menelusuri urutan boot dari `main.ts` → `kernel.boot()` → PID 1 (`init`) → login, sampai keep-alive 100ms di host.

Boot TSIX meniru boot UNIX/Linux: dari bootstrap host → kernel init subsistem → `init` (PID 1) → `rc.local` → `getty/login` per TTY. Yang menentukan sistem "hidup atau mati" adalah **keep-alive 100ms** di `main.ts`.

Tujuan Pembelajaran

- ☐ Menjelaskan peran `main.ts` dan keep-alive 100ms
- ☐ Menyebutkan urutan `initializeSubsystems()` di kernel
- ☐ Menjelaskan isi dan peran `init` (PID 1)
- ☐ Menjelaskan urutan daemon di `rc.local`
- ☐ Menjelaskan apa arti exit code 1 saat PID 1 mati (reboot)

Alur / Cara Kerja

Boot TSIX mengikuti pola UNIX/Linux klasik: bootstrap host → kernel init subsistem → `init` (PID 1) di Worker → `rc.local` → `getty/login` per TTY → keep-alive host. Diagram di bawah merangkum seluruh urutan, dari detik pertama host dijalankan sampai sistem "hidup".

```
sequenceDiagram
    participant H as Host (Node.js)
    participant M as main.ts
    participant K as Kernel.ts
    participant V as VFS (BKFS/SQLite)
    participant W as Worker Thread (Ring 4)
    participant I as init (PID 1)
    participant R as rc.local
    participant L as login (TTY1-6)

    H->>M: node src/main.ts
    M->>M: Config.load() → sysconfig.json
    M->>K: new Kernel() (Logger, PermissionManager, MountManager, GUIRegistry)
    M->>K: await kernel.boot()

    activate K
    K->>K: initializeSubsystems()
    K->>V: mount BKFS "/" (system.db)
    K->>K: processFstab() → /tmp (RamFS), /mnt/shared, /mnt/sbak
    K->>K: new Scheduler() + new PermissionManager() + new PortManager()
    K->>K: new SyscallDispatcher() → scheduler.setSyscallHandler(...)
    K->>K: rebuildVFSCache() → pre-compile /lib (DME)
    K->>K: TTYManager(32) + /dev devices (stdin, fb0, stdout, stderr, null, tty1-32)
    K->>K: network interfaces (SimpleMQTNLDriver) + loadAuxDevices() + SerialDeviceManager
    K->>K: applyDeviceConfigs() (udev) + pastikan /dev di VFS
    K->>K: load RSA identity → fingerprint visual
    K->>K: ensureDefaultAuth() + ensureDefaultGroups()
    K-->>M: boot() selesai

    M->>K: kernel.runInit()
    K->>V: resolve /bin/init.js (cfg.scheduler.bootEntry)
    K->>W: createProcess("init", fds:[tty1x3]) → PID 1
    K->>K: setForegroundProcess(1, 1)
    deactivate K

    activate W
    activate I
    I->>I: enforce setuid (/bin/passwd.js, /bin/sudo.js)
    I->>I: RSA identity: generate/verify keypair + fingerprint
```

```

I->>R: exec /etc/rc.local.js + waitpid
activate R
R->>R: airtermd → tpkgd → scpd → otad → iot-listener
R->>R: tunggu /var/run/dome.ready → dome → asteracea
R->>R: crond → mysqld
R->>I: exit 0
deactivate R

I->>L: spawnLogin TTY2-6 (monitorProcess → respawn)
I->>L: spawnLogin TTY1 (foreground)
I->>I: while(true) { wait 10s } ← PID 1 tak pernah exit
deactivate I
deactivate W

M->>M: keepAlive: setInterval(100ms)
M->>M: scheduler.getProcess(1) → EXITED?
M->>M: exitCode === 1 → reboot | else → power off

```

[!NOTE] Versi mermaid ini lebih lengkap daripada `wiki/boot_sequence.md` dan mengikuti kode terkini (`Kernel.ts`, `init.ts`, `rc.local.ts`).

Alur ringkas (ASCII)

```

main.ts
Config.load()           → baca sysconfig.json
new Kernel()            → Logger, PermissionManager, MountManager, GUIRegistry
kernel.boot()
  initializeSubsystems()
    mount BKFS("/")      → SQLite system.db (root filesystem)
    processFstab()       → /tmp (RamFS), /mnt/shared (HostVFS), /mnt/sbak (BKFS)
    new Scheduler()
    new PermissionManager()
    new PortManager()
    new SyscallDispatcher() → scheduler.setSyscallHandler(...)
    rebuildVFSCache()    → pre-compile /lib ke memori (DME)
  new TTYManager(32) + TTYDevice tty1..32
  devices{}              → stdin, fb0, stdout, stderr, null
  init network interfaces → SimpleMQTNLDriver per config
  loadAuxDevices()       → plugin driver (random, mysql, mcp23017)
  SerialDeviceManager    → auto-detect ttyUSB*
  applyDeviceConfigs()    → "udev": mode/uid/gid dari sysconfig
  pastikan /dev ada di VFS
  load RSA identity       → fingerprint visual di TTY
  ensureDefaultAuth()     → seed /etc/passwd + /etc/shadow (root)
  ensureDefaultGroups()   → seed users (100) + sudo (27)
  wire keyboard + TTY callbacks → Ctrl+C → SIGINT fg; Alt+F1-6 → switch
kernel.runInit()
  resolve /bin/init.js
  createProcess("init", fds:[tty1×3]) → PID 1
  setForegroundProcess(1, 1)
[init.ts di Worker]
  enforce setuid (passwd, sudo)
  generate/verify RSA identity
  exec /etc/rc.local.js + waitpid
  spawn login TTY2..6 (monitorProcess → respawn)
  spawn login TTY1 (foreground) → loop forever
  keepAlive (100ms di main.ts)
  jika PID 1 EXITED → process.exit(exitCode) (1 = reboot)

```

Urutan `initializeSubsystems()` (dari `Kernel.ts`)

```

this.bootLogStart("VFS: Mounting root filesystem (BKFS/SQLite)");
this.bkfs = new BKFS(cfg.kernel.database);
this.mountManager.mount("/", this.bkfs, "bkfs", cfg.kernel.database, false);
this.bootLogEnd(true);

await this.processFstab();           // /tmp (RamFS), /mnt/* (HostVFS/BKFS)

```

```
this.scheduler = new Scheduler();           // Core: Process Scheduler
this.satpam = new PermissionManager();      // Security: Permission Manager
this.portManager = new PortManager();      // Network: Port Manager
this.syscall = new SyscallDispatcher(
  this.bkfs, this.mountManager, this.scheduler, this, this.satpam,
);
this.scheduler.setSyscallHandler(async (req) => {
  return await this.syscall!.handleRequest(req);
});
this.rebuildVFSCache();                    // VFS: pre-compile /lib (DME)
this.scheduler.setVFSCacheProvider(() => this.vfsCache);
```

Urutan rc.local (daemon, dari rc.local.ts)

- 1. **SetUID** /bin/login.js → 0o4755, chown root:root.
- 2. airtermd (/sbin/airtermd.js) — remote access.
- 3. tpkgd (/sbin/tpkgd.js) — package server (TPKG).
- 4. scpd (/sbin/scpd.js) — transfer file SCP.
- 5. otad (/sbin/otad.js) — OTA firmware ESP.
- 6. iot-listener (/sbin/iot-listener.js) — sensor MQTNL.
- 7. Bersihkan marker lama /var/run/dome.ready, lalu **tunggu DOME siap** (polling 200ms, timeout 10s).
- 8. dome (/opt/dome/dome.js) — display server (PixelSpace).
- 9. asteracea (/opt/asteracea/asteracea.js) — window manager (setelah DOME siap).
- 10. crond (/sbin/crond.js) — cron scheduler.
- 11. mysqld (/opt/mysqld/mysqld.js) — database service.

[!WARNING] **Dokumentasi vs kode.** Sumber rc.local.ts memuat 11 langkah di atas. Namun file hasil compile rc.local.js (yang benar-benar dieksekusi) hanya memuat 3 daemon: airtermd, tpkgd, scpd. **Kode adalah kebenaran** — saat runtime, hanya 3 daemon pertama yang benar-benar hidup dari rc.local.

Kode Sumber

File	Peran
src/main.ts	Entry point host + keep-alive
src/kernel/Kernel.ts	boot() + initializeSubsystems() + runInit()
src/mirror/bin/init.ts	PID 1: setuid, identity, rc.local, spawn login
src/mirror/etc/rc.local.ts	Daftar daemon startup
src/common/Config.ts	Baca sysconfig.json

Snippet (level kode)

Keep-alive di main.ts (host tidak boleh mati)

```
const keepAlive = setInterval(() => {
  const scheduler = kernel.getScheduler();
  const p1 = scheduler?.getProcess(1);
  if (!p1 || p1.state === "EXITED") {
    clearInterval(keepAlive);
    if (process.stdin.isTTY) {
      (process.stdin as any).setRawMode(false);
    }
  }

  const exitCode = (kernel as any).wantedExitCode ?? (process.exitCode ?? 0);
  if (exitCode === 1) {
```

```

        console.log("\n[Kernel] System is rebooting...");
    } else {
        console.log("\n[Kernel] System halted. Powering off...");
    }
    process.exit(exitCode);
}
}, 100); // 100ms biar satset responnya

```

Kunci: **selama PID 1 hidup, host tidak berhenti**. PID 1 exit dengan **code 1** → reboot; selain itu → power off. Sebelum exit, raw mode terminal dipulihkan supaya terminal host tidak "rusak".

Log boot: bootLogStart() + bootLogEnd()

Kernel.ts mencetak progres boot dengan bracket [] yang di-update in-place (\r + ANSI \x1b[K). Pola pemakaiannya:

```

this.bootLogStart("VFS: Mounting root filesystem (BKFS/SQLite)");
this.bkfs = new BKFS(cfg.kernel.database);
this.mountManager.mount("/", this.bkfs, "bkfs", cfg.kernel.database, false);
this.bootLogEnd(true); // → [ OK ] VFS: Mounting root...

```

bootLogEnd(false, "pesan") → [FAIL]. Keduanya no-op jika cfg.kernel.verbose false.

runInit: spawn PID 1

```

const initPath = "/bin/" + cfg.scheduler.bootEntry; // bootEntry = "init.js"
const res = this.mountManager.resolve(initPath);
const raw = res.vfs.read(res.relativePath); // kontrak IVFS, bukan stat()

const initPcb = this.scheduler?.createProcess("init", {
  fds: [tty1, tty1, tty1],
  appName: "init",
  appContent: initContent,
  env: {
    PATH: cfg.scheduler.defaultPath,
    HOME: "/root",
    HOSTNAME: cfg.shell.defaultHostname,
    PROMPT_FORMAT: cfg.shell.promptFormat,
    LINES: (process.stdout.rows || cfg.shell.defaultRows).toString(),
    COLUMNS: (process.stdout.columns || cfg.shell.defaultColumns).toString(),
  },
  cwd: cfg.scheduler.defaultCwd,
  ttyId: 1,
});
if (initPcb) this.scheduler?.setForegroundProcess(initPcb.pid, 1);

```

init: enforce setuid

```

// Runtime mengeksekusi sidcar .js (bukan source .ts), jadi chmod harus di .js
await lib.fs.chmod("/bin/passwd.js", 2541); // 2541 = 0o4755 (setuid root)
await lib.std.print(`${ok} [INIT] SetUID bit applied to: /bin/passwd.js\n`);
await lib.fs.chmod("/bin/sudo.js", 2541);
await lib.std.print(`${ok} [INIT] SetUID bit applied to: /bin/sudo.js\n`);
await lib.std.print(`${ok} [INIT] System binary permissions enforced.\n`);

```

[!NOTE] 2541 desimal = 0o4755 oktal (setuid + rwxr-xr-x). SetUID ini membuat passwd dan sudo naik privilege ke root walau dijalankan user biasa.

init: exec rc.local + waitpid

```

const result = await lib.shell.exec("/etc/rc.local.js", [], undefined, undefined, undefined);
if (result) {
  const exitCode = await lib.shell.waitpid(result.pid);
}

```

```

if (exitCode === 0) {
  await lib.std.print(`${ok} [INIT] Startup scripts completed successfully.\n`);
} else {
  await lib.std.print(`Init: Warning - rc.local exited with code ${exitCode}\n`);
}
}

```

init: spawnLogin + monitorProcess (respawn anti-crash-loop)

```

const spawnLogin = async (ttyId: number) => {
  try {
    if (ttyId > 1)
      await lib.std.print(`${ok} Initializing session on TTY${ttyId}...\n`);
    const result = await lib.shell.exec("/bin/login.js", [], undefined, undefined, ttyId);
    if (result && result.pid) {
      terminals.set(ttyId, result.pid);
      await lib.std.log(`Login service spawned on TTY${ttyId} (PID ${result.pid})`, "init");
      // Kita nungguin di background (thread terpisah di Worker)
      this.monitorProcess(lib, ttyId, result.pid, spawnLogin);
    }
  } catch (e: any) {
    await lib.std.print(`Init: Error spawning login on TTY${ttyId}: ${e.message}\n`);
  }
};

for (let i = 2; i <= 6; i++) await spawnLogin(i); // TTY2-6 di background
await spawnLogin(1); // TTY1 foreground

```

`monitorProcess` menunggu `waitpid`; jika login mati, ia menunggu 1 detik lalu `respawn` — mencegah crash-loop spam:

```

private async monitorProcess(
  lib: UserLib, ttyId: number, pid: number, respawn: (tty: number) => Promise<void>,
) {
  const exitCode = await lib.shell.waitpid(pid);
  await lib.std.print(
    `Init: Process on TTY${ttyId} (PID ${pid}) exited with code ${exitCode}. Respawning...\n`,
  );
  await new Promise(r => setTimeout(r, 1000)); // delay anti crash-loop
  await respawn(ttyId);
}

```

init: loop abadi (PID 1 tidak boleh exit)

```

// Init process must never exit
while (true) {
  await new Promise(r => setTimeout(r, 10000));
}

```

rc.local: tunggu DOME siap (marker, bukan sleep tetap)

```

const DOME_READY_MARKER = "/var/run/dome.ready";
const DOME_WAIT_MS = 10000;
const DOME_POLL_MS = 200;
let domeReady = false;
const waitStart = Date.now();
while (Date.now() - waitStart < DOME_WAIT_MS) {
  try {
    if (await lib.fs.stat(DOME_READY_MARKER)) { domeReady = true; break; }
  } catch (_) { /* marker belum ada */ }
  await new Promise(r => setTimeout(r, DOME_POLL_MS));
}

```

Asteracea hanya di-start setelah marker terlihat — kalau tidak, `CREATE_WINDOW` ditolak kernel ("GUI_REQ: DOME engine is not running") dan layar blank.

ensureDefaultAuth / ensureDefaultGroups (seeding identitas)

Di akhir `boot()`, kernel menjamin file identitas ada sebelum userland jalan:

```
// ensureDefaultAuth() — seed bila belum ada
const rootPasswd = "root:x:0:0:root:/root:/bin/tsh.ts\n"; // /etc/passwd
const rootShadow = "root:$2b$10$...KupG:19750:0:99999:7:::\n"; // /etc/shadow (bcrypt "root")

// ensureDefaultGroups() — group wajib
missing.push("users:x:100:"); // GID 100
missing.push("sudo:x:27:"); // GID 27 (gaya Ubuntu)
```

Latihan / Praktik

1. Jalankan `npm start` — catat urutan log boot ([OK] / [FAIL]) dan bandingkan dengan diagram mermaid di atas.
2. Hentikan proses `init` dari dalam sistem (`kill 1` sebagai root) — apa yang terjadi pada host? (keep-alive 100ms → exit code → reboot/halt).
3. Baca `src/mirror/bin/init.ts` — di mana `enforce setuid` dan RSA identity dipanggil? Berapa TTY login yang di-spawn?
4. Baca `src/mirror/etc/rc.local.ts` — cocokkan 11 langkah daemon di atas dengan kode. Bandingkan dengan `rc.local.js` (hanya 3 daemon) — mengapa berbeda?
5. Baca `src/kernel/Kernel.ts` — temukan urutan `initializeSubsystems()` dan `rebuildVFSCache()`; jelaskan peran DME (Direct Memory Execution).

Referensi

- `wiki/boot_sequence.md` — diagram sequence mermaid
- `wiki/course/00-overview.md §3` — Boot Sequence
- `src/main.ts` — entry point host + keep-alive
- `src/kernel/Kernel.ts` — boot, `initializeSubsystems`, `runInit`, `ensureDefaultAuth`
- `src/mirror/bin/init.ts` — PID 1 (setuid, identity, rc.local, spawn login)
- `src/mirror/etc/rc.local.ts` — daemon startup
- `src/common/Config.ts` — baca `sysconfig.json`

Modul 03 — selesai. Lanjut ke [Modul 04 — Proses & Scheduler](#).

RFC-TSIX-EDU-002 | Modul keempat kurikulum TSIX. Memahami siklus hidup proses di TSIX: PCB, spawn → run → block → exit, zombie, orphan reparent, daemonize, dan reexec.

Satu worker thread = satu proses. Multitasking TSIX adalah **concurrency Node.js**, bukan preemption — kernel tidak menghentikan proses secara paksa. Yang diatur kernel adalah **siklus hidup dan sinyal**.

Tujuan Pembelajaran

- ☐ Menyebutkan field utama PCB dan membedakan state enum vs state transien
- ☐ Menjelaskan lifecycle: READY → RUNNING → BLOCKED → EXITED (termasuk zombie, orphan, reexec)
- ☐ Menjelaskan waitpid + reap (zombie) dan reparent orphan ke PID 1
- ☐ Menjelaskan daemonize (DETACH) dan REEXEC
- ☐ Menjelaskan model sinyal (SIGKILL, SIGINT, SIGTERM, SIGSTOP/CONT) dan cara deliver-nya
- ☐ Menelusuri alur nyata: spawn → waitpid → exit → reap

Konsep Inti

PCB (Process Control Block)

Setiap proses TSIX adalah satu **worker thread**. Kernel (main thread) tidak pernah menjalankan kode aplikasi — ia hanya mengelola metadata proses lewat **PCB** (interface PCB di `src/kernel/Scheduler.ts`). Satu PCB = satu proses.

Field PCB

Field	Type	Makna
pid	number	ID unik proses; dialokasikan dari nextPid++
ppid?	number	Parent PID — membentuk process tree
name	string	Nama proses (mis. "Shell", "init")
state	ProcessState	READY / RUNNING / BLOCKED / EXITED
pc	number	Program counter — alamat instruksi berikutnya
owner	string	Nama user pemilik (mis. "root")
uid	number	User ID (effective)
gid	number	Group ID (effective)
ruid	number	Real user ID — siapa yang menjalankan aslinya
groups	number[]	Supplementary groups
cwd	string	Current working directory
worker?	Worker	Worker thread — raga proses (ada setelah spawnWorker)
fdTable	(FDEntry null)[]	Tabel file descriptor; 0=stdin, 1=stdout, 2=stderr
env	Record<string, string>	Environment variables
exitCode?	number	Exit code eksplisit dari syscall EXIT
ttyId?	number	Virtual console (TTY) tempat proses berjalan
uuid?	string	Identitas aplikasi — persisten antar PID (untuk SEND_MSG by identity)


```
export interface PCB {
  pid: number;           // ID Unik Proses
  ppid?: number;         // Parent PID – buat process tree
  name: string;          // Nama Proses (misal: "Shell")
  state: ProcessState;   // Status Saat Ini
  pc: number;            // Program Counter (Alamat instruksi berikutnya)

  owner: string;         // User yang menjalankan (misal: "root")
  uid: number;           // User ID (Effective)
  gid: number;           // Group ID (Effective)
  ruid: number;          // Real User ID (Siapa yang aslinya nge-jalanin)
  groups: number[];      // Supplementary Groups
  cwd: string;           // Current Working Directory (Posisi sekarang)
  worker?: Worker;       // Raga asli proses di thread terpisah

  // FD TABLE sekarang berisi object FDEntry yang menunjuk ke Driver nyata.
  fdTable: (FDEntry | null)[];

  // Environment Variables
  env: Record<string, string>;
  exitCode?: number;     // Explicit exit code set by Syscall
  ttyId?: number;        // ID Virtual Console (TTY) tempat proses ini berjalan
  uuid?: string;         // Unique Application Identity (Persistent across PID changes)
}
```

[!NOTE] **State** di TSIX hanya empat nilai enum `ProcessState: READY, RUNNING, BLOCKED, EXITED`. Tidak ada state `NEW` — PCB sudah terbentuk penuh saat `createProcess()` mengembalikannya. Nilai `REEXECING` **bukan** bagian enum; ia state transien yang di-set lewat `(pcb as any).state = "REEXECING"` untuk mencegah hook cleanup saat `reexec()`.

Setiap entri `fdTable` adalah `FDEntry`:

```
export interface FDEntry {
  device: IDevice;       // Driver nyata (FileSystemDevice, TTYDevice, dll)
  context: any;          // Konteks driver (path, offset, ...)
  flags?: string;        // "r" | "w" | ...
}
```

Lifecycle

```
stateDiagram-v2
  [*] --> READY : createProcess()\nnpid = nextPid++ → PCB
  READY --> RUNNING : spawnWorker()\nWorker Thread dibuat
  RUNNING --> BLOCKED : SIGSTOP(19) / menunggu event / IPC
  BLOCKED --> RUNNING : SIGCONT(18) / event selesai
  RUNNING --> EXITED : worker.on("exit")
  EXITED --> ZOMBIE : tidak ada waiter waitpid
  ZOMBIE --> [*] : waitpid() → reap()
  RUNNING --> REEXECING : reexec()\nterminate worker lama
  REEXECING --> READY : PCB sama, worker baru (spawnWorker)
  note right of EXITED
    orphan: semua child (ppid === pid)
    di-reparent ke init (PID 1)
  end note
```

[!NOTE] `course-server.ts` belum men-render mermaid (tampil sebagai blok kode); versi teks: `READY → RUNNING → BLOCKED → EXITED`, di mana `EXITED` tanpa waiter menjadi **zombie** (tetap di tabel sampai `waitpid + reap`).

Peristiwa	Mekanisme
Spawn	<code>createProcess()</code> → <code>nextPid++</code> → PCB (<code>READY</code>) → <code>spawnWorker()</code> bila ada <code>appName/appPath</code> → state <code>RUNNING</code>

Peristiwa	Mekanisme
Exit	Syscall <code>EXIT(code)</code> → set <code>pcb.exitCode</code> → <code>cleanupProcess()</code> (tutup FD, lepas port) → <code>kill(pid)</code> → event worker "exit" → <code>EXITED</code>
Orphan	Handler <code>worker.on("exit")</code> → semua child (<code>ppid === pid</code> , belum <code>EXITED</code>) di-set <code>ppid = 1</code>
Zombie	Proses <code>EXITED</code> tanpa waiter → tetap di tabel sampai <code>waitpid + reap()</code>
waitpid	Sudah <code>EXITED</code> → <code>reap()</code> + return <code>exitCode</code> ; belum → daftar di <code>waitQueue</code>
daemonize	<code>DETACH</code> → FD 0/1/2 dialihkan ke <code>/dev/null</code> , resolve waiters, lepas foreground TTY, kosongkan <code>ttyId</code>
REEXEC	Set <code>(pcb as any).state = "REEXECING"</code> → terminate worker lama → PCB sama → <code>spawnWorker()</code> baru
REARENT (syscall)	<code>REARENT { pid, newPpid }</code> → <code>childPcb.ppid = newPpid</code> (manual; verifikasi parent ada atau PID 1)

Walkthrough: spawn → waitpid → exit → reap

Ikuti alur nyata satu proses `sleep 2` yang diluncurkan dari shell (`tsh`):

- Spawn.** `tsh` (mis. PID 3) memanggil syscall `EXEC` → kernel memanggil `createProcess("sleep", { appPath: "/bin/sleep.js", ppid: 3, fds: [...], ttyId: 1 })`. PCB dibuat state `READY`, lalu `spawnWorker()` membuat Worker Thread dan men-set state `RUNNING`. Proses mendapat `pid = 42`.
- waitpid.** `tsh` memanggil `WAITPID(42)` → `waitpid(42)`. Proses belum selesai → `setForegroundProcess(42, 1)` (biar Ctrl+C masuk ke `sleep`), lalu resolver didaftarkan di `waitQueue[42]`. `waitpid` mengembalikan Promise.
- Exit.** `sleep` selesai → mengirim syscall `EXIT(0)` → kernel set `pcb.exitCode = 0` → `cleanupProcess(42)` menutup semua FD & melepas port → `kill(42)` (default `SIGKILL`) → `worker.terminate()`.
- Reap.** Node memicu event worker "exit" → state `EXITED` → `finalCode = 0` → `onProcessExitCallback` (`cleanup global`) → reparent orphan (tidak ada) → `waitQueue[42]` punya waiter → `resolve(0)` (shell menerima exit code 0) → `reap(42)` menghapus PCB dari tabel. Tidak jadi zombie. Foreground TTY dikembalikan ke `null/shell`.
- Zombie (anti-pola).** Jika shell **tidak** memanggil `waitpid`, handler exit mencatat log `became a ZOMBIE` dan PCB tetap berada di tabel. PCB baru dihapus nanti saat `waitpid(42)` dipanggil (`reap`).

Sinyal

Sinyal dikirim lewat `scheduler.kill(pid, signal)` — satu-satunya gerbang. Ada dua syscall yang memakainya:

- `KILL(pid)` → selalu **SIGKILL (9)**.
- `SIGNAL({ pid, sig })` → sinyal bebas sesuai `sig`.

Keduanya melakukan cek yang sama: proses ada, **PID 1 diproteksi**, dan permission (root atau pemilik proses).

Sinyal	Nilai	Efek
SIGKILL	9	Hard kill: <code>worker.terminate()</code> langsung, tanpa event
SIGINT	2	Graceful: push event <code>SIGINT</code> , grace 100ms → terminate jika tak ditangani. Default exit 130
SIGTERM	15	Graceful: push event, grace 300ms → terminate. Default exit 143. Dipakai SHUTDOWN
SIGSTOP	19	Soft: <code>pcb.state = BLOCKED</code> + event
SIGCONT	18	Soft: <code>pcb.state = RUNNING</code> + event
SIGSEGV	—	Dikirim kernel saat proses melanggar kepemilikan window GUI
SIGHUP (1), SIGUSR1 (10), SIGUSR2 (12), dll	—	Push event generik <code>sendEvent(pid, "signal", name)</code>

 Ctrl+C → SIGINT ke foreground process TTY aktif *Sumber:* [wiki/diagram/Kernel-dan-Scheduler-3.mmd](https://wiki.diagram/kernel-dan-scheduler-3.mmd)

[!IMPORTANT] **PID 1 (init) diproteksi** — baik KILL maupun SIGNAL menolak target PID 1, bahkan untuk root. Satu-satunya jalan terminasi sistem adalah syscall SHUTDOWN / REBOOT .

SHUTDOWN bertingkat (root saja, args = 0 shutdown / 1 reboot):

1. broadcastEvent("signal", "SIGTERM") ke semua proses (kecuali diri sendiri & PID 1).
2. Tunggu grace sampai 5 s (polling 100 ms) — semua proses EXITED → sukses.
3. Sisa yang tidak merespons → kill(pid, 9) (SIGKILL).
4. Flush network 1 s (biar paket terakhir seperti !exit! terkirim).
5. kill(1, 9, exitCode) → PID 1 di-terminate; main.ts keepAlive membaca exit code (1 = reboot).

Kode Sumber

File	Peran
src/kernel/Scheduler.ts	PCB, lifecycle, waitpid, detach, sinyal, reexec
src/kernel/Syscalls.ts	Syscall KILL, WAITPID, EXEC, EXIT, DETACH, REEXEC
src/common/IPCTypes.ts	Kontrak event sinyal

Snippet (level kode)

Semua snippet di bawah ini disalin dari src/kernel/Scheduler.ts dan src/kernel/Syscalls.ts (kode adalah kebenaran).

createProcess (spawn)

```
public createProcess(name: string, options: SpawnOptions = {}): PCB | null {
  const pcb: PCB = {
    pid: this.nextPid++,
    name: name,
    state: ProcessState.READY,
    pc: 0,
    owner: options.owner || "root",
    uid: options.uid ?? 0,
    gid: options.gid ?? 0,
    ruid: options.ruid ?? (options.uid ?? 0),
    groups: options.groups ?? [],
    cwd: options.cwd ?? "/",
    fdTable: [],
    env: options.env ?? {},
    ttyId: options.ttyId,
    ppid: options.ppid ?? 0,
  };

  // Initialize FDs (0=stdin, 1=stdout, 2=stderr)
  if (options.fds && options.fds.length >= 3) {
    pcb.fdTable = [
      { device: options.fds[0], context: "", flags: "r" },
      { device: options.fds[1], context: "", flags: "w" },
      { device: options.fds[2], context: "", flags: "w" }
    ];
  }

  this.processes.push(pcb);

  // Worker baru hanya dibuat bila ada appName / appPath
  if (options.appName || options.appPath) {
    this.spawnWorker(pcb, options);
  }
}
```

```
    return pcb;
}
```

[!NOTE] createProcess hanya membuat PCB (state READY) + (jika ada app) Worker Thread. pid = nextPid++; FD standar (0/1/2) langsung terisi bila options.fds ≥ 3.

Transisi state (spawnWorker, SIGSTOP/CONT, exit)

```
// spawnWorker() - worker dibuat → RUNNING
pcb.worker = new Worker(workerPath, { workerData, execArgv });
pcb.state = ProcessState.RUNNING;

// kill(pid, 19) - SIGSTOP → BLOCKED
pcb.state = ProcessState.BLOCKED;

// kill(pid, 18) - SIGCONT → RUNNING
pcb.state = ProcessState.RUNNING;

// Handler worker.on("exit") → EXITED (kecuali sedang REEXECING)
pcb.state = ProcessState.EXITED;
```

waitpid + reap (zombie)

```
public async waitpid(pid: number): Promise<number> {
    const pcb = this.getProcess(pid);

    // Jika proses sudah EXITED, langsung balikin 0 (atau simpan exit code di PCB)
    if (!pcb || pcb.state === ProcessState.EXITED) {
        const code = pcb?.exitCode ?? 0;
        // Jika proses sudah EXITED, kita reap sekalian biar gak jadi zombie
        this.reap(pid);
        return code;
    }

    // Proses yang ditunggu (foreground) berhak menerima interrupt Ctrl+C
    this.setForegroundProcess(pid, pcb.ttyId);

    return new Promise((resolve) => {
        if (!this.waitQueue.has(pid)) {
            this.waitQueue.set(pid, []);
        }
        this.waitQueue.get(pid)!.push(resolve);
    });
}
```

```
public reap(pid: number): void {
    const index = this.processes.findIndex(p => p.pid === pid);
    if (index !== -1) {
        const pcb = this.processes[index];
        if (pcb.state === ProcessState.EXITED) {
            this.logger.debug(`Reaping Process ${pcb.name} (PID ${pid}). Memory freed.`);

            // Remove from uuidMap if registered
            if (pcb.uuid && this.uuidMap.get(pcb.uuid) === pid) {
                this.uuidMap.delete(pcb.uuid);
                this.logger.debug(`UUID ${pcb.uuid} released from PID ${pid}`);
            }

            this.processes.splice(index, 1); // PCB dihapus dari tabel
        }
    }
}
```

[!IMPORTANT] `reap()` hanya menghapus PCB yang ber-state `EXITED`. Selama belum `waitpid`, PCB tetap di tabel — itulah **zombie**.

Orphan auto-reparent + notifikasi waiter (`handler worker.on("exit")`)

```
pcb.worker.on("exit", (code) => {
  // If it was a reexec, we don't trigger the exit logic yet
  if (pcb.state === (ProcessState as any).REEXECING) return;

  pcb.state = ProcessState.EXITED;

  // Use explicit exitCode if set (from EXIT syscall), otherwise use worker exit code
  const finalCode = pcb.exitCode !== undefined ? pcb.exitCode : (code || 0);

  // Global Cleanup Hook (reset terminal, close FDs, etc)
  if (this.onProcessExitCallback) {
    this.onProcessExitCallback(pcb.pid);
  }

  // Auto-reparent orphans: semua child dari proses yang exit → init (PID 1)
  const allProcs = Array.from(this.processes.values());
  const orphans = allProcs.filter((p: PCB) => p.ppid === pcb.pid && p.state !== ProcessState.EXITED);
  for (const orphan of orphans) {
    orphan.ppid = 1;
    this.logger.info(`[REARENT] Auto-reparent orphan PID ${orphan.pid} (${orphan.name}) to init (PID 1)`);
  }

  // Notify Wait Queue
  if (this.waitQueue.has(pcb.pid)) {
    const waiters = this.waitQueue.get(pcb.pid)!;
    waiters.forEach(resolve => resolve(finalCode));
    this.waitQueue.delete(pcb.pid);
    // Jika ada yang nunggu (waitpid), kita bisa reap sekarang
    this.reap(pcb.pid);
  } else {
    this.logger.warn(`Process [${pcb.pid}] became a ZOMBIE (Needs waitpid)`);
  }

  // Jika proses foreground yang mati, kembalikan ke NULL
  if (this.getForegroundProcess(pcb.ttyId) === pcb.pid) {
    this.setForegroundProcess(null, pcb.ttyId);
  }
});
```

reexec (PID tetap, raga baru)

```
public async reexec(pid: number, appPath: string, args: string[]): Promise<boolean> {
  const pcb = this.getProcess(pid);
  if (!pcb || !pcb.worker) return false;

  // 1. Mark as Re-execing to prevent 'exit' hook cleanup
  (pcb as any).state = "REEXECING"; // Custom transient state

  // 2. Terminate current worker (Non-blocking to caller)
  pcb.worker.terminate().catch(() => {});
  pcb.worker = undefined;

  // 3. Update PCB info
  pcb.name = path.basename(appPath);
  pcb.state = ProcessState.READY;
  pcb.exitCode = undefined;

  // 4. Spawn new worker
  this.spawnWorker(pcb, { appPath, args });

  return true;
}
```

[!TIP] REEXECING bukan anggota enum `ProcessState` — ia state transien yang di-set via `(pcb as any).state`.
Tujuannya: handler `worker.on("exit")` melewati logika cleanup karena `return` di baris pertama.

detach (daemonize)

```
public async detach(pid: number): Promise<boolean> {
  const pcb = this.getProcess(pid);
  if (!pcb) return false;

  // 1. Resolve Wait Queue (si Shell jadi gak nungguin lagi)
  if (this.waitQueue.has(pid)) {
    const waiters = this.waitQueue.get(pid)!;
    waiters.forEach(resolve => resolve(0)); // Kasih status sukses (0)
    this.waitQueue.delete(pid);
  }

  // 2. Lepas dari Foreground TTY (biar Ctrl+C gak masuk sini lagi)
  if (pcb.ttyId && this.ttyForegroundPids.get(pcb.ttyId) === pid) {
    this.ttyForegroundPids.delete(pcb.ttyId);
  }

  // 3. Clear TTY Association (Crucial for ps display)
  pcb.ttyId = undefined;

  return true;
}
```

[!NOTE] Di level syscall, `DETACH` juga mengarahkan ulang `FD 0/1/2` ke `/dev/null` sebelum memanggil `scheduler.detach()` — agar daemon tidak membocorkan log ke TTY awal.

Pengiriman sinyal (scheduler.kill)

```
public async kill(pid: number, signal: number = 9, exitCode: number = 0): Promise<boolean> {
  const pcb = this.getProcess(pid);
  if (!pcb || !pcb.worker) return false;

  if (signal === 9) { // SIGKILL (Hard Kill)
    if (pcb.worker) {
      await pcb.worker.terminate().catch(() => { });
    }
    return true;
  }

  if (signal === 2) { // SIGINT (Ctrl+C)
    // Send event first to allow graceful handling
    const eventSent = this.sendEvent(pid, "signal", "SIGINT");
    // If no listener handled it, terminate after a brief delay
    // This preserves default Unix behavior: unhandled SIGINT = process exit
    setTimeout(async () => {
      const stillRunning = this.getProcess(pid);
      if (stillRunning && stillRunning.state !== ProcessState.EXITED && stillRunning.worker) {
        this.logger.debug(`PID ${pid} did not handle SIGINT, terminating...`);
        stillRunning.worker.terminate().catch(() => { });
      }
    }, 100); // 100ms grace period for handler to respond
    return eventSent;
  }

  if (signal === 19) { // SIGSTOP (Pause)
    pcb.state = ProcessState.BLOCKED;
    this.logger.info(`Process [${pid}] ${pcb.name} PAUSED (SIGSTOP)`);
    return this.sendEvent(pid, "signal", "SIGSTOP");
  }

  if (signal === 18) { // SIGCONT (Resume)
```

```

    pcb.state = ProcessState.RUNNING;
    this.logger.info(`Process [${pid}] ${pcb.name} RESUMED (SIGCONT)`);
    return this.sendEvent(pid, "signal", "SIGCONT");
}

if (signal === 15) { // SIGTERM (15)
    const eventSent = this.sendEvent(pid, "signal", "SIGTERM");
    setTimeout(async () => {
        const stillRunning = this.getProcess(pid);
        if (stillRunning && stillRunning.state !== ProcessState.EXITED && stillRunning.worker) {
            this.logger.debug(`PID ${pid} did not handle SIGTERM, terminating...`);
            stillRunning.worker.terminate().catch(() => { });
        }
    }, 300);
    return eventSent;
}

// Generic Signal Handling (HUP, USR1, etc)
const sigNames: Record<number, string> = { 1: "SIGHUP", 10: "SIGUSR1", 12: "SIGUSR2" };
const sigName = sigNames[signal] || `SIG${signal}`;
return this.sendEvent(pid, "signal", sigName);
}

```

Gerbang syscall: KILL / SIGNAL / WAITPID

```

case SyscallCode.KILL: {
    const targetPid = args as number;
    const targetPcb = this.scheduler.getProcess(targetPid);
    if (!targetPcb)
        throw new Error(`kill: No such process (PID ${targetPid})`);
    // PID 1 (init) cannot be killed – not even by root
    if (targetPid === 1) {
        throw new Error(`kill: PID 1 (init) is protected – cannot be killed directly. Use 'shutdown' or 'reboot' instead.`);
    }
    // Permission check: root can kill anyone, non-root can only kill own processes
    if (!this.isRoot(pcb) && targetPcb.uid !== pcb.uid) {
        throw new Error(`kill: Permission denied – you do not own process ${targetPid} (owned by UID ${targetPcb.uid})`);
    }
    return await this.scheduler.kill(targetPid, 9); // SIGKILL
}

case SyscallCode.SIGNAL: {
    const { pid: targetPid, sig } = args as { pid: number; sig: number };
    // cek sama: proses ada → PID 1 diproteksi → permission →
    return await this.scheduler.kill(targetPid, sig);
}

case SyscallCode.WAITPID: {
    const targetPid = args as number;
    return await this.scheduler.waitpid(targetPid);
}

```

[!WARNING] Syscall KILL selalu memetakan ke **SIGKILL (9)** — tidak ada sinyal lain. Untuk sinyal lain (SIGTERM, SIGINT, SIGSTOP, ...) gunakan syscall SIGNAL { pid, sig }. Keduanya menolak PID 1 dan mengecek kepemilikan proses.

Latihan / Praktik

1. Jalankan `ps` di shell — identifikasi PID, PPID, state, dan TTY setiap proses.
2. Jalankan `sleep 30 &` lalu `ps` — cari proses yang berjalan di background (state & ttyld kosong).
3. Jalankan sebuah app, catat PID-nya, lalu `kill <pid>` (SIGKILL) dan `kill -15 <pid>` (SIGTERM) — amati perbedaan perilaku di log kernel.
4. Luncurkan proses foreground (tanpa `&`), tekan **Ctrl+C** — amati SIGINT dikirim ke foreground process via `sendInterruptSignal()`.

5. Telusuri walkthrough modul ini (spawn → waitpid → exit → reap) langsung di kode `Scheduler.ts`: `createProcess()`, `waitpid()`, dan handler `worker.on("exit")`.
6. Baca `src/kernel/Syscalls.ts` — bandingkan case KILL (selalu SIGKILL) vs SIGNAL (sig bebas) dan proteksi PID 1.

Referensi

- `wiki/Kernel-dan-Scheduler.md` — detail proses & scheduler
- `wiki/course/00-overview.md` §4.1, §4.3
- `src/kernel/Scheduler.ts` — PCB, lifecycle, waitpid, detach, sinyal, reexec
- `src/kernel/Syscalls.ts` — case KILL, SIGNAL, WAITPID, EXIT, DETACH, REEXEC, SHUTDOWN
- `src/kernel/Scheduler.test.ts` — konfirmasi perilaku (A2.19–A2.21 zombie/waitpid, A2.26 detach, reexec)

Modul 04 — selesai. Lanjut ke [Modul 05 — Syscall & IPC](#).

RFC-TSIX-EDU-002 | Modul kelima kurikulum TSIX. Memahami ABI mini TSIX: bagaimana aplikasi memanggil kernel via syscall, korelasi request-response, dan push event dari kernel ke worker.

Syscall TSIX adalah **RPC asinkron murni**. Tidak ada urutan yang dijamin — setiap request membawa `requestId` (UUID) dan dijawab dengan `requestId` yang sama. Ini berbeda dari trap sinkron di Linux.

Tujuan Pembelajaran

- ☐ Menjelaskan alur satu syscall lengkap (dari app sampai balik)
- ☐ Menjelaskan peran `requestId` + `responseMap`
- ☐ Membedakan request-response vs push event
- ☐ Menyebutkan beberapa kode syscall penting (1-73)
- ☐ Menjelaskan peran `validateArgs`

Konsep Inti

TSIX memakai IPC *request-response* asinkron — bukan trap sinkron ala Linux. Aplikasi dan kernel terpisah *thread* (Worker vs main thread). Satu-satunya jembatan adalah `postMessage` di kedua arah.

Konsep	Penjelasan
<code>SyscallRequest</code>	Paket app → kernel: { <code>requestId</code> , <code>pid</code> , <code>code</code> , <code>args</code> }
<code>SyscallResponse</code>	Paket kernel → app: { <code>requestId</code> , <code>success</code> , <code>data?</code> , <code>error?</code> }
<code>requestId</code> (UUID)	Korelasi permintaan ↔ jawaban. Dibuat di <code>UserLib.dispatch</code> , dipakai lagi kernel saat membalas.
<code>responseMap</code>	Map di sisi app: <code>requestId</code> → callback <code>resolve/reject</code> . Balasan dicocokkan lewat kunci ini.
Push event	Notifikasi kernel → app <i>tanpa</i> <code>requestId</code> : { <code>type</code> , <code>data</code> } (signal, ipc_message, gui_request, db_request, resize).
<code>validateArgs</code>	Kontrak argumen di kernel: cek syscall mana yang wajib punya <code>args</code> , plus cek tipe spesifik per syscall.

[!IMPORTANT] Karena RPC asinkron murni, **urutan balasan tidak dijamin**. Dua syscall berurutan (A lalu B) bisa dibalas B lebih dulu. Korelasi selalu lewat `requestId`, bukan urutan.

Alur / Cara Kerja

Alur 1 syscall lengkap: PRINT

Contoh paling sederhana: aplikasi memanggil `std.print("hello")`. Berikut jalur lengkapnya (kode sebagai kebenaran):

```
// src/mirror/lib/UserLib.ts - StdLib
public async print(text: string) {
  return await this.dispatch(SyscallCode.PRINT, text);
}
```

Langkah demi langkah:

- `std.print("hello")` memanggil `UserLib.dispatch(PRINT, "hello")` di sisi worker.
- `dispatch()` membuat `requestId = uuidv4()`, menyimpan callback ke `responseMap`, lalu `parentPort.postMessage({ requestId, pid, code: PRINT, args: "hello" })`.

3. Kernel menerima pesan di `Scheduler.spawnWorker()` → handler `pcb.worker.on("message", ...)`.
4. Handler memanggil `syscallHandler(request)` — di-wire di `Kernel.initializeSubsystems()` lewat `scheduler.setSyscallHandler(...)`.
5. `SyscallDispatcher.handleRequest(req)` → `validateArgs(PRINT, args)` → `dispatch(pid, PRINT, args)`.
6. Case `PRINT` mengambil FD 1 dari `pcb.fdTable`, lalu `entry.device.write("hello")` → output ke layar.
7. Hasil (0) kembali ke handler Scheduler; Scheduler membalas `postMessage({ requestId, success: true, data: 0 })`.
8. Di worker, `parentPort.on("message")` menemukan `requestId` di `responseMap`, memanggil callback → Promise `resolve(0)` → await print selesai.

```
sequenceDiagram
    participant App as Aplikasi (Worker)
    participant Lib as UserLib (dispatch)
    participant Sch as Scheduler (worker.on message)
    participant Disp as SyscallDispatcher
    participant Dev as Device (fdTable[1])

    App->>Lib: std.print("hello")
    Lib->>Lib: requestId = uuidv4(); responseMap.set(requestId, cb)
    Lib->>Sch: postMessage({ requestId, pid, code: PRINT, args })
    Sch->>Disp: await syscallHandler(request)
    Disp->>Disp: validateArgs(PRINT, args)
    Disp->>Dev: dispatch(pid, PRINT, args) → fdTable[1].device.write()
    Dev-->>Disp: return 0
    Disp-->>Sch: return 0
    Sch->>Lib: postMessage({ requestId, success: true, data: 0 })
    Lib->>Lib: cocokkan requestId di responseMap → resolve(0)
    Lib-->>App: await print selesai
```

Korelasi request-response via `requestId` (UUID) + `responseMap` — murni RPC asinkron. Banyak syscall bisa in-flight sekaligus.

Push event (kernel → worker, tanpa requestId)

Berbeda dari request-response, push event tidak punya `requestId`. Kernel mengirim `{ type, data }` kapan pun dibutuhkan:

```
{ type, data }
```

Tipe yang benar-benar dipakai di kode (`Scheduler.ts`, `Syscalls.ts`, `Kernel.ts`):

type	Pemakaian di kode	Tujuan
signal	<code>sendEvent(pid, "signal", "SIGINT")</code> dll.	Sinyal ke proses (SIGINT, SIGTERM, SIGSTOP, SIGCONT, SIGWINCH, SIGSEGV...)
ipc_message	<code>SEND_MSG</code> antar-app; forward paket <code>NET_SNIFF</code>	Horizontal IPC / sniffer
gui_request	Cleanup window & forward ke GUI daemon	GUI daemon (PixelSpace)
db_request	<code>forwardDbRequest</code> → DB service daemon	Transport alternatif database
resize	<code>broadcastEvent("resize", { lines, columns })</code> saat terminal host berubah	Semua proses aktif

[!NOTE] Di sisi worker, `UserLib.parentPort.on("message")` membedakan tiga hal: (1) balasan syscall — ada `requestId` yang cocok di `responseMap`; (2) event — ada `type` dan `type !== "signal"`; (3) sinyal — `type === "signal"`, dengan default aksi `SIGINT` → `exit(130)` dan `SIGTERM` → `exit(143)` bila tidak ada listener.

Daftar Syscall Utama (ABI mini)

Kode	Nama	Kode	Nama
1	PRINT	29	PIPE
2	MKDIR	30–34	SOCKET/BIND/SENDTO/RECVFROM/NETSTAT
3	LS	35	UPTIME
4	EXIT	36	DETACH
5–8	OPEN/READ/WRITE/CLOSE	37	UNAME
9	SCREEN_INFO	38	SETGROUPS
10–12	PS/KILL/EXEC	40–41	GET_SYSPATH/GET_USAGE
13–14	CHDIR/GETCWD	50	SHUTDOWN
15–16	CHMOD/CHOWN	53–55	SYNC_TO_HOST/REEXEC/SYNC_FROM_HOST
17–19	WHOAMI/GETENV/SETENV	56–58	MOUNT/UMOUNT/GET_MOUNTS
20–23	STAT/UNLINK/RMDIR/IOCTL	60	SET_IDENTITY
24–28	SEND_MSG/WAITPID/SIGNAL/SETUID/SETGID	61–66	GUI_REQ/GET_PPID/READ_CHUNK/WRITE_CHUNK/GET_SIZE/REARENT
—	—	67–71	DB_CONNECT/DB_QUERY/DB_DISCONNECT/DB_SERVICE_REGISTER/DB_SERVICE_REPLY
—	—	72–73	NET_SNIFFER_REGISTER/NET_SNIFFER_UNREGISTER

SyscallCode adalah enum di `src/common/SyscallCode.ts` — kontrak ABI. Jangan mengubah nomor yang sudah ada (breaking change).

Beberapa kode kunci (nomor → nama → tujuan):

Nomor	Nama	Tujuan
1	PRINT	Mencetak teks ke layar (via FD 1)
14	GETCWD	Mendapatkan direktori kerja proses
24	SEND_MSG	Horizontal IPC (app-to-app)
25	WAITPID	Menunggu proses selesai berdasarkan PID
26	SIGNAL	Mengirim sinyal ke proses
61	GUI_REQ	PixelSpace Display Protocol (RFC-TSIX-002)
67	DB_CONNECT	Membuka koneksi database eksternal

Kode Sumber

File	Isi	Relevan
<code>src/common/SyscallCode.ts</code>	Enum ABI 1–73	Nomor syscall
<code>src/common/IPCTypes.ts</code>	Bentuk paket SyscallRequest, SyscallResponse, IPCEvent	Kontrak paket

File	Isi	Relevan
src/kernel/Syscalls.ts	SyscallDispatcher: <code>handleRequest</code> , <code>validateArgs</code> , <code>dispatch</code>	Dispatcher
src/kernel/Scheduler.ts	<code>spawnWorker</code> + <code>worker.on("message")</code> (balasan) + <code>sendEvent</code>	Jembatan & reply
src/kernel/Kernel.ts	Wiring <code>setSyscallHandler(...)</code>	Wiring
src/mirror/lib/UserLib.ts	<code>dispatch()</code> , <code>responseMap</code> , <code>parentPort.on("message")</code> , <code>onEvent</code>	Sisi worker

Snippet (level kode)

Semua snippet di bawah ini disalin dari sumber. "Kode adalah kebenaran."

1. UserLib dispatch (sisi worker) — src/mirror/lib/UserLib.ts

```
private async dispatch(code: SyscallCode, args: any): Promise<any> {
  return new Promise((resolve, reject) => {
    const requestId = uuidv4();
    const request: SyscallRequest = { requestId, pid: this.pid, code, args };

    this.responseMap.set(requestId, (response: SyscallResponse) => {
      if (response.success) {
        resolve(response.data);
      } else {
        reject(new Error(response.error || "Syscall Failed"));
      }
    });

    if (parentPort) {
      parentPort.postMessage(request);
    } else {
      reject(new Error("No parentPort found! Are you running in a Worker?"));
    }
  });
}
```

`responseMap` bertipe `Map<string, (res: SyscallResponse) => void>` — nilainya **callback**, bukan objek `{ resolve, reject }`. Promise di-resolve/reject di dalam callback saat balasan tiba.

2. Sisi penerima (worker) — parentPort.on("message")

```
parentPort.on("message", (msg: any) => {
  // 1. Balasan Syscall
  if (msg.requestId && this.responseMap.has(msg.requestId)) {
    const resolve = this.responseMap.get(msg.requestId)!;
    resolve(msg as SyscallResponse);
    this.responseMap.delete(msg.requestId);
  }
  // 2. Event (Push Notification) – selain signal
  else if (
    msg.type &&
    msg.type !== "signal" &&
    this.eventListeners.has(msg.type)
  ) {
    const callbacks = this.eventListeners.get(msg.type)!;
    callbacks.forEach((cb) => cb(msg.data));
  }
  // 3. Signal (SIGINT/SIGTERM) dengan default action
  else if (msg.type === "signal") {
    const sig = msg.data;
    // ... default: exit(130) untuk SIGINT, exit(143) untuk SIGTERM
  }
});
```

```
}  
});
```

3. Kernel handleRequest — src/kernel/Syscalls.ts

```
public async handleRequest(req: SyscallRequest): Promise<any> {  
  const silentCodes = [  
    SyscallCode.READ,  
    SyscallCode.PS,  
    SyscallCode.GETCWD,  
    SyscallCode.WHOAMI,  
  ];  
  if (!silentCodes.includes(req.code)) {  
    this.logger.debug(  
      `[IPC] PID ${req.pid} Request: ${SyscallCode[req.code]} (${req.requestId})`,  
    );  
  }  
  
  try {  
    this.validateArgs(req.code, req.args);  
    return await this.dispatch(req.pid, req.code, req.args);  
  } catch (e: any) {  
    this.logger.error(  
      `Syscall Error [${SyscallCode[req.code]}]: ${e.message}`,  
    );  
    throw e;  
  }  
}
```

⚠ **Koreksi penting:** `handleRequest` **tidak membalas sendiri**. Ia hanya memvalidasi, mengeksekusi, lalu mengembalikan hasil (atau melempar error). Balasan `postMessage` dikirim oleh **Scheduler** di handler `worker.on("message")` (lihat snippet 4).

4. Balasan dikirim oleh Scheduler — src/kernel/Scheduler.ts (spawnWorker)

```
pcb.worker.on("message", async (request: SyscallRequest) => {  
  if (this.syscallHandler) {  
    try {  
      const result = await this.syscallHandler(request);  
      if (pcb.worker) {  
        pcb.worker.postMessage({  
          requestId: request.requestId,  
          success: true,  
          data: result  
        });  
      }  
    } catch (error: any) {  
      if (pcb.worker) {  
        pcb.worker.postMessage({  
          requestId: request.requestId,  
          success: false,  
          error: error.message  
        });  
      }  
    }  
  }  
});
```

5. Contoh handler syscall nyata — src/kernel/Syscalls.ts

```
case SyscallCode.PRINT: {  
  const entry = pcb.fdTable[1];  
  if (!entry) return -1;  
  entry.device.write(args as string);  
  return 0;  
}
```

```

}

case SyscallCode.GETCWD:
    return pcb.cwd;

```

PRINT tidak menulis langsung ke layar — ia menulis lewat **FD 1** (stdout) di fdTable. Ini wujud "everything is a file": output hanyalah device yang terpasang di fdTable. GETCWD sebaliknya, cukup mengembalikan pcb.cwd (state proses di PCB).

6. Push event — sendEvent di src/kernel/Scheduler.ts

```

public sendEvent(pid: number, type: string, data: any): boolean {
    const pcb = this.getProcess(pid);
    if (pcb && pcb.worker && pcb.state !== ProcessState.EXITED) {
        pcb.worker.postMessage({ type, data });
        return true;
    }
    return false;
}

```

Contoh pemakaian (forward paket sniffer di Syscalls.ts):

```

this.scheduler.sendEvent(pid, "ipc_message", { data: sniff });

```

Latihan / Praktik

1. Baca src/common/SyscallCode.ts — catat semua nomor syscall (1-73). Perhatikan celah nomor (39, 42-49, 51-52, 59) yang sengaja direservasi.
2. Baca src/common/IPCTypes.ts — kenali bentuk SyscallRequest, SyscallResponse, IPCEvent.
3. Tambahkan log di handleRequest untuk syscall PRINT — lalu jalankan echo hello di shell. Amati alurnya (requestId yang sama di request & response).
4. Jelaskan mengapa requestId diperlukan — apa yang terjadi tanpa korelasi?
5. Kirim dua syscall sekaligus (mis. READ file besar + PRINT) dan amati urutan balasannya — buktikan urutan tidak dijamin.
6. Dari sisi worker, cek isi responseMap segera setelah dispatch() dipanggil (sebelum balasan tiba) — buktikan nilainya berupa callback.

Referensi

- src/common/SyscallCode.ts — enum ABI (kebenaran nomor syscall)
- src/common/IPCTypes.ts — bentuk paket IPC
- src/kernel/Syscalls.ts — handleRequest, validateArgs, dispatch
- src/kernel/Scheduler.ts — worker.on("message") (balasan) + sendEvent (push)
- src/kernel/Kernel.ts — wiring setSyscallHandler
- src/mirror/lib/UserLib.ts — dispatch(), responseMap, onEvent
- wiki/identity_guid_ipc_walkthrough.md — walkthrough IPC & identitas
- wiki/course/00-overview.md §4.2

Modul 05 — selesai. Lanjut ke [Modul 06 — Permission & Security](#).

RFC-TSIX-EDU-002 | Modul keenam kurikulum TSIX. Memahami model izin rwx, setuid bit, root bypass, dan kelemahan yang diketahui pada sistem permission TSIX.

PermissionManager adalah "satpam" kernel. Semua akses file melewati cek berlapis: root bypass → owner → group → others. Ditambah setuid bit untuk binary istimewa seperti `login` dan `sudo`.

Tujuan Pembelajaran

- ☐ Menjelaskan urutan cek izin (root → owner → group → others)
- ☐ Menjelaskan bit permission `r=4, w=2, x=1`
- ☐ Membaca matriks izin dan mode contoh (`600`, `644`, `755`, `4755`) beserta nilai decimalnya
- ☐ Menjelaskan setuid bit (`0o4000` = 2048) dan penanganannya saat `EXEC`
- ☐ Menjelaskan mengapa `parseMode("4755")` bernilai 2541 (decimal)
- ☐ Menjelaskan aturan `CHMOD` (owner atau root) vs `CHOWN` (root saja)
- ☐ Menjelaskan kelemahan privilege berbasis nama app dan mitigasinya

Konsep Inti

Cek rwx berlapis

```
check(pcb, node, requested):
  1. root (uid 0)          → true (God Mode)
  2. owner (uid == node.uid) → bit owner  ((mode >> 6) & 0x7)
  3. group (gid match)     → bit group   ((mode >> 3) & 0x7)
  4. others                → bit others  (mode & 0x7)
```

 Alur cek izin PermissionManager (root → owner → group → others) Sumber: wiki/diagram/Keamanan-dan-Sandboxing-3.mmd

Matriks izin rwx

Satu mode = **3 digit octal**, masing-masing untuk kelas owner, group, dan others. `check()` menggeser bit sesuai kelas:

Kelas	Perhitungan di <code>check()</code>	Bit	Contoh <code>644</code> (decimal <code>420</code>)
owner (user)	<code>(mode >> 6) & 0x7</code>	<code>r=4, w=2, x=1</code>	digit 6 → <code>rw-</code>
group	<code>(mode >> 3) & 0x7</code>	<code>r=4, w=2, x=1</code>	digit 4 → <code>r--</code>
others	<code>mode & 0x7</code>	<code>r=4, w=2, x=1</code>	digit 4 → <code>r--</code>

Encode permission

Bit	Nilai	Makna
<code>r</code> (read)	4	Membaca konten
<code>w</code> (write)	2	Menulis / mengubah
<code>x</code> (execute)	1	Mengeksekusi

Kombinasi bit per digit octal (dipakai dalam tabel mode di bawah):

Digit	Biner	rwX	Arti
0	000	---	tanpa akses
1	001	--X	execute
2	010	-W-	write
3	011	-WX	write + execute
4	100	r--	read
5	101	r-X	read + execute
6	110	rw-	read + write
7	111	rwx	read + write + execute

Mode umum & nilai decimal

Mode disimpan sebagai **decimal di SQLite** (bukan string octal). `parseMode("4755") = parseInt("4755", 8) = 2541` (decimal) — inilah nilai yang dipakai `init.ts` saat `chmod("/bin/passwd.js", 2541)`.

Mode	Octal	Decimal	owner	group	others	Contoh penggunaan
600	0o600	384	rw-	---	---	file pribadi: <code>/etc/shadow</code> , kunci SSH
644	0o644	420	rw-	r--	r--	file teks biasa (mode default <code>touch</code> di syscall <code>OPEN</code>)
755	0o755	493	rwx	r-x	r-x	direktori & binary (mode default <code>mkdir</code>)
4755	0o4755	2541	rwS	r-x	r-x	binary setuid: <code>/bin/passwd.js</code> , <code>/bin/sudo.js</code>

s pada posisi owner = **setuid bit aktif** (`0o4000` = 2048), bukan `x` biasa. Nilai decimal dikonfirmasi test A5.10 (`644` → 420, `755` → 493) dan A5.13 (`4755` & `0o4000`).

SetUID bit (0o4000)

SetUID = eksekusi sebagai **pemilik file**, bukan pemanggil. Contoh:

- `/bin/passwd.js` → `chmod 2541 (0o4755)`: user biasa menjalankan `passwd`, proses berjalan sebagai root agar bisa mengubah `/etc/shadow`.
- `/bin/sudo.js` → sama.

Penanganan saat EXEC (di `Syscalls.ts`): bit `0o4000` = **2048** (decimal). Saat mengeksekusi binary, kernel mengecek `node.mode` & 2048. Jika aktif, `targetUid = node.uid` dan `targetGid = node.gid`; proses baru lahir dengan uid/gid pemilik file, sementara `ruid` (real UID) tetap milik pemanggil. Lihat snippet di bawah.

Catatan: runtime mengeksekusi sidecar `.js`, jadi SetUID harus dipasang di `/bin/*.js`, bukan source `.ts`-nya. Inilah yang dilakukan `init.ts`: `lib.fs.chmod("/bin/passwd.js", 2541)` dan `lib.fs.chmod("/bin/sudo.js", 2541)`. `rc.local.ts` juga memasang `chmod("/bin/login.js", 0o4755)`.

Urutan POSIX saat login: `setgroups` → `setgid` → `setuid` (harusurut ini agar group benar). Terlihat di `login.ts`: `setgroups(supplementaryGids) → setgid(gid) → setuid(uid)`.

Snippet (level kode)

PermissionManager.check()

```
public check(pcb: PCB, node: any, requested: Permission): boolean {
  if (pcb.uid === 0) return true; // 1. root bypass
```



```

const mode = node.mode; // decimal di SQLite

if (pcb.uid === node.uid) { // 2. owner
  const userMode = (mode >> 6) & 0x7;
  if ((userMode & requested) === requested) return true;
}

if (pcb.gid === node.gid || (pcb.groups && pcb.groups.includes(node.gid))) {
  const groupMode = (mode >> 3) & 0x7; // 3. group
  if ((groupMode & requested) === requested) return true;
}

const otherMode = mode & 0x7; // 4. others
if ((otherMode & requested) === requested) return true;

return false;
}

public static parseMode(octal: string | number): number {
  return parseInt(octal.toString(), 8); // "4755" → 2541
}

```

EXEC — penegakan izin & setuid (Syscalls.ts)

```

// 1. Cek x bit dulu – tanpa izin eksekusi, langsung ditolak
if (!this.satpam.check(pcb, node, Permission.EXECUTE)) {
  throw new Error(`Permission Denied: Cannot execute ${absoluteExecPath}`);
}

let targetUid = pcb.uid;
let targetGid = pcb.gid;
let targetOwner = pcb.owner;

// 2. SETUID BIT SUPPORT (0o4000 = 2048)
if (node && node.mode & 2048) {
  targetUid = node.uid; // eksekusi sebagai pemilik file
  targetGid = node.gid;
  if (targetUid === 0) targetOwner = "root";
  this.logger.info(
    `SetUID Execution detected for ${absoluteExecPath}: Running as UID ${targetUid}`,
  );
}

// 3. Proses baru lahir dengan uid/gid target; ruid (real UID) dipertahankan
const newPcb = this.scheduler.createProcess(binaryName, {
  // ...
  uid: targetUid,
  gid: targetGid,
  ruid: pcb.ruid, // Preserve Real UID
  owner: targetOwner,
  // ...
});

```

Jalur ini adalah kunci keamanan `/bin/login.js`, `/bin/passwd.js`, dan `/bin/sudo.js`: user biasa tidak bisa setuid langsung, tapi mengeksekusi binary setuid root membuat proses berjalan sebagai uid 0.

OPEN — read vs write (Syscalls.ts)

```

const requiredPerm =
  flags.includes("w") || flags.includes("+")
    ? Permission.WRITE
    : Permission.READ;

const node = vfs.stat(relativePath);

if (node) {

```

```
// File sudah ada → cek izin file itu sendiri
if (!this.satpam.check(pcb, node, requiredPerm)) {
  throw new Error(
    `Permission Denied: Cannot open ${absoluteOpenPath} for ${Permission[requiredPerm]}`,
  );
}
} else if (requiredPerm === Permission.WRITE) {
  // File belum ada → cek WRITE ke direktori parent, lalu buat mode 420 (644)
  // ...
  vfs.touch(relativePath, "", pcb.uid, pcb.gid, 420); // 644 equivalent
}
}
```

CHMOD — ubah mode (Syscalls.ts)

```
case SyscallCode.CHMOD: {
  const { path: argPath, mode } = args as { path: string; mode: number };
  const absolutePath = PathResolver.resolve(pcb.cwd, argPath);

  if (absolutePath.startsWith("/dev/")) {
    if (!this.isRoot(pcb)) return false; // chmod /dev/* → root only
    const devName = absolutePath.replace("/dev/", "");
    const device = this.kernel.devices[devName];
    if (!device) return false;
    device.mode = mode;
    return true;
  }

  const { vfs, relativePath } = this.mountManager.resolve(absolutePath);
  const node = vfs.stat(relativePath);
  if (!node) return false;
  if (pcb.uid !== 0 && pcb.uid !== node.uid) return false; // root ATAU owner
  return vfs.chmod(relativePath, mode);
}
```

Aturan **CHMOD**: file biasa boleh diubah **owner-nya sendiri atau root**. Diverifikasi test A5.11.

CHOWN — ubah owner/group (Syscalls.ts)

```
case SyscallCode.CHOWN: {
  const {
    path: argPath,
    uid: targetUid,
    gid: targetGid,
  } = args as { path: string; uid: number; gid: number };
  const absolutePath = PathResolver.resolve(pcb.cwd, argPath);

  if (absolutePath.startsWith("/dev/")) {
    if (!this.isRoot(pcb)) return false; // chown /dev/* → root only
    // ...
  }

  if (!this.isRoot(pcb)) return false; // chown file → ROOT SAJA
  const { vfs, relativePath } = this.mountManager.resolve(absolutePath);
  return vfs.chown(relativePath, targetUid, targetGid);
}
```

Aturan **CHOWN**: **hanya root** yang boleh mengganti owner file (test A5.15). Berbeda dari **CHMOD** yang juga boleh dilakukan oleh owner.

Kelemahan yang Diketahui

[!WARNING] **Privilege berbasis nama app (rapuh).** Di `WorkerEntry.ts`, `restrictHostAPI(appName)` menandai app sebagai "privileged" jika **substring** namanya (setelah `toLowerCase()`) mengandung salah satu dari:

```
const isPrivileged = appName.toLowerCase().includes("server") ||
  appName.toLowerCase().includes("daemon") ||
  appName.toLowerCase().includes("dome") ||
  appName.toLowerCase().includes("tbuild") ||
  appName.toLowerCase().includes("vfs") ||
  appName.toLowerCase().includes("mysqld");
```

App privileged mendapat **allow-list modul host**:

```
const allowedModules = ["http", "ws", "path", "fs", "url", "esbuild", "crypto", "os", "bcryptjs", "mysql2", "mysql2/promise"];
```

Skenario eksploitasi (contoh):

1. Attacker meng-upload app bernama `evil-daemon` (nama mengandung `daemon`).
2. Saat dieksekusi, `isPrivileged = true` → `global.require = privilegedRequire`.
3. App kini bisa `require("fs")`, `require("crypto")`, `require("http")`, dst. — modul Node.js **host langsung**, bukan syscall TSIX.
4. Dengan `fs`, attacker bisa membaca file di luar sandbox VFS TSIX (mis. `/etc/shadow` host Linux) → **escape sandbox**.

[!IMPORTANT] Mitigasi.

- **Jangka panjang (arsitektural):** ganti heuristik substring dengan **capability-based** — daftar eksplisit pasangan `appName` → `capabilities` (atau manifes per-app).
- **Sekarang (defense-in-depth):** meski app "privileged" di sandbox, kernel tetap punya `PermissionManager` + `validateArgs` + `SETUID` root-only. Nama app hanya membuka pintu modul host — **bukan** akses kernel penuh; setiap syscall tetap diperiksa per-invocation di sisi kernel.

Latihan / Praktik

1. Jalankan `ls -l /bin/passwd.js` — amati bit `setuid (s)` pada mode owner (mode `rwsr-xr-x`).
2. Jalankan `stat /bin/passwd.js` — cek nilai mode decimal (harus `2541 = 0o4755`).
3. Jalankan `chmod 755 /bin/passwd.js` lalu `ls -l` lagi — bit `s` hilang; `init` akan memasangnya kembali (`init.ts`).
4. Sebagai user non-root, coba baca `/etc/shadow` — amati error permission.
5. Sebagai user non-root, jalankan `chown root /tmp/foo` — amati error (hanya root yang boleh `chown`); lalu `chmod 600 /tmp/foo` — harus berhasil karena owner boleh `chmod`.
6. Baca `src/kernel/PermissionManager.ts` dan `src/kernel/Syscalls.ts` — temukan semua titik panggil `satpam.check()`.
7. Baca `src/userland/WorkerEntry.ts` → fungsi `restrictHostAPI` — uji: jalankan app bernama `evil-daemon` yang memanggil `require("fs")`, bandingkan dengan app bernama `hitung` biasa.

Referensi

- `wiki/Keamanan-dan-Sandboxing.md` — model keamanan lengkap
- `wiki/course/00-overview.md` §4.3 (model ring & batas privilege)
- `src/kernel/PermissionManager.ts` — implementasi `check()` & `parseMode()`
- `src/kernel/PermissionManager.test.ts` — test A5.1-A5.30 (`rx`, `chmod`, `chown`, `umask`, `setuid`, `sudo`)
- `src/kernel/Syscalls.ts` — penegakan izin `OPEN/EXEC/CHMOD/CHOWN` + `setuid` di `EXEC`
- `src/userland/WorkerEntry.ts` — sandbox `restrictHostAPI` (privilege berbasis nama)
- `src/mirror/bin/init.ts`, `src/mirror/bin/rc.local.ts` — pemasangan bit `setuid`

- `src/mirror/bin/login.ts`, `src/mirror/bin/sudo.ts` — alur setgroups/setgid/setuid
-

Modul 06 — selesai. Lanjut ke [Modul 07 — Mount & Path Resolution](#).

RFC-TSIX-EDU-002 | Modul ketujuh kurikulum TSIX. Memahami bagaimana path VFS dirutekan ke backend filesystem yang tepat — dengan prinsip **prefix terpanjang menang**.

MountManager adalah "router path". Saat aplikasi membuka `/tmp/foo`, kernel harus tahu bahwa itu milik RamFS, bukan BKFS. Aturannya sederhana: **prefix terpanjang yang cocok adalah pemenangnya**.

Tujuan Pembelajaran

- ☐ Menjelaskan struktur `MountPoint`
- ☐ Menjelaskan prinsip "prefix terpanjang menang"
- ☐ Menjelaskan isi `fstab.json` (mount default)
- ☐ Membedakan resolve via MountManager vs device `/dev/*`
- ☐ Menjelaskan peran `PathResolver`
- ☐ Melakukan walkthrough `PathResolver.resolve()` untuk path relatif, absolut, `.` dan `..`
- ☐ Memprediksi hasil `resolve()` dengan membaca urutan sortir mount

Konsep Inti

Dua lapis resolusi

Resolusi path di TSIX berjalan dua lapis:

1. `PathResolver.resolve()` — menormalisasi path string. Ia hanya memproses teks (`//`, `.`, `..`), tanpa tahu soal filesystem.
2. `MountManager.resolve()` — merutekan path ternormalisasi ke backend `IVFS` yang tepat, dengan aturan "prefix terpanjang menang".

[!IMPORTANT] `PathResolver` bekerja di level **string**. `MountManager` bekerja di level **backend filesystem**. Keduanya adalah fungsi yang berbeda.

MountPoint

```
export interface MountPoint {
  vfsPath: string;    // titik kait di VFS, misal "/tmp"
  vfs: IVFS;          // backend filesystem
  type: string;       // "bkfs" | "ramfs" | "host"
  source: string;     // "system.db" | "shared" | "systembak.db" | ...
  readOnly: boolean;
  uid?: number;
  gid?: number;
}
```

`type` memakai nilai nyata dari `fstab.json`: `"bkfs"`, `"ramfs"`, dan `"host"` (HostVFS).

PathResolver.resolve() — walkthrough

`PathResolver.resolve(cwd, targetPath)` menerima dua argumen: direktori kerja (`cwd`) dan path target. Hasilnya selalu path absolut ternormalisasi (tanpa `//`, `.`, atau `..`).

#	cwd	targetPath	Hasil	Penjelasan
1	/	/etc/passwd	/etc/passwd	Path absolut — cwd diabaikan
2	/	etc/passwd	/etc/passwd	Path relatif di-root
3	/bin	../etc/passwd	/etc/passwd	.. naik satu level keluar dari /bin
4	/home/user	docs../a.txt	/home/user/a.txt	.. membatalkan segmen docs
5	/	/tmp../foo//bar	/tmp/foo/bar	// dan . dibersihkan
6	/	/../x	/x	.. di atas root tidak bisa naik lagi

[!NOTE] Contoh 4 dan 5 menunjukkan bahwa normalisasi terjadi **sebelum** pencarian mount. Jadi `resolve("/tmp../foo")` dan `resolve("/tmp/foo")` dianggap sama oleh `MountManager`.

Mount default

Root / **tidak** ada di `fstab.json`. Ia di-mount langsung di `initializeSubsystems()`:

```
this.mountManager.mount("/", this.bkfs, "bkfs", cfg.kernel.database, false);
```

`cfg.kernel.database` bernilai "system.db" (lihat `src/sysconfig.json`). Sisanya dimuat dari `src/mirror/etc/fstab.json` oleh `processFstab()`:

Path	type	Backend	Sumber	readOnly	uid/gid	mode	Sifat
/	bkfs	BKFS (SQLite)	system.db	false	—	—	persisten, root fs
/tmp	ramfs	RamFS	in-memory (hostPath "RAM" diabaikan)	false	0/100	0o1777	volatile, sticky
/mnt/shared	host	HostVFS	folder host shared	false	1000/100	0o775	bridge host, anti-escape
/mnt/sbak	bkfs	BKFS	systembak.db	false	1000/100	0o775	backup DB

[!NOTE] `fstab.json` hanya memuat `/tmp`, `/mnt/shared`, dan `/mnt/sbak`. Entry `/tmp` memakai `active: true`; sisanya default aktif. Mode `1023 = 0o1777` (sticky, semua bisa tulis — khas `/tmp`), `509 = 0o775`.

Resolve: prefix terpanjang menang

Setelah semua mount terpasang, daftar di-sort **descending by panjang path**. Urutan hasil sortir untuk mount default:

```
/mnt/shared (11) ← diperiksa dulu
/mnt/sbak   ( 9)
/tmp        ( 4)
/           ( 1) ← fallback terakhir
```

Contoh kerja `MountManager.resolve()`:

Path diminta	MountPoint terpilih	relativePath	vfs
/tmp/a.txt	/tmp	/a.txt	RamFS
/tmp	/tmp (exact)	/	RamFS
/etc/passwd	/	/etc/passwd	BKFS
/mnt/shared/readme.md	/mnt/shared	/readme.md	HostVFS

Path diminta	MountPoint terpilih	relativePath	vfs
/mnt/shared	/mnt/shared (exact)	/	HostVFS
/mnt/sbak/backup.tar	/mnt/sbak	/backup.tar	BKFS
/mnt/sharedx/foo	/	/mnt/sharedx/foo	BKFS

[!WARNING] Baris terakhir adalah gotcha penting. /mnt/sharedx/foo **tidak** cocok dengan prefix /mnt/shared/ (karena setelah shared ada x, bukan /). Karena itu ia jatuh ke / → BKFS. Inilah kenapa prefix memakai slash tambahan (m.vfsPath + "/"), bukan sekadar startsWith(m.vfsPath) .

 Resolusi path: syscall → MountManager → backend VFS Sumber: [wiki/diagram/Home-2.mmd](#)

Path /dev/* tidak lewat MountManager

Path dev/xxx di-handle oleh **HAL** (kernel.devices[xxx]), bypass MountManager. Ini konsisten dengan prinsip "everything is a file" — device bukan vnode filesystem.

Alur / Cara Kerja

Alur mount saat boot

```
initializeSubsystems()
└─ new BKFS("system.db")           → root filesystem
└─ mount("/", bkfs, "bkfs", false)
└─ processFstab()                  → baca /etc/fstab.json
    └─ untuk tiap entry:
        active === false → skip
        pastikan dir mount point ada (mode ?? 0o755)
        pilih driver: bkfs → BKFS, ramfs → RamFS, else → HostVFS
        mountManager.mount(vfsPath, driver, type, hostPath, ...)
└─ mount() normalisasi path + sort descending by panjang
```

Alur resolve saat syscall file

```
Aplikasi → syscall file (mis. OPEN, path mentah)
→ MountManager.resolve(path)
  1. normalized = PathResolver.resolve("/", path)
  2. loop mounts (terpanjang dulu):
      exact match → relativePath = "/"
      prefix match → relativePath = sisa path
  3. hasil: { vfs, relativePath, mountPoint }
→ vfs.<op>(relativePath)           // backend mengeksekusi
Path /dev/* → kernel.devices[xxx] // HAL, bukan MountManager
```

Kode Sumber

File	Peran
src/kernel/MountManager.ts	mount/unmount/resolve/listMounts
src/common/PathResolver.ts	Normalisasi path (// , . , ..)
src/kernel/Kernel.ts	initializeSubsystems() + processFstab() saat boot
src/mirror/etc/fstab.json	Daftar mount default
src/vfs/HostVFS.ts, RamFS.ts, BKFS.ts	Backend

Snippet (level kode)

PathResolver.resolve()

```
public static resolve(cwd: string, targetPath: string): string {
    // 1. Jika path diawali '/', berarti absolute
    let absolutePath = targetPath.startsWith("/")
        ? targetPath
        : (cwd === "/" ? "/" + targetPath : cwd + "/" + targetPath);

    // 2. Normalisasi (Bersihkan //, ./ dan ..)
    const parts = absolutePath.split("/").filter(p => p.length > 0 && p !== ".");
    const stack: string[] = [];

    for (const part of parts) {
        if (part === "..") {
            stack.pop();
        } else {
            stack.push(part);
        }
    }

    return "/" + stack.join("/");
}
```

MountManager.resolve()

```
public resolve(path: string): {
    vfs: IVFS;
    relativePath: string;
    mountPoint: string;
} {
    // Normalize: robustly handle //, ./ and .. using PathResolver
    const normalized = PathResolver.resolve("/", path);

    for (const m of this.mounts) {
        // Case 1: Exact match
        if (normalized === m.vfsPath) {
            return { vfs: m.vfs, relativePath: "/", mountPoint: m.vfsPath };
        }

        // Case 2: Sub-path match
        // Special handling for root mount point "/"
        const prefix = m.vfsPath === "/" ? "/" : m.vfsPath + "/";
        if (normalized.startsWith(prefix)) {
            let relative = normalized.substring(
                m.vfsPath === "/" ? 0 : m.vfsPath.length,
            );
            if (!relative.startsWith("/")) relative = "/" + relative;
            return { vfs: m.vfs, relativePath: relative, mountPoint: m.vfsPath };
        }
    }

    throw new Error(`Path not found in any mounted filesystem: ${path}`);
}
```

MountManager.mount()

```
public mount(
    vfsPath: string,
    vfs: IVFS,
    type: string = "bkfs",
    source: string = "system.db",
    readOnly: boolean = false,
    uid?: number,
    gid?: number,
) {
    // Normalisasi path menggunakan PathResolver agar konsisten
```



```

const cleanPath = PathResolver.resolve("/", vfsPath);

// Cek apakah sudah ada
const existing = this.mounts.find((m) => m.vfsPath === cleanPath);
if (existing) {
  this.logger.warn(`Mount point ${cleanPath} already exists. Replacing...`);
  existing.vfs = vfs;
  existing.type = type;
  existing.source = source;
  existing.readOnly = readOnly;
  existing.uid = uid;
  existing.gid = gid;
} else {
  this.mounts.push({
    vfsPath: cleanPath,
    vfs,
    type,
    source,
    readOnly,
    uid,
    gid,
  });
  // Sort by path length descending so longest match wins
  this.mounts.sort((a, b) => b.vfsPath.length - a.vfsPath.length);
}

this.logger.info(
  `Mounted ${type} filesystem at ${cleanPath} from ${source} (${readOnly ? "ro" : "rw"})`,
);
}

```

[!NOTE] `mount()` men-sort hanya saat entry **baru** ditambahkan. Jika path sudah ada, `mount()` mengganti backend-nya tanpa menyentuh urutan sortir.

Root mount di initializeSubsystems()

```

this.bkfs = new BKFS(cfg.kernel.database);
this.mountManager.mount("/", this.bkfs, "bkfs", cfg.kernel.database, false);

```

processFstab() — loop pemrosesan fstab

```

private async processFstab() {
  if (!this.bkfs) return;

  const fstabPath = "/etc/fstab.json";
  if (!this.bkfs.exists(fstabPath)) return;

  try {
    const content = this.bkfs.read(fstabPath);
    if (!content) return;

    const entries = JSON.parse(content) as any[];
    for (const entry of entries) {
      const { vfsPath, hostPath, type, readOnly, uid, gid, active, mode } =
        entry;

      // Skip if explicitly marked inactive (default: active = true)
      if (active === false) {
        this.bootLogStart(`FSTAB: Skipping ${vfsPath} (inactive)`);
        this.bootLogEnd(true);
        continue;
      }

      // Ensure mount point exists with correct ownership & permissions
      const dirMode = mode ?? 0o755;
      if (!this.bkfs.exists(vfsPath)) {
        this.bkfs.mkdir(vfsPath, uid ?? 0, gid ?? 0, dirMode);
      }
    }
  }
}

```

```

    } else {
      if (uid !== undefined || gid !== undefined) {
        // Re-apply ownership if dir already existed (e.g. created by ensureDefaultAuth)
        this.bkfs.chown(vfsPath, uid ?? 0, gid ?? 0);
      }
      if (mode !== undefined) {
        this.bkfs.chmod(vfsPath, dirMode);
      }
    }
  }

  let driver: IVFS;
  if (type === "bkfs") {
    driver = new BKFS(
      path.resolve(process.cwd(), hostPath),
      readOnly || false,
      uid,
      gid,
      dirMode,
    );
  } else if (type === "ramfs") {
    // RamFS tidak butuh hostPath — murni di RAM
    const label = vfsPath.replace(/\\/g, "_").replace(/^_/, "");
    driver = new RamFS(label, uid, gid, dirMode);
  } else {
    driver = new HostVFS(hostPath, readOnly || false, uid, gid, dirMode);
  }

  this.bootLogStart(`FSTAB: Mounting ${vfsPath} (${type})`);
  this.mountManager.mount(
    vfsPath,
    driver,
    type,
    hostPath,
    readOnly || false,
    uid,
    gid,
  );
  this.bootLogEnd(true);
}
} catch (e: any) {
  this.bootLog(`FSTAB: Error processing fstab: ${e.message}`, false);
}
}
}

```

Latihan / Praktik

1. Jalankan `mount` (atau `cat /proc/mounts` jika ada) — amati daftar mount point dan urutannya.
2. Tulis file di `/tmp/x` lalu reboot — apakah file masih ada? (RamFS = volatile)
3. Baca `src/kernel/Kernel.ts` — cari `initializeSubsystems()` (root mount) dan `processFstab()` (fstab).
4. Hitung hasil `MountManager.resolve()` untuk `/mnt/shared/./sbak/x` sebelum menjalankannya. Cocokkan dengan perilaku nyata.
5. Ubah `resolve()` agar log path yang di-resolve, lalu buka file dari shell.

Referensi

- `wiki/Virtual-File-System.md` — lapisan VFS
- `wiki/course/00-overview.md` §4.4
- `src/kernel/MountManager.ts`, `src/common/PathResolver.ts`
- `src/kernel/Kernel.ts` — `initializeSubsystems()` & `processFstab()`
- `src/mirror/etc/fstab.json` — daftar mount default

Modul 07 — selesai. Bagian II tuntas. Lanjut ke [Modul 08 — VFS](#).

RFC-TSIX-EDU-002 | Modul kedelapan kurikulum TSIX. Memahami lapisan Virtual File System: satu kontrak `IVFS`, tiga backend (BKFS/RamFS/HostVFS), dan chunked I/O yang dieksekusi dalam SQL.

Kunci desain: **IVFS adalah satu kontrak, banyak backend**. Syscall tidak peduli apakah file berada di SQLite, RAM, atau folder host — ia hanya memanggil `vfs.read()`. Ini memungkinkan "swap backend" tanpa mengubah kernel.

Tujuan Pembelajaran

- ☐ Menyebutkan metode inti kontrak `IVFS`
- ☐ Menjelaskan perbedaan BKFS, RamFS, HostVFS
- ☐ Menjelaskan mengapa chunked I/O dieksekusi dalam SQL (SUBSTR/CONCAT)
- ☐ Menjelaskan struktur tabel `vnodes` di BKFS
- ☐ Menjelaskan peran `getSize` yang tidak fetch konten
- ☐ Menjelaskan alur baca satu file: syscall `OPEN` → `VFS.read` → SQL

Konsep Inti

Kontrak IVFS

```
export interface IVFS {
  ls(path: string): any[];
  mkdir(path: string, uid?: number, gid?: number, mode?: number): boolean;
  read(path: string): string | null;
  touch(path: string, content?: string, uid?: number, gid?: number, mode?: number): boolean;
  stat(path: string): any;
  chmod(path: string, mode: number): boolean;
  chown(path: string, uid: number, gid: number): boolean;
  unlink(path: string): boolean;
  rmdir(path: string): boolean;
  exists(path: string, type?: VNodeType): boolean;
  getUsage(): Promise<{ size: number, files: number, dirs: number, diskSize?: number }>;
  append(path: string, content: string): boolean;

  // --- Chunked I/O (Progress-aware, untuk file besar) ---
  /** Membaca sebagian konten file (offset-based, return null jika di luar range) */
  readChunk(path: string, offset: number, length: number): string | null;
  /** Menulis (replace) sebagian konten file di offset tertentu */
  writeChunk(path: string, chunk: string, offset: number): boolean;
  /** Mendapatkan ukuran file dalam byte, atau -1 jika tidak ditemukan */
  getSize(path: string): number;
}
```

Tiga Backend — Perbandingan

Backend	Sumber	Persistensi	Kecepatan	Kasus pakai
BKFS	SQLite <code>system.db</code> , tabel <code>vnodes</code>	✅ persisten (disk)	sedang; chunked I/O dioptimalkan di dalam SQL	root filesystem <code>/</code> , backup <code>/mnt/sbak</code>
RamFS	memori (pohon <code>VNode</code>)	❌ volatile (hilang saat restart)	⚡ sangat cepat	<code>/tmp</code> (data sementara)
HostVFS	folder nyata di host (<code>fs</code> Node)	✅ persisten (file host)	sedang (syscall host)	<code>/mnt/shared</code> (bridge + anti-escape)

Kapan backend dipakai ditentukan oleh `MountManager` saat boot (lihat `src/mirror/etc/fstab.json`):

Mount point	Type	Source	Keterangan
/	bkfs	system.db	root filesystem persisten
/tmp	ramfs	RAM	volatile, mode 1023 = 0o1777 (sticky)
/mnt/shared	host	folder shared	bridge ke host, uid 1000
/mnt/sbak	bkfs	systembak.db	backup root di file SQLite terpisah

[!NOTE] **"Swap backend" tanpa ubah kernel** Kernel hanya memegang referensi IVFS. Karena semua backend mengimplementasi kontrak yang sama, mengganti penyimpanan (mis. /tmp dari RamFS ke BKFS) cukup dengan mengubah entri fstab.json — syscall tidak berubah sama sekali.

 Routing VFS: app → syscall → MountManager → BKFS/RamFS/HostVFS Sumber: [wiki/diagram/Virtual-File-System-1.mmd](#)

BKFS: struktur vnodes

File di BKFS disimpan sebagai **baris** di tabel `vnodes` — struktur tree dengan `parent_id` menunjuk ke baris induknya (NULL untuk root /):

```
CREATE TABLE IF NOT EXISTS vnodes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  parent_id INTEGER,          -- FK ke id parent (NULL untuk root "/")
  name TEXT NOT NULL,
  type TEXT NOT NULL,        -- 'FILE' | 'DIRECTORY'
  content TEXT,              -- isi file (NULL untuk direktori)
  size INTEGER DEFAULT 0,
  uid INTEGER DEFAULT 0,     -- User ID (0 = root)
  gid INTEGER DEFAULT 0,     -- Group ID (0 = root)
  mode INTEGER DEFAULT 420,  -- Permission desimal (octal 644 = 420, 755 = 493)
  created_at INTEGER,
  modified_at INTEGER,
  FOREIGN KEY (parent_id) REFERENCES vnodes(id),
  UNIQUE(parent_id, name)    -- satu nama per folder induk
);
```

Catatan penting:

- **uid/gid/mode disimpan sebagai desimal**, bukan string octal. Contoh: mode 420 = 0o644, 493 = 0o755. Root / disisipkan saat init dengan mode 493; /tmp di fstab.json memakai 1023 = 0o1777 (sticky bit).
- **Navigasi path** dilakukan baris-per-baris: setiap segmen dicari dengan `SELECT id FROM vnodes WHERE name = ? AND parent_id = ? AND type = 'DIRECTORY'`, lalu `parent_id` bergeser ke id hasil. Ini terjadi di `getNodeId()` / `getNodeIdAndSize()`.
- **UNIQUE(parent_id, name)** mencegah duplikasi nama di folder yang sama. Schema ini juga migrasi DB lama: kolom `uid/gid/mode/size/modified_at` ditambahkan via `ALTER TABLE` jika belum ada.

Chunked I/O dalam SQL — dan kenapa penting

Trik utama: **jangan pernah menarik konten penuh ke memori JS**. Untuk file besar, membaca utuh sekali jalan mahal: memori terpakai dan UI terasa "hang" saat menunggu. Chunked I/O memecahnya menjadi potongan kecil (`offset + length`) — dipakai untuk progress-aware read/write, streaming, dan OTA firmware.

Semua pemotongan dilakukan **di dalam SQLite**:

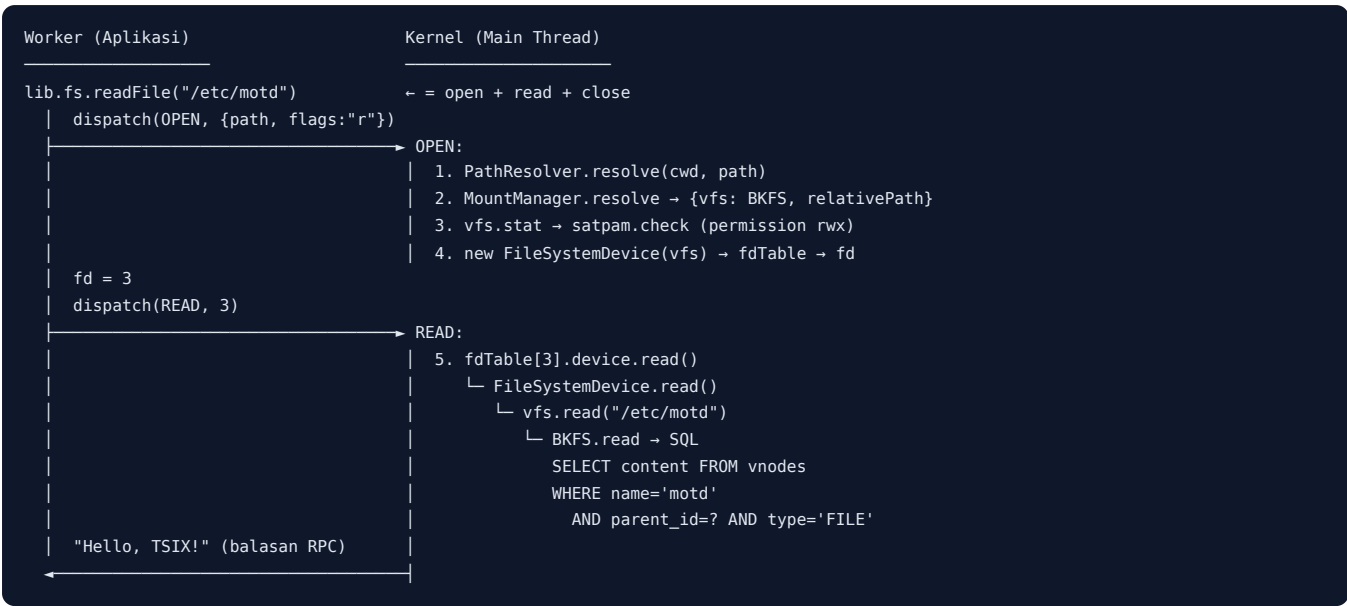
- `readChunk` → `SELECT SUBSTR(content, ? + 1, ?)` — hanya byte yang diminta yang dibaca.
- `writeChunk sequential append (offset >= currentSize)` → simple `CONCAT ( IFNULL(content, '') || ? )` — **sangat cepat**, tidak menulis ulang konten.
- `writeChunk random write di tengah` → `SUBSTR(content, 1, ?) || ? || SUBSTR(content, ? + 1)` — menempa bagian tertentu.

- `getSize` → baca kolom `size` saja (via `getNodeIdAndSize`), **tanpa fetch konten** — murah untuk cek progress sebelum `readChunk`.

[!IMPORTANT] **Kenapa SUBSTR(content, ? + 1, ?) ?** SQLite SUBSTR memakai indeks 1-based. Untuk membaca dari byte offset (0-based di API), parameter pertama harus `offset + 1`. Contoh: `readChunk(path, 2, 4) → SUBSTR(content, 3, 4)`.

Alur / Cara Kerja

Semua akses file melewati satu jalan: **syscall → MountManager → IVFS → backend**. Berikut alur lengkap membaca `/etc/motd` dari aplikasi (Worker) sampai SQLite:



Langkah demi langkah:

#	Lapisan	Yang terjadi
1	UserLib (Worker)	<code>lib.fs.readFile('/etc/motd')</code> → <code>open</code> → kirim syscall <code>OPEN</code> via <code>postMessage</code> (RPC).
2	Syscall <code>OPEN</code>	<code>PathResolver.resolve(pcb.cwd, path)</code> menormalkan path (<code>//</code> , <code>.</code> , <code>..</code>) → absolut.
3	<code>MountManager.resolve()</code>	Cari mount point prefix terpanjang (<code>/</code> → <code>BKFS</code>) → <code>{ vfs, relativePath, mountPoint }</code> .
4	PermissionManager	<code>vfs.stat()</code> ambil metadata (uid/gid/mode) → <code>satpam.check()</code> verifikasi izin baca.
5	FD Table	File dibungkus <code>FileSystemDevice</code> , <code>setPath(relativePath, flags)</code> , masuk <code>pcb.fdTable</code> → <code>fd</code> dikembalikan.
6	Syscall <code>READ</code>	<code>pcb.fdTable[fd].device.read()</code> memanggil driver.
7	<code>FileSystemDevice.read()</code>	Delegasi polimorfik: <code>vfs.read(this.currentPath)</code> — kernel tak peduli backendnya.
8	<code>BKFS.read()</code>	Navigasi tree per segmen di SQL → <code>SELECT content ...</code> → string dikirim balik via RPC.

[!TIP] **Path relatif vs absolut** Syscall menerima path relatif terhadap `pcb.cwd`; `PathResolver.resolve()` menormalkannya dulu sebelum `MountManager`. Jadi `/etc/../etc/motd` tetap membuka file yang sama.

Varian file besar — syscall `READ_CHUNK`:

Alih-alih `READ` (yang menarik seluruh konten ke memori), aplikasi memakai `lib.fs.readChunk(path, offset, length)`:

```
lib.fs.readChunk("/firmware.bin", 65536, 4096)
→ dispatch(READ_CHUNK, {path, offset, length})
→ MountManager.resolve → vfs.stat → satpam.check(READ)
→ vfs.readChunk(relativePath, offset, length)
  └─ BKFS: SELECT SUBSTR(content, ? + 1, ?) ... -- hanya byte yang diminta
```

Hasilnya: memori JS tidak pernah menampung file utuh — pemotongan terjadi di dalam SQLite.

Kode Sumber

File	Peran
src/vfs/IVFS.ts	Kontrak filesystem
src/vfs/BKFS.ts	Backend SQLite + chunked I/O
src/vfs/RamFS.ts	Backend in-memory
src/vfs/HostVFS.ts	Backend host (anti-escape)

Snippet (level kode)

BKFS.read — baca konten penuh

```
public read(path: string): string | null {
  const parts = path
    .split("/")
    .filter((p) => p.length > 0 && p !== "." && p !== "..");
  const fileName = parts.pop();
  if (!fileName) return null;

  let parentId = this.getRootId();
  for (const part of parts) {
    let node = this.db
      .prepare(
        "SELECT id FROM vnodes WHERE name = ? AND parent_id = ? AND type = 'DIRECTORY'",
      )
      .get(part, parentId) as { id: number } | undefined;
    if (!node) return null;
    parentId = node.id;
  }

  const file = this.db
    .prepare(
      "SELECT content FROM vnodes WHERE name = ? AND parent_id = ? AND type = 'FILE'",
    )
    .get(fileName, parentId) as { content: string } | undefined;

  return file ? file.content : null;
}
```

Poin kunci: path dipecah per segmen, lalu tiap segmen dicari dengan `name + parent_id` (tree). Query terakhir menargetkan `type = 'FILE'` — direktori tidak ikut terbaca.

BKFS.touch — buat / timpa file (upsert)

```
public touch(
  path: string,
  content: string = "",
  uid: number = 0,
  gid: number = 0,
  mode: number = 420,
): boolean {
  if (this.readOnly) throw new Error("Read-only filesystem");
  const parts = path
```

```

    .split("/")
    .filter((p) => p.length > 0 && p !== "." && p !== "..");
const fileName = parts.pop();
if (!fileName) return false;

let parentId = this.getRootId();
// Navigasi ke folder tujuan
for (const part of parts) {
    let node = this.db
        .prepare(
            "SELECT id FROM vnodes WHERE name = ? AND parent_id = ? AND type = 'DIRECTORY'",
        )
        .get(part, parentId) as { id: number } | undefined;
    if (!node) return false;
    parentId = node.id;
}

// Cek apakah file sudah ada
const existing = this.db
    .prepare(
        "SELECT id FROM vnodes WHERE name = ? AND parent_id = ? AND type = 'FILE'",
    )
    .get(fileName, parentId);

if (existing) {
    this.db
        .prepare(
            "UPDATE vnodes SET content = ?, size = ?, modified_at = ? WHERE id = ?",
        )
        .run(content, content.length, Date.now(), (existing as any).id);
    return true;
}

const now = Date.now();
this.db
    .prepare(
        "INSERT INTO vnodes (parent_id, name, type, content, size, uid, gid, mode, created_at, modified_at) VALUES (?, ?, 'FILE', ?, ?, ?, ?, ?, ?, ?)",
    )
    .run(
        parentId,
        fileName,
        content,
        content.length,
        uid,
        gid,
        mode,
        now,
        now,
    );

this.logger.debug(
    `File created/updated in BKFS: ${fileName} (Mode: ${mode.toString(8)})`,
);
return true;
}

```

Poin kunci: `touch` adalah **upsert** — file yang sudah ada di-`UPDATE` (konten + `size`), yang belum ada di-`INSERT`. Mode default `420` = `0o644`. Syscall `OPEN` dengan flag `w` memakai ini untuk membuat/truncate file.

readChunk (BKFS) — potongan konten

```

public readChunk(path: string, offset: number, length: number): string | null {
    const nodeId = this.getNodeId(path);
    if (nodeId < 0) return null;

    const row = this.db
        .prepare(
            "SELECT SUBSTR(content, ? + 1, ?) as chunk FROM vnodes WHERE id = ? AND type = 'FILE'",
        )
        .get(offset, length, nodeId) as { chunk: string | null } | undefined;
}

```

```
return row?.chunk ?? null;
}
```

writeChunk (BKFS) — dua jalur

```
if (offset >= currentSize) {
  // Sequential append — simple CONCAT (paling cepat)
  UPDATE vnodes SET content = IFNULL(content, '') || ?, size = ?, modified_at = ? WHERE id = ?;
} else {
  // Random write di tengah — SUBSTR + CONCAT
  UPDATE vnodes SET
    content = SUBSTR(content, 1, ?) || ? || SUBSTR(content, ? + 1),
    size = ?, modified_at = ? WHERE id = ?;
}
```

Latihan / Praktik

1. Buka `system.db` dengan SQLite CLI: `.schema vnodes` — amati struktur tabel.
2. Baca `src/vfs/HostVFS.ts` — cari `toHostPath()` dan cek anti-escape.
3. Tulis file besar (misal 1MB) lalu panggil `readChunk` dengan offset acak — bandingkan kecepatan vs `read` penuh.
4. Baca `src/vfs/IVFS.ts` — kenali seluruh kontrak yang harus dipenuhi backend baru.
5. Letakkan breakpoint di `src/kernel/Syscalls.ts` case `OPEN` dan `READ`, lalu baca `/etc/motd` dari shell — telusuri sampai `BKFS.read` dan amati query SQL-nya.

Referensi

- [wiki/Virtual-File-System.md](#) — lapisan VFS lengkap
- [wiki/course/00-overview.md §5.1](#)
- `src/vfs/IVFS.ts`, `src/vfs/BKFS.ts`, `src/vfs/RamFS.ts`, `src/vfs/HostVFS.ts`
- `src/kernel/MountManager.ts` — pemilihan backend dari path (prefix terpanjang)
- `src/kernel/Syscalls.ts` — syscall `OPEN`, `READ`, `READ_CHUNK`, `WRITE_CHUNK`, `GET_SIZE`
- `src/kernel/devices/FileSystemDevice.ts` — jembatan `FD` → `IVFS`
- `src/mirror/etc/fstab.json` — konfigurasi mount saat boot

Modul 08 — selesai. Lanjut ke [Modul 09 — FD Table & File Syscalls](#).

RFC-TSIX-EDU-002 | Modul kesembilan kurikulum TSIX. Memahami "everything is a file" di level FD table: bagaimana `open` → `FDEntry` → device, dan bagaimana file biasa dibungkus `FileSystemDevice`.

Di TSIX, **file descriptor bukan index ke array file** — ia index ke `FDEntry { device, context, flags }` yang menunjuk ke objek `IDevice` apa pun. File biasa, TTY, pipe, socket — semuanya `IDevice`.

Tujuan Pembelajaran

- ☐ Menjelaskan struktur `FDEntry { device, context, flags }` dan `fdTable`
- ☐ Menjelaskan alur `OPEN`: `resolve path` → cek permission → pilih device → `fdTable.push`
- ☐ Menjelaskan bagaimana `READ/WRITE` me-dispatch ke `entry.device`
- ☐ Menjelaskan "everything is a file": file biasa vs `/dev/*`
- ☐ Menjelaskan pipe refcount via `ioctl ( INC_REF / DEC_REF )`
- ☐ Menjelaskan cleanup FD saat proses exit

Konsep Inti

FDEntry

```
export interface FDEntry {
  device: IDevice; // objek device nyata
  context: any;    // state per-FD
  flags?: string;  // "r" | "w" | "a" | "+"
}
```

`fdTable` ada di PCB — setiap proses punya array `(FDEntry | null)[]`. FD 0/1/2 (`stdin/stdout/stderr`) diisi saat spawn dari device TTY.

Everything is a File

Yang dibuka	Device di FD table
File biasa (<code>/etc/passwd</code>)	<code>FileSystemDevice</code> (bungkus IVFS)
<code>/dev/tty1</code>	<code>TTYDevice</code>
<code>/dev/null</code>	<code>NullDevice</code>
Pipe (syscall <code>PIPE</code>)	<code>PipeDevice</code>
Socket (syscall <code>SOCKET</code>)	<code>SocketDevice</code>

Tidak ada perbedaan *tipe* antara file dan perangkat di FD table — keduanya hanyalah objek yang mengimplementasikan `IDevice ( read(), write(), ioctl() )`. Buka file biasa → kernel membuat `FileSystemDevice` baru yang membungkus backend IVFS hasil `mountManager.resolve()`. Buka `/dev/*` → kernel mengambil device yang sudah terdaftar di `kernel.devices` (TTY, null, stdin, dll). Inilah inti "everything is a file": `read/write` selalu polimorfik ke `entry.device`.

FD Standar (0/1/2)

Saat proses dibuat (`Scheduler.createProcess`) atau di-EXEC, kernel mengisi tiga FD pertama dari device stdio. Default di EXEC adalah device global `kernel.devices.stdin/stdout/stderr`; jika proses berjalan di sebuah TTY (`pcb.ttyId`), ketiganya diarahkan ke `tty{N}` yang sama.

FD	Nama	flags	Device (umum)
0	stdin	"r"	TTYDevice (ttyN) saat proses punya TTY; fallback KeyboardDevice (kernel.devices.stdin)
1	stdout	"w"	TTYDevice (kernel.devices.stdout = tty1)
2	stderr	"w"	TTYDevice (kernel.devices.stderr = tty1)

[!NOTE] flags FD 0 adalah "r" (baca), sedangkan FD 1 dan 2 adalah "w" (tuliskan). Inilah yang membuat WRITE bisa menolak menulis ke FD yang tidak dibuka untuk menulis ("Bad File Descriptor").

Pipe refcount

Pipe punya refcount pada sisi read dan write. EOF terjadi saat `writeRefs == 0`. Kernel mengelola refcount lewat `ioctl` device — nomor perintah:

ioctl cmd	Makna	Dipanggil dari
10	INC_READ_REF	OPEN saat flags mengandung "r"
20	INC_WRITE_REF	OPEN saat flags mengandung "w" / "a"
11	DEC_READ_REF	CLOSE saat flags mengandung "r"
21	DEC_WRITE_REF	CLOSE saat flags mengandung "w" / "a"

Setiap `open` menambah, setiap `close` mengurangi. `FileSystemDevice` tidak memakai refcount — `ioctl()`-nya mengembalikan `-1` (no-op). Refcount hanya bermakna untuk device nyata seperti pipe, TTY, dan serial.

Alur / Cara Kerja

OPEN: dari path menuju FD

Langkah pada case `SyscallCode.OPEN` (`src/kernel/Syscalls.ts`):

- Resolve path** — `PathResolver.resolve(pcb.cwd, args)` mengubah path (relatif/absolut) menjadi absolut.
- Resolve VFS** — `mountManager.resolve(absolutePath) → { vfs, relativePath }`. Backend dipilih dari prefix mount terpanjang (`/ → BKFS`, `/tmp → RamFS`, `/mnt/* → HostVFS`).
- Cek permission** — `vfs.stat(relativePath)` untuk mendapat node. `requiredPerm = WRITE` jika flags mengandung "w" / "+", selain itu `READ`.
 - Node ada → `satpam.check(pcb, node, requiredPerm)`; gagal → `Permission Denied`.
 - Node tidak ada → flags "r" → `File not found`; flags "w" / "+" → cek permission direktori induk, lalu `vfs.touch` membuat file (mode `0644`), dan truncate bila flags "w" tanpa "a".
- Pilih device:**
 - Path `/dev/*` → cari `kernel.devices[devName]` (alias: `screen/console/stdout/fb0` dan `keyboard/stdin`; `tty → tty{pcb.ttyId}`). Refcount dinaikkan via `ioctl INC_*_REF`, lalu `device.open()` dipanggil secara lazy.
 - Path biasa → `new FileSystemDevice(vfs) + setPath(relativePath, flags)`.
- Masukkan ke tabel** — `const fd = pcb.fdTable.length; pcb.fdTable.push({ device, context, flags }); return fd;`. `fd` adalah index baru di array, bukan angka bebas. Untuk file biasa, `context` diisi `relativePath`; untuk `/dev/*`, `context` diisi path absolut.

READ / WRITE: dispatch ke entry.device

READ dan WRITE tidak peduli jenis device. Mereka mengambil `entry = pcb.fdTable[fd]`, lalu memanggil metode polimorfik pada `entry.device`:

- `READ(fd) → entry.device.read()` — TTY membaca buffer, file membaca VFS.
- `WRITE(fd, content) → cek entry.flags` (harus mengandung "w" / "a" / "+", selain itu `Bad File Descriptor`), lalu `entry.device.write(content)`.

Ada rute khusus IPC: `WRITE` dengan `{ pid, content }` menulis ke `fdTable[0]` proses target (TTY → injeksi via `ioctl(0x2001)`); `READ` dengan `{ pid }` membaca dari `fdTable[1]` proses target (TTY → `ioctl(0x2002)`).

Walkthrough: membuka `/etc/passwd`

Trace satu proses shell yang mengeksekusi `cat /etc/passwd`:

```
cat /etc/passwd
|
| OPEN("/etc/passwd", "r")
v
+-----+
| case OPEN (kernel) |
| 1. PathResolver.resolve(cwd, path) |
|   -> "/etc/passwd" (absolut) |
| 2. mountManager.resolve("/etc/passwd") |
|   -> { vfs: BKFS, relativePath: "etc/passwd" } |
| 3. vfs.stat("etc/passwd") -> node ada |
|   satpam.check(pcb, node, READ) -> lolos |
| 4. const fsDriver = new FileSystemDevice(BKFS) |
|   fsDriver.setPath("etc/passwd", "r") |
| 5. const fd = pcb.fdTable.length // misal = 3 |
|   pcb.fdTable.push({ device: fsDriver, |
|                     context: "etc/passwd", |
|                     flags: "r" }) |
| return fd; // fd = 3 |
+-----+
|
| fd = 3, lalu READ(3)
v
+-----+
| case READ |
| const entry = pcb.fdTable[3]; |
| return entry.device.read(); |
|   -> FileSystemDevice.read() |
|   -> vfs.read("etc/passwd") // BKFS: SQL SELECT |
+-----+
|
| isi file /etc/passwd
v
| CLOSE(3)
v
+-----+
| case CLOSE |
| device.ioctl(11 / 21) // DEC_REF; FileSystemDevice no-op |
| pcb.fdTable[3] = null; |
+-----+
```

[!NOTE] `FileSystemDevice` mengembalikan `null` saat `currentPath` kosong dan `false` saat menulis tanpa path — ini "fail safe" sebelum menyentuh VFS.

Cleanup saat exit

Kernel memasang global exit hook di konstruktor `SyscallHandler` (`setOnProcessExit`). Saat proses EXITED:

1. **Tutup semua FD** — iterasi `pcb.fdTable`; tiap entry yang tidak `null` di-dispatch `CLOSE` (memicu `DEC_*_REF` pada device yang mendukung). Error saat cleanup massal diabaikan.
2. **Jaring pengaman port** — `portManager.releasePortsByPid(pid)` membebaskan semua port MQTNL milik proses (antisipasi socket yang lolos dari `CLOSE`).
3. **GUI cleanup** — `guiRegistry.destroyAllForPid(pid)` menghancurkan semua window milik proses dan mengirim `GUI_REQ DESTROY_WINDOW` ke daemon GUI (`gued`).

Kode Sumber

File	Peran
src/kernel/Scheduler.ts	Definisikan <code>FDEntry</code> + <code>fdTable</code> di PCB
src/kernel/devices/FileSystemDevice.ts	Jembatan file ↔ IVFS
src/kernel/devices/PipeDevice.ts	Pipe + refcount
src/kernel/Syscalls.ts	OPEN/READ/WRITE/CLOSE/PIPE + cleanup

Snippet (level kode)

OPEN — resolve, permission, pilih device, `fdTable.push`

```
case SyscallCode.OPEN: {
  // ... PathResolver.resolve + mountManager.resolve (lihat "Alur / Cara Kerja") ...

  // 1. Permission Check
  const requiredPerm =
    flags.includes("w") || flags.includes("+")
      ? Permission.WRITE
      : Permission.READ;

  // Existence Check
  const node = vfs.stat(relativePath);

  if (node) {
    // Check File Permission
    if (!this.satpam.check(pcb, node, requiredPerm)) {
      throw new Error(
        `Permission Denied: Cannot open ${absoluteOpenPath} for ${Permission[requiredPerm]}`,
      );
    }
  } else {
    // ... "File not found", cek parent-dir, touch buat file, truncate ...
  }

  // ... branch /dev/* -> kernel.devices[devName] + INC_REF + lazy open ...

  // File biasa -> bungkus FileSystemDevice
  const fsDriver = new FileSystemDevice(vfs);
  fsDriver.setPath(relativePath, flags);

  // TRUNCATE: If opened with 'w' (and not 'a'), we must clear content first
  if (
    flags &&
    flags.includes("w") &&
    !flags.includes("a") &&
    !flags.includes("+")
  ) {
    vfs.touch(relativePath, "", pcb.uid, pcb.gid, 420);
  }

  const fd = pcb.fdTable.length;
  pcb.fdTable.push({ device: fsDriver, context: relativePath, flags });
  return fd;
}
```

[!IMPORTANT] Perhatikan dua detail: (1) `fd` adalah `pcb.fdTable.length` — index baru di akhir array, bukan angka bebas; (2) untuk file biasa `context` diisi `relativePath`, sedangkan untuk `/dev/*` diisi path absolut. `context` bebas diinterpretasi device.

READ / WRITE — dispatch ke `entry.device`

```
// WRITE
case SyscallCode.WRITE: {
  // ... route khusus pid -> targetPcb.fdTable[0] (TTY: ioctl(0x2001)) ...

  const { fd, content } = args;
  const entry = pcb.fdTable[fd];
  if (!entry) return false;

  // Check if FD was opened with Write permission
  const flags = entry.flags || "r";
  if (
    !flags.includes("w") &&
    !flags.includes("a") &&
    !flags.includes("+")
  ) {
    throw new Error("Bad File Descriptor: Not open for writing");
  }

  return entry.device.write(content);
}

// READ
case SyscallCode.READ: {
  // ... route khusus pid -> targetPcb.fdTable[1] (TTY: ioctl(0x2002)) ...

  const fd = args as number;
  const entry = pcb.fdTable[fd];
  if (!entry) throw new Error(`FD NOT FOUND: ${fd}`);
  return entry.device.read();
}
}
```

FileSystemDevice — jembatan file ↔ VFS

```
export class FileSystemDevice implements IDevice {
  name = "FileSystem";
  private vfs: IVFS;
  private currentPath: string = "";
  private flags: string = "r";

  constructor(vfs: IVFS) {
    this.vfs = vfs;
  }

  public setPath(path: string, flags: string = "r") {
    this.currentPath = path;
    this.flags = flags;
  }

  read() {
    if (!this.currentPath) return null;
    return this.vfs.read(this.currentPath);
  }

  write(data: any) {
    if (!this.currentPath) return false;

    const isAppend = this.flags.includes("a");
    const isWrite = this.flags.includes("w");
    const isUpdate = this.flags.includes("+");

    if (isAppend || isWrite || isUpdate) {
      return this.vfs.append(this.currentPath, data);
    }

    // Fallback: Default Overwrite (Backward Compatibility)
    return this.vfs.touch(this.currentPath, data);
  }

  ioctl(cmd: number, arg: any) { return -1; }
}
```

Cleanup FD saat proses exit

```
this.scheduler.setOnProcessExit(async (pid) => {
  const pcb = this.scheduler.getProcess(pid);
  if (pcb) {
    // Tutup semua FD — trigger DEC_REF via dispatch CLOSE
    for (let fd = 0; fd < pcb.fdTable.length; fd++) {
      if (pcb.fdTable[fd]) {
        try {
          await this.dispatch(pid, SyscallCode.CLOSE, fd);
        } catch (e) {
          // Ignore errors during mass cleanup
        }
      }
    }
  }

  // === Force release semua port milik PID ini ===
  try {
    this.kernel.getPortManager().releasePortsByPid(pid);
  } catch (_) {}

  // --- GUI Cleanup ---
  const guiRegistry = this.kernel.guiRegistry;
  if (guiRegistry) {
    const ownedWindows = guiRegistry.destroyAllForPid(pid);
    if (ownedWindows.length > 0) {
      const gudedPid = guiRegistry.getDaemonPid();
      if (gudedPid !== null) {
        for (const wid of ownedWindows) {
          this.scheduler.sendEvent(gudedPid, "gui_request", {
            syscall: "GUI_REQ",
            pid,
            wid,
            action: GUIAction.DESTROY_WINDOW,
          } as IGUIPayload);
        }
      }
    }
  }
});
```

Latihan / Praktik

1. Baca `src/kernel/devices/PipeDevice.ts` — jelaskan kapan pipe mengembalikan EOF (hubungkan ke `writeRefs` dan `ioctl DEC_WRITE_REF`).
2. Dari shell, `echo hello > /tmp/x` lalu `cat /tmp/x` — trace syscall OPEN/WRITE/READ/CLOSE di log. Perhatikan fd yang dikembalikan OPEN dan device di belakangnya.
3. Baca `src/kernel/Syscalls.ts` — temukan implementasi OPEN dan bagaimana ia memilih device (`FileSystemDevice` vs `/dev/*`).
4. Baca `src/kernel/Scheduler.ts` `createProcess` — jelaskan bagaimana fd 0/1/2 (stdin/stdout/stderr) diisi dari `options.fds`.

Referensi

- `wiki/Virtual-File-System.md` — \$device model
- `wiki/course/00-overview.md` — \$5 VFS & Device Drivers
- `src/kernel/devices/FileSystemDevice.ts` — jembatan file ↔ IVFS
- `src/kernel/Syscalls.ts` — OPEN/READ/WRITE/CLOSE/EXIT cleanup
- `src/kernel/Scheduler.ts` — FDEntry + fdTable di PCB

RFC-TSIX-EDU-002 | Modul kesepuluh kurikulum TSIX. Memahami kontrak `IDevice`, plugin `aux-devices`, konfigurasi `udev-style`, dan tabel perangkat `/dev virtual`.

`/dev` di TSIX adalah **registry device di kernel** (`kernel.devices[xxx]`) — bukan `vnode filesystem`. Setiap perangkat adalah objek `IDevice` yang bisa dibuka seperti file.

Tujuan Pembelajaran

- ☐ Menjelaskan kontrak `IDevice` (`read/write/ioctl` + opsional `init/open/close/present`)
- ☐ Menjelaskan mengapa `/dev virtual` — bukan `vnode filesystem`
- ☐ Menjelaskan urutan registrasi device saat boot
- ☐ Menjelaskan plugin `aux-devices`: konvensi `export default + static autoRegister`
- ☐ Menjelaskan `applyDeviceConfigs()` (`udev-style` dari `sysconfig.json`)
- ☐ Menjelaskan `present()` untuk hotplug (`udev-like`)
- ☐ Menjelaskan dual transport `DbLib`: `/dev/mysql` vs daemon `mysqld`
- ☐ Menulis driver sendiri (checklist 10 langkah)

Konsep Inti

Kontrak `IDevice`

Semua device adalah objek yang mengimplementasikan `IDevice`. Kernel cukup memanggil tiga method wajib — `read`, `write`, `ioctl` — tanpa tahu isi di dalamnya. Ini abstraksi HAL: satu kontrak untuk semua hardware.

```
export interface IDevice {
  name: string;
  read(offset?: number, length?: number): any;
  write(data: any, offset?: number): boolean;
  ioctl(cmd: number, arg: any): any;

  init?(ctx: KContext): void; // Opsional: Untuk terima 'suntikan' dari Kernel
  open?(): boolean; // Opsional: Lazy open device hardware
  close?(): boolean; // Opsional: Close device hardware (with refcounting)

  /**
   * Opsional: Apakah hardware benar-benar tersedia SEKARANG?
   * Dipakai `ls /dev` untuk menyembunyikan device yang tidak present
   * (udev-like hotplug). Jika tidak diimplementasikan → selalu present.
   */
  present?(): boolean;

  uid?: number;
  gid?: number;
  mode?: number;
  disabled?: boolean;
}
```

`KContext` adalah suntikan utility dari kernel ke driver (diberikan lewat `init`):

```
export interface KContext {
  syslog: (message: string) => void;
}
```

Anggota	Wajib	Makna
name	✓	Nama driver; jadi nama path <code>/dev/<name></code> (huruf kecil)
<code>read(offset?, length?)</code>	✓	Baca data dari device
<code>write(data, offset?)</code>	✓	Tulis data; <code>boolean</code> sukses/gagal
<code>ioctl(cmd, arg)</code>	✓	Kontrol device (clear, raw mode, refcount, dst.)
<code>init(ctx)</code>	☐	Dipanggil sekali saat boot; <code>ctx.syslog()</code> untuk log
<code>open()</code>	☐	Lazy open saat device dibuka (mis. <code>SerialDevice</code>)
<code>close()</code>	☐	Tutup device dengan refcounting
<code>present()</code>	☐	Hotplug: <code>ls /dev</code> menyembunyikan device yang tidak present
uid/gid/mode	☐	Metadata izin (dipakai cek permission <code>OPEN</code>)
disabled	☐	<code>true</code> → dilewati plugin loader saat boot

/dev Virtual — Bukan Vnode

/dev **tidak** menyimpan device node di database VFS. Komentar kode `OPEN` menyebutnya eksplisit: *"we don't want to create device nodes in VFS database"*. Yang ada hanyalah direktori `/dev` (dibuat `bkfs.mkdir("/dev", 0, 0, 493)` saat boot) dan tabel `kernel.devices`.

Operasi	Penanganan di syscall
<code>ls /dev</code>	Khusus: enumerasi <code>Object.keys(kernel.devices)</code> , filter <code>present()</code> , laporkan metadata
<code>open("/dev/xxx")</code>	<code>devName = path.replace("/dev/", "") → kernel.devices[devName] → FDEntry {device, ...}</code>
<code>stat("/dev/xxx")</code>	Khusus: ambil metadata device (uid/gid/mode)
<code>chmod / chown /dev/*</code>	Root only; langsung set ke objek device
<code>/dev/</code> tanpa driver	Jatuh ke VFS node (jika ada) dengan cek izin biasa

Alias yang dikenali `OPEN` :

- `tty` → TTY proses pemanggil (`tty${pcb.ttyId || 1}`).
- `screen`, `console`, `stdout`, `fb0` → TTY aktif pemanggil (fallback `fb0`).
- `keyboard`, `stdin` → `kernel.devices.stdin`.

Cek permission `OPEN` memakai metadata device dengan **default** `0o600` (root only) jika `mode` tidak di-set:

```
const devPerm = {
  name: devName,
  uid: device.uid ?? 0,
  gid: device.gid ?? 0,
  mode: device.mode ?? 0o600, // Default: root only (rw-----)
};
```

Device standar I/O (`stdout` / `stdin` / `fb0` / `console` / `screen` / `keyboard`) **bypass** cek ini. Saat dibuka, kernel memanggil `ioctl(10)` (`INC_READ_REF`) / `ioctl(20)` (`INC_WRITE_REF`) untuk refcounting, lalu `device.open()` jika tersedia (lazy open). Gagal `open()` → error `Failed to open device /dev/xxx`.

Tabel perangkat /dev

/dev/	Class	Fungsi
<code>stdin</code>	<code>KeyboardDevice</code>	Input keyboard (cooked/raw)
<code>fb0/stdout/stderr</code>	<code>TTYDevice</code>	Output standar → TTY aktif
<code>tty1..tty32</code>	<code>TTYDevice</code>	Virtual console

/dev/	Class	Fungsi
null	NullDevice	Lubang hitam
smqtnl0/1	SimpleMQTNLDriver	Network interface MQTNL
randomdevice	RandomDevice	Angka acak (0-999 per baca)
mcp23017	MCP23017Device	GPIO I2C (autoRegister)
joystick	JoystickDevice	Gamepad USB HID; present() = hotplug
ttyUSB*	SerialDevice	Auto-detect serial
mysql (eksperimental)	MySQLDevice	Koneksi DB eksternal — disabled: true default
(virtual)	PipeDevice / SocketDevice	Instance runtime, bukan path

[!NOTE] MySQLDevice punya disabled: true di kode sumber — jadi /dev/mysql **tidak terdaftar secara default**. Aktifkan hanya saat transport device diperlukan. Lihat catatan dual transport di bawah.

Registrasi Device saat Boot

Kernel.boot() mendaftarkan device dalam urutan tetap:

```
// 1. TTYManager(32) → tty1..tty32 (TTYDevice)
// 2. Map device inti:
this.devices = {
  stdin: new KeyboardDevice(),
  fb0: ttysDevs.tty1,    // Alias fb0 ke TTY1
  stdout: ttysDevs.tty1, // Alias stdout ke TTY1
  stderr: ttysDevs.tty1, // Alias stderr ke TTY1
  null: new NullDevice(),
  ...ttysDevs,
};
// 3. Interface jaringan (cfg.network.interfaces) → SimpleMQTNLDriver
// 4. loadAuxDevices() → plugin dari folder aux-devices
// 5. SerialDeviceManager → auto-detect ttyUSB*
// 6. applyDeviceConfigs() → udev-style dari sysconfig.json
// 7. init() semua driver → suntikan KContext (syslog)
// 8. pastikan /dev ada di VFS (mkdir 493)
```

Urutannya penting: applyDeviceConfigs() harus jalan **setelah** semua device terdaftar, dan init() setelah konfigurasi diterapkan.

Plugin aux-devices & Konvensi autoRegister

Folder src/kernel/devices/aux-devices/ di-scan loadAuxDevices() saat boot. Dua konvensi:

1. **export default wajib** — loader membaca module.default || module. Tanpa default export, class tidak dikenali.
2. **static autoRegister(kernel) opsional** — untuk konfigurasi hardware platform-specific (mis. MCP23017 + bus I2C). Dipanggil setelah instance terdaftar.

applyDeviceConfigs (udev-style)

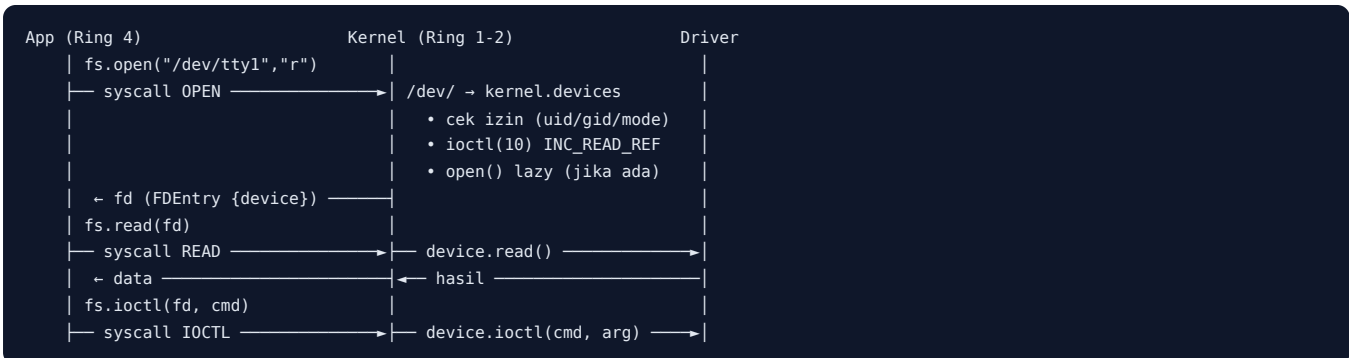
Blok devices di sysconfig.json → aturan "udev": set mode/uid/gid per nama device — persis aturan udev Linux.

```
{
  "devices": {
    "randomdevice": {
      "mode": 438
    }
  }
}
```

438 (decimal) = 00666 (rw-rw-rw-), diterapkan ke /dev/randomdevice. Lihat snippet `applyDeviceConfigs()`.

Alur / Cara Kerja

Alur akses device (read/write)



Alur registrasi saat boot

1. `TTYManager(32)` membuat 32 konsol → `TTYDevice tty1..32`.
2. Map inti `this.devices: stdin/fb0/stdout/stderr/null + tty1..32`.
3. Interface jaringan dari `cfg.network.interfaces` → `SimpleMQTNLDriver`.
4. `loadAuxDevices()`: scan `aux-devices/` → export default → register → `autoRegister`.
5. `SerialDeviceManager` mendeteksi `ttyUSB*`.
6. `applyDeviceConfigs()`: terapkan mode/uid/gid dari `sysconfig.json`.
7. Loop `init()` semua driver → suntik `KContext.syslog`.
8. Direktori `/dev` dijamin ada di VFS.

Alur `ls /dev` (hotplug udev-like)

```
ls /dev
→ syscall LS dengan path /dev
→ enumerasi kernel.devices
→ filter: device dengan present() hanya tampil jika present() true
→ laporan {name, type:"DEVICE", size:0, uid, gid, mode}
```

Contoh: `JoystickDevice.present()` mengembalikan `this.connected` — saat gamepad dicabut, `/dev/joystick` hilang dari `ls /dev`; saat dicolok lagi, muncul kembali.

Catatan Khusus: `MySQLDevice` & `DbLib` (Dual Transport)

[!IMPORTANT] `/dev/mysql` **bukan driver hardware**. `MySQLDevice` adalah **transport pertama** untuk integrasi database eksternal lewat model device — contoh perluasan HAL ke arah yang tidak biasa.

DbLib sudah **terimplementasi** (sub-library `UserLib`, pola `lib.fs/lib.net`) dengan **dual transport pluggable**:

- **Device** (`/dev/mysql`): syscall `DB_*` (67-69) → kernel → device → `mysql2`
- **Service daemon** (`mysqld`): syscall `DB_*` → kernel → daemon Ring 4 → `mysql2`

Kernel me-route **secara dinamis**: jika `mysqld` terdaftar (syscall `DB_SERVICE_REGISTER=70`) → ke daemon; jika tidak → ke device (fallback). Daemon membalas via `DB_SERVICE_REPLY=71`. Aplikasi hanya melihat `db.connect()/query()/disconnect()` — medium tak terlihat. Sandbox aman karena `mysql2` hanya disentuh kernel atau daemon privileged.

Kode syscall DB

Kode	Nama	Fungsi
67	DB_CONNECT	Buka koneksi ({host, user, password, database})
68	DB_QUERY	Eksekusi SQL (SELECT → rows; INSERT/UPDATE/DELETE → ResultSetHeader)
69	DB_DISCONNECT	Tutup koneksi
70	DB_SERVICE_REGISTER	Daemon <code>mysqld</code> mendaftar sebagai transport service
71	DB_SERVICE_REPLY	Daemon mengirim hasil {requestId, result} ke kernel

Alur routing dinamis

```

App (Ring 4)  lib.db.query(sql)                ← DbLib (@tsix/DbLib)
  ▼
UserLib.dispatch(DB_QUERY = 68)
  ▼
Kernel (Syscalls.ts)
| if (this.dbServicePid !== null)
| └─ YA → forwardDbRequest(pid, "query", sql)
|      → sendEvent(mysqld, "db_request") → daemon → mysql2
|      → DB_SERVICE_REPLY=71 (requestId, result) → resolve pending
| └─ TIDAK → /dev/mysql (MySQLDevice) → device.query(sql, pid) → mysql2
  ▼
resolve → App (rows)

```

Logika routing persis sama di setiap case `DB_CONNECT` / `DB_QUERY` / `DB_DISCONNECT`: cek `dbServicePid` dulu, baru fallback ke device. Saat daemon keluar, `Syscalls.ts` mereset `dbServicePid = null` sehingga route otomatis kembali ke device.

Detail implementasi

- `MySQLDevice` (`aux-devices/MySQLDevice.ts`) punya `disabled: true` — **tidak terdaftar secara default**.
- **Multi-instance**: koneksi disimpan per-PID (`Map<pid, Connection>`); tiap app punya koneksi sendiri, dua app bisa akses server/database berbeda paralel. Jalur raw ioctl tanpa pid memakai slot anonim (`ANON_PID = 0`): `ioctl(0x2001) connect / ioctl(0x2002) disconnect`.
- `release(pid)`: dipanggil kernel saat proses mati → koneksi ditutup paksa (anti-bocor).
- **Daemon `mysqld`** (`src/mirror/etc/mysqld/mysqld.ts`): Ring 4 privileged — allow-list modul host mencakup `mysql2` dan `mysql2/promise`.
- **App tidak pernah menyentuh `mysql2`** — hanya `lib.db.*` atau `import { db } from "@tsix/Application"`.

Snippet (level kode)

Semua snippet di bawah **salinan persis dari source** (kode adalah kebenaran).

Kontrak `IDevice` — `src/kernel/devices/IDevice.ts`

```

export interface IDevice {
  name: string;
  read(offset?: number, length?: number): any;
  write(data: any, offset?: number): boolean;
  ioctl(cmd: number, arg: any): any;

  init?(ctx: KContext): void; // Opsional: Untuk terima 'suntikan' dari Kernel
  open?(): boolean; // Opsional: Lazy open device hardware
  close?(): boolean; // Opsional: Close device hardware (with refcounting)

  /**
   * Opsional: Apakah hardware benar-benar tersedia SEKARANG?
   * Dipakai `ls /dev` untuk menyembunyikan device yang tidak present
   * (udev-like hotplug). Jika tidak diimplementasikan → selalu present.
   */
  present?(): boolean;

  uid?: number;
}

```

```

gid?: number;
mode?: number;
disabled?: boolean;
}

```

Driver nyata kecil — NullDevice (/dev/null)

```

// src/kernel/devices/NullDevice.ts
import { IDevice } from "../IDevice";

/**
 * NULL DEVICE (/dev/null)
 *
 * Lubang hitam kernel.
 * Apapun yang ditulis ke sini akan dibuang.
 * Apapun yang dibaca dari sini akan langsung EOF (null/empty).
 */
export class NullDevice implements IDevice {
    name = "NullDevice";

    read() {
        return "";
    }

    write(_data: any): boolean {
        return true;
    }

    ioctl(_cmd: number, _arg: any): any {
        return true;
    }
}

```

Catatan: IDevice.ts juga memuat NullDevice ringkas (nama "Null", ioctl → -1) sebagai contoh belajar. Driver /dev/null yang dipakai boot adalah NullDevice.ts di atas.

Driver dengan init + default export — RandomDevice (/dev/randomdevice)

```

// src/kernel/devices/aux-devices/RandomDevice.ts
import { IDevice, KContext } from "../IDevice";

/**
 * RANDOM DEVICE (/dev/random)
 *
 * Menghasilkan angka acak sebagai string.
 * Implementasi sederhana untuk demonstrasi plugin system.
 */
export class RandomDevice implements IDevice {
    name = "RandomDevice";
    private kctx: KContext | null = null;

    init(ctx: KContext) {
        this.kctx = ctx;
        this.kctx.syslog("Driver initialized and ready.");
    }

    read() {
        // Balikin angka acak 0-999 sebagai string
        const val = Math.floor(Math.random() * 1000);
        if (this.kctx) {
            this.kctx.syslog(`Random number generated: ${val}`);
        }
        return val.toString() + "\n";
    }

    write(_data: any): boolean {
        // Menulis ke random device biasanya diabaikan atau buat seeding
    }
}

```

```

        if (this.kctx) {
            this.kctx.syslog("Write attempt to RandomDevice ignored.");
        }
        return true;
    }

    ioctl(_cmd: number, _arg: any): any {
        return true;
    }
}

// Plugin Export: Harus export default class yang implement IDevice
export default RandomDevice;

```

loadAuxDevices() — plugin loader (src/kernel/Kernel.ts)

```

private loadAuxDevices() {
    if (!this.devices) return;

    const auxPath = path.resolve(__dirname, "devices/aux-devices");
    if (!fs.existsSync(auxPath)) {
        this.logger.debug(`Auxiliary devices directory not found: ${auxPath}`);
        return;
    }

    const files = fs.readdirSync(auxPath);
    files.forEach((file) => {
        if (file.endsWith(".ts") || file.endsWith(".js")) {
            try {
                const fullPath = path.join(auxPath, file);
                const module = require(fullPath);
                const DeviceClass = module.default || module;

                if (typeof DeviceClass === "function") {
                    // 1. Try auto-loading as device instance (original behavior)
                    const instance = new DeviceClass() as IDevice;

                    // Check if device is explicitly disabled
                    if (instance.disabled === true) {
                        this.logger.debug(
                            `[Dynamic HAL] Kernel Plugin ${file} is disabled, skipping.`
                        );
                        return;
                    }

                    const devName = (
                        instance.name || file.replace(".ts", "").replace(".js", "")
                    ).toLowerCase();
                    this.devices![devName] = instance;
                    this.logger.info(
                        `[Dynamic HAL] Kernel Plugin Loaded: /dev/${devName}`
                    );

                    // 2. Check for static autoRegister method (new convention)
                    if (typeof (DeviceClass as any).autoRegister === "function") {
                        try {
                            (DeviceClass as any).autoRegister(this);
                            this.logger.debug(
                                `[Dynamic HAL] Auto-register called for ${file}`
                            );
                        } catch (e: any) {
                            this.logger.debug(
                                `[Dynamic HAL] Auto-register skipped for ${file}: ${e.message}`
                            );
                        }
                    }
                }
            } catch (e: any) {
                this.logger.error(`Failed to load aux device ${file}: ${e.message}`);
            }
        }
    });
}

```

```
});  
}
```

Perhatikan `const DeviceClass = module.default || module;` — ini alasan `export default` **wajib** agar class ter-instantiate.

Konvensi `autoRegister` — `MCP23017Device`

```
// src/kernel/devices/aux-devices/MCP23017Device.ts (bagian)  
  
// Platform-specific hardware configuration  
const HARDWARE_CONFIGS = [  
  { bus: 2, address: 0x20, name: "mcp23017" } // Orange Pi 3B default  
];  
  
export class MCP23017Device implements IDevice {  
  name: string;  
  uid: number = 0;  
  gid: number = 0;  
  mode: number = 0o660;  
  disabled: boolean = false;  
  
  static autoRegister(kernel: any): void {  
    for (const config of HARDWARE_CONFIGS) {  
      try {  
        const device = new MCP23017Device(config.bus, config.address, config.name);  
        kernel.devices[config.name] = device;  
      } catch (e: any) {  
        // Silently skip if hardware not present or i2c-bus not available  
      }  
    }  
  }  
  // ... (pinMode/digitalWrite/digitalRead)  
}  
  
export default MCP23017Device;
```

`applyDeviceConfigs()` — `udev-style` (`src/kernel/Kernel.ts`)

```
private applyDeviceConfigs() {  
  const cfg = Config.get();  
  if (!cfg.devices) return;  
  
  for (const devName in cfg.devices) {  
    const device = this.devices[devName];  
    if (device) {  
      const devCfg = cfg.devices[devName];  
      if (devCfg.mode !== undefined) device.mode = devCfg.mode;  
      if (devCfg.uid !== undefined) device.uid = devCfg.uid;  
      if (devCfg.gid !== undefined) device.gid = devCfg.gid;  
      this.logger.info(  
        `[udev] Configuration applied to / dev / ${devName}: mode = ${device.mode?.toString(8)}, uid = ${device.uid}, gid =  
        ${device.gid}`,  
      );  
    }  
  }  
}
```

Routing `OPEN /dev/*` — `src/kernel/Syscalls.ts` (bagian inti)

```
if (absoluteOpenPath.startsWith("/dev/")) {  
  const devName = absoluteOpenPath.replace("/dev/", "");  
  let device: IDevice | null = null;  
  if (this.kernel.devices && this.kernel.devices[devName]) {  
    device = this.kernel.devices[devName];  
  }  
}
```

```
// ... (alias: tty / screen,console,stdout,fb0 / keyboard,stdin - diringkas)
if (device) {
  const devPerm = {
    name: devName,
    uid: device.uid ?? 0,
    gid: device.gid ?? 0,
    mode: device.mode ?? 0o600, // Default: root only (rw-----)
  };
  if (!this.satpam.check(pcb, devPerm, requiredPerm)) {
    throw new Error(
      `Permission Denied: You cannot access device /dev/${devName}`,
    );
  }
  const f = flags || "r";
  if (f.includes("r") && device.ioctl) await device.ioctl(10, null); // INC_READ_REF
  if ((f.includes("w") || f.includes("a")) && device.ioctl)
    await device.ioctl(20, null); // INC_WRITE_REF
  if (device.open) {
    const opened = device.open();
    if (!opened) {
      throw new Error(`Failed to open device /dev/${devName}`);
    }
  }
  const fd = pcb.fdTable.length;
  pcb.fdTable.push({ device, context: absoluteOpenPath, flags });
  return fd;
}
}
```

Panduan: Menulis Driver Sendiri

[!TIP] Ini menjawab "Langkah berikutnya" di ToC Modul 10: *tutorial menulis driver sendiri*. Contoh terkecil sudah ada: `NullDevice` dan `RandomDevice`.

Lokasi file

Jenis driver	Lokasi	Cara daftar
Core device (stdin, fb0, tty, null)	<code>src/kernel/devices/*.ts</code>	Langsung di <code>Kernel.boot()</code> (map <code>this.devices</code>)
Hardware / eksperimental	<code>src/kernel/devices/aux-devices/<Nama>Device.ts</code>	Auto via <code>loadAuxDevices()</code> + <code>export default</code>
Interface jaringan	—	Via <code>cfg.network.interfaces</code> di <code>sysconfig.json</code>

Checklist 10 langkah

- Buat file:** `src/kernel/devices/aux-devices/EchoDevice.ts`.
- Import kontrak:** `import { IDevice, KContext } from "../IDevice";`
- Tulis class:** `export class EchoDevice implements IDevice { name = "echo"; ... }` — `name` menentukan path `/dev/echo` (huruf kecil).
- Implement wajib:** `read()`, `write()`, `ioctl()`. Kernel cukup memanggil ketiganya.
- Optional lifecycle:** `init(ctx)` untuk syslog, `open()/close()` untuk lazy open, `present()` untuk hotplug.
- Optional metadata:** `uid/gid/mode` (default permission `0o600`), `disabled` untuk mematikan saat boot.
- export default EchoDevice;** — WAJIB. Tanpa ini `loadAuxDevices()` tidak mengenali plugin (`module.default || module`).
- Optional static autoRegister(kernel)** — untuk konfigurasi hardware platform-specific (bus, address, name).
- Atur izin udev-style:** `sysconfig.json` → `"devices": { "echo": { "mode": 438 } }`.
- Test:** `boot` → `ls /dev/echo` → tulis/baca dari shell → cek syslog [Dynamic HAL] Kernel Plugin Loaded: `/dev/echo`. Tambah unit test di `src/kernel/devices/aux-devices/C10-AuxDevices.test.ts` (pola C10.08-C10.12).

[!IMPORTANT] Konvensi default export: loader memakai `const DeviceClass = module.default || module`; lalu `if (typeof DeviceClass === "function")`. Class driver **harus** di-export default agar ter-instantiate. `MCP23017Device` memenuhi keduanya: `export default + static autoRegister`.

Latihan / Praktik

1. Baca `wiki/DEVELOPER_GUIDE_DEVICES.md` — panduan menulis driver.
2. Baca `src/kernel/devices/aux-devices/RandomDevice.ts` — pola device minimal (`export default + init`).
3. Baca `src/kernel/devices/aux-devices/MCP23017Device.ts` — pelajari `autoRegister + disabled`.
4. Baca `src/kernel/devices/aux-devices/joystick.ts` — pelajari `present()` (hotplug udev-like).
5. Jalankan `ls /dev` setelah boot — perhatikan `/dev/randomdevice` dan metadata-nya.
6. Ubah mode di `sysconfig.json` (blok `devices.randomdevice`) → boot ulang → amati hasil `ls /dev`.
7. (Tantangan) Tulis driver `/dev/echo`: `read()` mengembalikan teks yang ditulis terakhir, `write()` menyimpannya. Ikuti checklist 10 langkah di atas, lalu uji dari shell.

Referensi

- `wiki/DEVELOPER_GUIDE_DEVICES.md` — panduan menulis driver
- `wiki/mcp23017-registration.md` — contoh registrasi plugin
- `wiki/course/00-overview.md §5.2` — device model (HAL) + catatan DbLib
- `src/kernel/devices/IDevice.ts` — kontrak `IDevice + KContext`
- `src/kernel/devices/NullDevice.ts`, `src/kernel/devices/aux-devices/RandomDevice.ts`, `src/kernel/devices/aux-devices/joystick.ts` — contoh driver
- `src/kernel/devices/aux-devices/MCP23017Device.ts`, `src/kernel/devices/aux-devices/MySQLDevice.ts` — `autoRegister & transport DB`
- `src/kernel/Kernel.ts` — `boot`, `loadAuxDevices()`, `applyDeviceConfigs()`
- `src/kernel/Syscalls.ts` — routing `OPEN/LS/STAT/CHMOD/CHOWN` untuk `/dev`, `DB_*` 67-71
- `src/common/SyscallCode.ts` — kode syscall `DB_*`
- `src/mirror/lib/DbLib.ts` — API `db.connect/query/disconnect`
- `src/mirror/etc/mysqld/mysqld.ts` — service daemon (transport alternatif)
- `src/kernel/devices/aux-devices/C10-AuxDevices.test.ts` — test driver (C10.08-C10.12)
- `src/sysconfig.json` — blok `devices` (udev-style)

Modul 10 — selesai. Bagian III tuntas. Lanjut ke [Modul 11 — Worker Thread & Sandbox](#).

RFC-TSIX-EDU-002 | Modul kesebelas kurikulum TSIX. Memahami bagaimana satu worker thread menjadi satu proses TSIX, dan bagaimana sandbox mengunci pintu host API.

`WorkerEntry.ts` adalah **bootloader** yang berjalan pertama kali di dalam setiap worker. Ia menginisialisasi `UserLib`, memuat aplikasi, lalu **mengunci pintu** (`restrictHostAPI`) sebelum aplikasi berjalan. Sandbox ini adalah salah satu dari dua lapisan batas privilege nyata TSIX.

Tujuan Pembelajaran

- ☐ Menjelaskan dua lapisan batas privilege nyata TSIX (thread + `PermissionManager`, dan sandbox `WorkerEntry`)
- ☐ Menjelaskan urutan kerja `WorkerEntry` (bootloader)
- ☐ Menjelaskan hijack `Module._load` dan resolusi `@tsix/*` / `@common/*` dari `vfsCache`
- ☐ Menjelaskan Direct Memory Execution (`_compile` dengan dummy filename)
- ☐ Menjelaskan apa yang disabotase di process (`exit` / `kill`)
- ☐ Menjelaskan aturan `isPrivileged` dan allow-list modul host
- ☐ Menjelaskan mengapa framework `@tsix/*` selalu boleh diakses
- ☐ Menjelaskan penanganan `unhandledRejection/uncaughtException`
- ☐ Menjelaskan kelemahan heuristik substring nama dan pertahanan berlapis

Konsep Inti

TSIX punya **dua lapisan batas privilege nyata**. Keduanya bekerja sama, tapi punya sifat berbeda.

 Empat lapis keamanan TSIX Sumber: <wiki/diagram/Keamanan-dan-Sandboxing-1.mmd>

1. Batas thread + IPC + `PermissionManager` (kernel)

Kernel berjalan di **main thread**. Setiap aplikasi berjalan di **worker thread sendiri**. App tidak pernah menyentuh memori kernel; satu-satunya jembatan adalah `postMessage` (request syscall → respons). Di sisi kernel, `PermissionManager` memeriksa `rxw` (root bypass → owner → group → others), `validateArgs` memvalidasi kontrak argumen syscall, dan bit `SETUID` memungkinkan kenaikan privilege root-only (mis. `/bin/login`).

Lapisan ini **tidak bisa diakali** oleh kode app — app tidak punya referensi ke objek kernel.

 Isolasi: main thread (kernel) vs worker thread (app) — komunikasi via IPC saja Sumber: <wiki/diagram/Keamanan-dan-Sandboxing-2.mmd>

2. Sandbox `WorkerEntry` (worker-local)

`WorkerEntry.ts` adalah **bootloader** pertama yang berjalan di dalam worker. Ia menyabotase API host yang berbahaya dan membatasi `require`, agar app **dipaksa** memakai syscall lewat `UserLib`. Mekanisme inti:

Mekanisme	Isi
Hijack <code>Module._load</code>	<code>@tsix/*</code> & <code>@common/*</code> di-resolve dari <code>vfsCache</code> (memori), bukan filesystem
Direct Memory Execution (DME)	<code>_compile</code> konten dari memori dengan dummy filename ; tanpa hit disk
<code>restrictHostAPI(appName)</code>	Kunci pintu: ganti <code>global.require</code> , sabotase <code>process.exit</code> / <code>process.kill</code>
Cek privileged	Substring nama app (rapuh): <code>server</code> , <code>daemon</code> , <code>dome</code> , <code>tbuild</code> , <code>vfs</code> , <code>mysqld</code>
Allow-list modul host	Hanya app privileged yang boleh <code>require</code> modul host tertentu
Fatal handler	<code>unhandledRejection</code> / <code>uncaughtException</code> → kirim <code>GUI_WINDOW_ERROR</code> ke parent → <code>realExit(1)</code>

Perilaku require: non-privileged vs privileged

Kebutuhan require	App non-privileged	App privileged
Framework @tsix/*, @common/*	✔ selalu boleh	✔ selalu boleh
Path /lib/... atau /common/...	✔ selalu boleh	✔ selalu boleh
Modul host allow-list (http, ws, path, fs, url, esbuild, crypto, os, bcryptjs, mysql2, mysql2/promise)	❌ throw	✔ boleh
Modul host lain (net, child_process, ...)	❌ throw	❌ throw (pesan spesifik)
process.exit / process.kill	❌ throw	❌ throw

[!IMPORTANT] Framework @tsix/* dan @common/* **selalu** bisa diakses, bahkan di sandbox terkunci. Sebab: framework adalah satu-satunya jembatan sah ke syscall — tanpa itu, app tidak bisa melakukan apa pun yang berguna.

Kelemahan yang diketahui

[!WARNING] Cek privileged adalah heuristik **substring nama app** — rapuh. Siapa pun bisa menamai app-nya evil-daemon dan mendapat allow-list modul host. Ini **bukan** batas keamanan nyata; ia hanya pembatas kenyamanan. Pertahanan sebenarnya tetap di kernel: PermissionManager + validateArgs + SETUID. Rencana jangka panjang: ganti heuristik ini dengan *capability-based* (app menyatakan permission yang dibutuhkan).

Alur / Cara Kerja

Walkthrough singkat

```
app dipanggil (tsh / rc.local / spawnProcess)
  | syscall EXEC
  ▼
Kernel.spawnWorker()
  | new Worker(WorkerEntry.ts, { workerData, execArgv })
  ▼
WorkerEntry.ts (bootloader di dalam worker)
1. simpan realExit = process.exit (sebelum disabotase)
2. hijack Module._load → @tsix/* & @common/* dari vfsCache
3. pasang unhandledRejection / uncaughtException
4. UserLib ← hijackRequire("@tsix/UserLib") dari memory cache
   lib = new UserLibClass(pid) → global._tsixLib
5. DME aplikasi: transpile appContent → _compile (dummy filename)
6. cari export main / Main / default
7. restrictHostAPI(appName) ← KUNCI PINTU (sebelum app jalan)
8. new AppClass() → await execute(lib, args)
9. hasil string → lib.std.print(); lalu lib.shell.exit(0)
  |
  ▼
KERNEL — tiap akses resource = syscall → PermissionManager
```

Langkah detail (sesuai kode)

1. **Kernel.spawnWorker** (src/kernel/Scheduler.ts) membentuk workerData: WorkerInitData:

```
const workerData: WorkerInitData = {
  pid: pcb.pid,
  appName: options.appName || pcb.name,
  args: options.args || [],
```

```

appPath: options.appPath,
stackBkfsPath: options.stackBkfsPath,
appContent: options.appContent,
env: pcb.env,
vfsCache: this.vfsCacheProvider ? this.vfsCacheProvider() : {}
};

```

`vfsCache` adalah hasil `rebuildVFSCache()`: framework `/lib` di-pre-compile ke memori saat boot.

2. **execArgv** dipilih berdasarkan ekstensi target:

- `*.js` → **JS-Direct (FAST)**: hanya `--enable-source-maps`.
- selain itu (`.ts`) → **TS-Transpile**: ditambah `-r esbuild-register` dan `-r tsconfig-paths/register`.

3. **Bootloader** (`WorkerEntry.ts`) langsung menyimpan `realExit = process.exit.bind(process)` — dipakai nanti untuk keluar sungguhan meski `process.exit` sudah disabotase.

4. **Hijack Module._load**: semua `require("@tsix/...")` / `require("@common/...")` (dan alias relatif) di-resolve dari `vfsCache`. Jika ada, konten di-`_compile` dari memori (DME) dengan **dummy filename**, lalu di-cache. Jika tidak ada, fallback ke `originalLoad` (host require).

5. **UserLib dimuat dari memory cache**: `hijackRequire("@tsix/UserLib")` → `new UserLibClass(pid)` → `global._tsixLib`. Ini yang menyediakan `lib.fs`, `lib.shell`, `lib.std`, `lib.net`, dll.

6. **DME aplikasi**: jika tidak ada `appPath` fisik, `appContent` (dari `workerData`) di-transpile (TS → JS via `esbuild.transformSync`) lalu `_compile` dengan **dua filename**:

- `moduleFilename` (fisik) → biar `require` menemukan `node_modules`.
- `stackFilename` (BKFS path, `.ts` → `.js`) → biar stack trace menunjuk path BKFS yang benar.

7. **Cari AppClass**: `exports.main` || `exports.Main` || `exports.default` || export fungsi pertama.

8. **restrictHostAPI(appName)** **dipanggil tepat setelah AppClass ditemukan dan sebelum new AppClass()** — "kunci pintu sebelum aplikasi berjalan". Dari sini, `require` dibatasi dan `process.exit/process.kill` → `throw`.

9. **Jalankan**: `new AppClass()` → `await app.execute(lib, args)`. Jika mengembalikan string non-kosong, dicetak via `lib.std.print`, lalu `lib.shell.exit(0)`.

10. **Jika error runtime**: kirim `GUI_WINDOW_ERROR` ke parent (WM/Asteracea), cetak ke `lib.std.error`, lalu `realExit(1)`.

Kode Sumber

File	Peran
<code>src/userland/WorkerEntry.ts</code>	Bootloader worker + sandbox (<code>restrictHostAPI</code>)
<code>src/kernel/Scheduler.ts</code>	<code>spawnWorker()</code> — bikin <code>workerData</code> , <code>vfsCache</code> , <code>execArgv</code>
<code>src/common/IPCTypes.ts</code>	Tipe <code>WorkerInitData</code> & <code>SyscallResponse</code>

Snippet (level kode)

[!NOTE] Semua potongan di bawah disalin dari `src/userland/WorkerEntry.ts` (verifikasi langsung ke kode).

1. Hijack Module._load (resolusi framework dari vfsCache)

```

const originalLoad = Module._load;
const vfsCache = (workerData as any).vfsCache || {};
const moduleCache: Record<string, any> = {};

```

```

Module._load = function (request: string, parent: any, isMain: boolean) {
  let normalizedRequest = request;

  // Resolve relative paths
  if (request.startsWith(".")) {
    if (request.includes("/common/")) {
      normalizedRequest = "@common/" + request.split("/common/")[1];
    } else if (request.includes("/lib/")) {
      normalizedRequest = "@tsix/" + request.split("/lib/")[1];
    } else if (parent && parent.filename) {
      const basename = path!.basename(parent.filename);
      if (basename.startsWith("@tsix_") && request.startsWith("./")) {
        normalizedRequest = "@tsix/" + request.substring(2);
      } else if (basename.startsWith("@common_") && request.startsWith("./")) {
        normalizedRequest = "@common/" + request.substring(2);
      }
    }
  }

  // Cached Module
  if (moduleCache[normalizedRequest]) return moduleCache[normalizedRequest];

  // Resolusi Memory Framework (VFS)
  let vfsPath = null;
  if (normalizedRequest.startsWith("@tsix/")) {
    vfsPath = "/lib/" + normalizedRequest.substring(6) + ".ts";
  } else if (normalizedRequest.startsWith("@common/")) {
    vfsPath = "/lib/common/" + normalizedRequest.substring(8) + ".ts";
  }

  if (vfsPath && vfsCache[vfsPath]) {
    const content = vfsCache[vfsPath];
    const dummyFilename = path!.join(process.cwd(), normalizedRequest.replace("/", "_") + ".js");

    const newMod = new Module(dummyFilename, parent);
    newMod.filename = dummyFilename;
    newMod.paths = Module._nodeModulePaths(process.cwd());

    // Framework modules are now pre-compiled in Kernel.
    // Direct execution for maximum performance.
    (newMod as any)._compile(content, dummyFilename);

    moduleCache[normalizedRequest] = newMod.exports;
    return newMod.exports;
  }

  return originalLoad.apply(this, arguments);
};

```

Pemetaan alias → path VFS: @tsix/x → /lib/x.ts; @common/y → /lib/common/y.ts. Import relatif dari dalam framework juga dinormalisasi (./z di @tsix_... → @tsix/z).

2. restrictHostAPI — kunci pintu (full)

```

const restrictHostAPI = (appName: string) => {
  const forbidden = (msg: string = "Security Violation: Direct Host API access is forbidden in TSIX Sandbox.") => {
    throw new Error(msg);
  };

  const isPrivileged = appName.toLowerCase().includes("server") ||
    appName.toLowerCase().includes("daemon") ||
    appName.toLowerCase().includes("dome") ||
    appName.toLowerCase().includes("tbuild") ||
    appName.toLowerCase().includes("vfs") ||
    appName.toLowerCase().includes("mysqld");
  const allowedModules = ["http", "ws", "path", "fs", "url", "esbuild", "crypto", "os", "bcryptjs", "mysql2", "mysql2/promise"];

  const privilegedRequire = (mod: string) => {
    // Framework aliases are ALWAYS allowed, even in sandbox
    if (mod.startsWith("@tsix/") || mod.startsWith("@common/") || mod.includes("/lib/") || mod.includes("/common/")) {

```

```

        return hijackRequire(mod);
    }

    if (allowedModules.includes(mod)) {
        return hostRequire!(mod);
    }
    forbidden(`Security Violation: Module '${mod}' is not in the privileged allow-list.`);
};

// Sembunyikan require jika ada (tergantung module loader)
if (typeof require !== "undefined") {
    (global as any).require = isPrivileged ? privilegedRequire : (mod: string) => {
        // Even in sandbox, framework cores MUST be accessible
        if (mod.startsWith("@tsix/") || mod.startsWith("@common/") || mod.includes("/lib/") || mod.includes("/common/")) {
            return hijackRequire(mod);
        }
        forbidden();
    };
}

// Batasi akses process yang sensitif
const p = (global as any).process;
if (p) {
    p.exit = forbidden;
    p.kill = forbidden;
}
};

```

Perhatikan: `allowedModules` berisi **11 entri** — termasuk `mysql2` **dan** `mysql2/promise`. Cek `privileged` memakai `substring`: `server`, `daemon`, `dome`, `tbuild`, `vfs`, `mysql` (dicek dengan `toLowerCase()`).

3. Direct Memory Execution — `_compile` aplikasi (DME path)

```

// Module filename HARUS physical path untuk require() nemu node_modules
const moduleFilename = path!.join(process.cwd(), appName + ".js");
// stackBkfsPath = BKFS path untuk stack trace (/opt/test/gui-test.js)
const stackBkfsPath = (data as any).stackBkfsPath;
const stackFilename = stackBkfsPath
    ? stackBkfsPath.replace(/\.ts$/, '.js')
    : moduleFilename;
// sourcefile untuk esbuild sourcemap — cukup nama file aja (tanpa path)
const sourceFileName = (stackBkfsPath || moduleFilename).split(/[\\\/]/).pop()!.replace(/\.js$/, '.ts');

// Jika content adalah TypeScript, transpile dulu ke JavaScript
if (isTypeScript) {
    const esbuild = hostRequire!("esbuild");
    const result = esbuild.transformSync(content, {
        loader: "ts",
        format: "cjs",
        target: "node18",
        sourcemap: "inline",
        sourcefile: sourceFileName,
    });
    content = result.code;
}

const appModule = new Module(moduleFilename, module.parent);
appModule.filename = stackFilename; // __filename = BKFS path
appModule.paths = Module._nodeModulePaths(path!.dirname(moduleFilename));

// _compile dengan stackFilename agar stack traceunjuk BKFS path
(appModule as any)._compile(content, stackFilename);

```

4. Penanganan fatal error

```

process.on("unhandledRejection", (reason) => {
    const msg = reason instanceof Error ? reason.message : String(reason);
    console.error("[Worker Fatal] Unhandled Rejection:", msg);
    trySendErrorToParent(msg);
});

```

```
    realExit(1);
  });

  process.on("uncaughtException", (err) => {
    const msg = err instanceof Error ? err.message : String(err);
    console.error("[Worker Fatal] Uncaught Exception:", msg);
    trySendErrorToParent(msg);
    realExit(1);
  });
```

`trySendErrorToParent` memakai `global._tsixLib: lib.getParentPid()` lalu `lib.shell.send(parentPid, { type: "GUI_WINDOW_ERROR", ... })` — jadi error tampil di window Asteracea.

Latihan / Praktik

1. Baca `src/userland/WorkerEntry.ts` — temukan baris `restrictHostAPI(appName)` dipanggil. Catat: dipanggil setelah `AppClass` ditemukan, sebelum `new AppClass()`.
2. Tulis app yang memanggil `process.exit(0)` — amati error "Security Violation". Bandingkan dengan `lib.shell.exit(0)` (cara yang benar).
3. Tulis app yang `require("fs")` — bandingkan error antara app biasa vs app bernama `my-daemon` (privileged).
4. Tulis app bernama `evil-daemon` yang `require("mysql2")` — buktikan heuristik substring bisa ditembus. Lalu jelaskan mengapa ini tidak berbahaya secara nyata (kernel tetap mengawal syscall).
5. Jelaskan mengapa framework `@tsix/*` harus selalu bisa diakses bahkan di sandbox.

Referensi

- `wiki/Keamanan-dan-Sandboxing.md` — model sandbox lengkap
- `wiki/course/00-overview.md §6` — Worker Thread & Sandboxing (ringkasan)
- `src/userland/WorkerEntry.ts` — bootloader + sandbox (kode sumber utama)
- `src/kernel/Scheduler.ts` — `spawnWorker()` (`workerData` + `vfsCache`)
- `src/common/IPCTypes.ts` — `WorkerInitData`

Modul 11 — selesai. Lanjut ke [Modul 12 — Module Resolution & DME](#).

RFC-TSIX-EDU-002 | Modul kedua belas kurikulum TSIX. Mengungkap salah satu bagian paling "ajaib" TSIX: kernel pre-compile `/lib` ke memori, worker mengeksekusi dari memori tanpa hit disk, dan trik alias filename agar import relatif tetap bekerja.

DME (Direct Memory Execution) adalah cara TSIX memuat framework `@tsix/*` dan `@common/*` **tanpa menyentuh filesystem** saat runtime. Kernel pre-compile `/lib` sekali saat boot, cache disalin ke tiap worker via `workerData`, lalu `WorkerEntry` meng-compile modul dari string di memori dengan **filename palsu** (dummy filename) agar import relatif tetap jalan.

Tujuan Pembelajaran

- ☐ Menjelaskan konsep DME (Direct Memory Execution)
- ☐ Menjelaskan hijack `Module._load` dan rewrite alias `@tsix/*` / `@common/*`
- ☐ Menjelaskan trik dummy-filename untuk import relatif `./x`
- ☐ Menjelaskan dual identity filename (fisik vs BKFS)
- ☐ Menjelaskan `moduleCache` dan kenapa framework terasa instan
- ☐ Menjelaskan perbedaan jalur TS-transpile vs JS-Direct

Konsep Inti

Apa itu Direct Memory Execution

DME menjawab dua masalah:

- Kinerja** — `require()` biasa melakukan resolusi dan pembacaan file dari disk setiap kali modul dimuat. Untuk framework yang dipakai semua app, ini mahal. DME mengganti pembacaan dari disk dengan **penemuan di object JavaScript** (`vfsCache`).
- Konsistensi alias** — semua app harus melihat framework yang **sama persis** (`@tsix/*` / `@common/*`) dengan perilaku yang sama, apa pun backend VFS-nya.

KERNEL (boot sekali)

```
rebuildVFSCache()
  fetchDir("/lib") ← rekursif dari BKFS
    baca /lib/UserLib.ts, /lib/emerald.ts, /lib/common/*.ts, ...
    .ts → esbuild.transformSync() → JS (CJS, target node18)
    simpan cache["/lib/...ts"] = kode JS (objek di memori)

vfsCache = { "/lib/UserLib.ts": "...js...", "/lib/Application.ts": "...js..." }

scheduler.setVFSProvider(() => vfsCache) ← snapshot per spawn
```

workerData.vfsCache (postMessage)

WORKER (WorkerEntry.ts – bootloader)

```
Module._load = hijack(request, parent, isMain)
require("@tsix/UserLib")
  → vfsPath = "/lib/UserLib.ts"
  → vfsCache["/lib/UserLib.ts"] ketemu? —YA→
  → dummyFilename = "<cwd>/@tsix_UserLib.js"
  → new Module(dummyFilename, parent)
  → _compile(kodeJS, dummyFilename) ← compile dari MEMORI
  → moduleCache["@tsix/UserLib"] = exports
  → return exports (TANPA hit filesystem)
```

```
require("@tsix/UserLib") berikutnya → moduleCache (cached)
```

Dua lapisan cache: vfsCache vs moduleCache

Cache	Di mana	Kunci	Isi	Siklus hidup
vfsCache	Kernel (Kernel.ts)	path BKFS (/lib/*.ts)	kode JS hasil pre-compile	satu kali per boot
moduleCache	per worker (WorkerEntry.ts)	alias (@tsix/UserLib)	objek exports modul	sekali per worker

vfsCache hanya berisi **string kode**. moduleCache berisi **objek exports siap pakai** — dibuat worker saat pertama kali memuat modul. Satu file framework di-pre-compile sekali (kernel), tapi di-`_compile` dan di-cache sekali **per worker**.

Alias dan pemetaan path

Alias	Direwrite menjadi	vfsPath (kunci vfsCache)
@tsix/UserLib	—	/lib/UserLib.ts
@tsix/emerald	—	/lib/emerald.ts
@common/GUITypes	—	/lib/common/GUITypes.ts
./UserLib (parent @tsix_*)	@tsix/UserLib	/lib/UserLib.ts
./GUITypes (parent @common_*)	@common/GUITypes	/lib/common/GUITypes.ts

[!IMPORTANT] Pemetaan @tsix/X → /lib/X.ts selalu menambahkan ekstensi .ts. Karena itu file framework di BKFS **harus** berupa .ts — konvensi yang dijamin vfs-bootstrap. Kernel ikut meng-cache .js/.json di /lib, tapi jalur DME @tsix/* hanya menarget .ts.

Kenapa ini penting

1. **Kinerja: nol FS-hit per require.** Framework dimuat sekali saat boot, lalu dieksekusi dari memori di tiap worker. Tanpa DME, tiap app yang import "@tsix/UserLib" harus membaca dan meng-compile ulang file dari VFS — mahal untuk GUI/daemon yang startup-nya harus cepat.
2. **Konsistensi alias.** Rewrite relatif ./x → @tsix/x / @common/x memastikan framework internal (mis. Application memakai UserLib) selalu memuat **instance yang sama** dari cache — tidak ada duplikasi path fisik vs VFS.
3. **Satu sumber kebenaran.** Kernel adalah satu-satunya yang membaca /lib dari BKFS. Worker tidak punya akses langsung ke disk framework — memperkuat batas sandbox (lihat Modul 11).

[!TIP] DME adalah salah satu dari **tiga prinsip inti** TSIX (syscall = satu-satunya pintu, everything-is-a-file, direct memory execution). Lihat wiki/course/00-overview.md §1 dan §6.2.

Dua jalur eksekusi app

Jalur	Kondisi	Metode
DME (VFS)	appPath kosong, appContent ada	_compile dari string; file .ts di-transpile esbuild di worker
Fisik (host)	appPath terisi	hostRequire(finalAppPath) — jalur Node.js normal, -r esbuild-register

Deteksi tipe: `isTypeScript = !(appPath || appName || "").toLowerCase().endsWith(".js")`.

Alur / Cara Kerja

1. **Boot kernel** (Kernel.initializeSubsystems) memanggil `rebuildVFSCache()`.
2. `rebuildVFSCache()` berjalan rekursif (fetchDir) dari /lib di BKFS; file .ts di-transpile esbuild → vfsCache.

3. Kernel menghubungkan scheduler: `scheduler.setVFSCacheProvider(() => vfsCache)`.
4. Saat `spawnWorker()`, scheduler menyalin cache: `vfsCache: this.vfsCacheProvider()` masuk ke `workerData`.
5. `new Worker(workerPath, { workerData, execArgv })` — thread baru dengan snapshot cache.
6. `WorkerEntry.ts` (modul scope) menyimpan `originalLoad = Module._load`, lalu mengganti `Module._load` dengan versi hijack.
7. `main()` memuat `UserLib`: `hijackRequire("@tsix/UserLib")` → lewat `Module._load` → `vfsCache` → `_compile` dari memori.
8. App dimuat via DME (jika konten dari VFS) atau fisik (jika `appPath`).
9. Sandbox diaktifkan (`restrictHostAPI`) — lihat [Modul 11](#) — sebelum app berjalan.

Kode Sumber

File	Peran
<code>src/kernel/Kernel.ts</code>	<code>rebuildVFSCache()</code> — pre-compile <code>/lib</code> → <code>vfsCache</code>
<code>src/kernel/Scheduler.ts</code>	<code>setVFSCacheProvider()</code> + <code>spawnWorker()</code> — kirim cache via <code>workerData</code>
<code>src/userland/WorkerEntry.ts</code>	Hijack <code>Module._load</code> , DME, dummy filename, dual identity

Snippet (level kode)

1. Kernel: pre-compile `/lib` ke memori

`rebuildVFSCache()` di `src/kernel/Kernel.ts` — disalin persis dari sumber:

```
private rebuildVFSCache() {
  this.bootLogStart(
    "VFS: Pre-compiling framework libraries (Memory Cache)... ",
  );
  try {
    const esbuild = require("esbuild");
    const cache: Record<string, string> = {};
    const fetchDir = (dir: string) => {
      if (!this.bkfs!.exists(dir)) return;
      const items = this.bkfs!.ls(dir);
      for (const item of items) {
        const p = `${dir}/${item.name}`;
        if (item.type === "DIRECTORY") {
          fetchDir(p);
        } else if (
          item.type === "FILE" &&
          (item.name.endsWith(".ts") ||
            item.name.endsWith(".js") ||
            item.name.endsWith(".json"))
        ) {
          let content = this.bkfs!.read(p);
          if (!content) continue;

          let code = content;

          // Transpile TS to JS for framework files
          if (item.name.endsWith(".ts")) {
            try {
              const result = esbuild.transformSync(code, {
                loader: "ts",
                format: "cjs",
                target: "node18",
                sourcemap: "inline",
              });
              code = result.code;
            } catch (err: any) {
              this.logger.error(`Failed to pre-compile ${p}: ${err.message}`);
            }
          }
        }
      }
    };
  }
}
```

```

    }

    cache[p] = code;
  }
}
};
fetchDir("/lib");
this.vfsCache = cache;
this.bootLogEnd(true, "OK");
} catch (e: any) {
  this.bootLogEnd(false, `Error: ${e.message}`);
}
}
}

```

Yang perlu diperhatikan:

- `fetchDir` **rekursif** — memasukkan subfolder seperti `/lib/common/`.
- `.ts` di-transpile via `esbuild.transformSync` (loader `ts`, format `cjs`, target `node18`). `.js/.json` disalin apa adanya.
- Kunci cache = **path BKFS** (`/lib/UserLib.ts`), nilai = **string kode JS**.

2. Kernel → Scheduler: saluran cache

```

// Kernel.ts — saat boot
this.rebuildVFSCache();
this.scheduler.setVFSCacheProvider(() => {
  return this.vfsCache;
});

// Scheduler.ts — spawnWorker()
const workerData: WorkerInitData = {
  pid: pcb.pid,
  appName: options.appName || pcb.name,
  args: options.args || [],
  appPath: options.appPath,
  stackBkfsPath: options.stackBkfsPath,
  appContent: options.appContent,
  env: pcb.env,
  vfsCache: this.vfsCacheProvider ? this.vfsCacheProvider() : {}
};

```

`setVFSCacheProvider()` menyimpan sebuah *callback*, bukan snapshot. Setiap `spawnWorker()` memanggil callback itu untuk mengambil cache **saat ini** dan menyalinnya ke `workerData`.

3. Worker: hijack `Module._load` + dummy filename + `_compile`

Di `src/userland/WorkerEntry.ts` — disalin persis dari sumber:

```

const originalLoad = Module._load;
const vfsCache = (workerData as any).vfsCache || {};
const moduleCache: Record<string, any> = {};

Module._load = function (request: string, parent: any, isMain: boolean) {
  let normalizedRequest = request;

  // Resolve relative paths
  if (request.startsWith(".")) {
    if (request.includes("/common/")) {
      normalizedRequest = "@common/" + request.split("/common/")[1];
    } else if (request.includes("/lib/")) {
      normalizedRequest = "@tsix/" + request.split("/lib/")[1];
    } else if (parent && parent.filename) {
      const basename = path!.basename(parent.filename);
      if (basename.startsWith("@tsix_") && request.startsWith("./")) {
        normalizedRequest = "@tsix/" + request.substring(2);
      } else if (basename.startsWith("@common_") && request.startsWith("./")) {
        normalizedRequest = "@common/" + request.substring(2);
      }
    }
  }

```

```

    }
  }

  // Cached Module
  if (moduleCache[normalizedRequest]) return moduleCache[normalizedRequest];

  // Resolusi Memory Framework (VFS)
  let vfsPath = null;
  if (normalizedRequest.startsWith("@tsix/")) {
    vfsPath = "/lib/" + normalizedRequest.substring(6) + ".ts";
  } else if (normalizedRequest.startsWith("@common/")) {
    vfsPath = "/lib/common/" + normalizedRequest.substring(8) + ".ts";
  }

  if (vfsPath && vfsCache[vfsPath]) {
    const content = vfsCache[vfsPath];
    const dummyFilename = path!.join(process.cwd(), normalizedRequest.replace("/", "_") + ".js");

    const newMod = new Module(dummyFilename, parent);
    newMod.filename = dummyFilename;
    newMod.paths = Module._nodeModulePaths(process.cwd());

    // Framework modules are now pre-compiled in Kernel.
    // Direct execution for maximum performance.
    (newMod as any)._compile(content, dummyFilename);

    moduleCache[normalizedRequest] = newMod.exports;
    return newMod.exports;
  }

  return originalLoad.apply(this, arguments);
};

```

Langkah hijack, satu per satu:

1. **Normalisasi request** — relatif `./x / .../lib/x` di-rewrite ke alias.
2. **Cek moduleCache** — kalau sudah pernah dimuat, langsung return exports (tanpa compile ulang).
3. **Petakan alias → vfsPath** (`@tsix/X → /lib/X.ts`, `@common/X → /lib/common/X.ts`).
4. **Cek vfsCache** — jika ketemu, ambil kode JS dari memori.
5. **Dummy filename** — `<cwd>/@tsix_X.js` (lihat di bawah).
6. `_compile(content, dummyFilename)` — Node.js meng-compile modul dari string, **tanpa membaca file**.
7. **Simpan ke moduleCache** dan return exports.
8. Jika tidak ada di cache → **fallback** ke `originalLoad.apply(...)` (jalur normal Node.js).

4. Trik dummy filename — identitas modul framework

Kernel hanya menyimpan kode di `vfsCache` dengan kunci path BKFS (`/lib/UserLib.ts`). Nama `@tsix_*.js` **dibuat saat load di worker**, bukan saat pre-compile:

```
const dummyFilename = path!.join(process.cwd(), normalizedRequest.replace("/", "_") + ".js");
```

Alias	replace("/", "_")	dummyFilename (basename)
@tsix/UserLib	@tsix_UserLib	<cwd>/@tsix_UserLib.js
@tsix/Application	@tsix_Application	<cwd>/@tsix_Application.js
@common/GUITypes	@common_GUITypes	<cwd>/@common_GUITypes.js

Kenapa awalan `@tsix_` / `@common_` penting? Karena rewrite relatif di dalam framework terjadi **berdasarkan basename parent**:

```

const basename = path!.basename(parent.filename);
if (basename.startsWith("@tsix_") && request.startsWith("./")) {
  normalizedRequest = "@tsix/" + request.substring(2);
}

```

Artinya, jika `Application.ts` (dimuat dengan filename `<cwd>/@tsix_Application.js`) melakukan `import x from "/UserLib"`, maka `./UserLib` → `@tsix/UserLib` → `/lib/UserLib.ts`. Siklus alias tetap konsisten di seluruh framework. Inilah yang membuat `@tsix/*` terlihat seperti paket instan.

[!NOTE] Jadi "dual identity" modul framework adalah: **identitas konten** = path BKFS (`/lib/UserLib.ts`, kunci `vfsCache`), sedangkan **identitas filename** yang dilihat Node = dummy path (`<cwd>/@tsix_UserLib.js`).

5. Dual identity filename untuk aplikasi

Saat worker memuat **app** via DME (blok di `main()`), ada dua path yang dibuat:

```
let content = (data as any).appContent;
const isTypeScript = !(appPath || appName || "").toLowerCase().endsWith(".js");
// Module filename HARUS physical path untuk require() nemu node_modules
const moduleFilename = path!.join(process.cwd(), appName + ".js");
// Stack filename = BKFS path biar stack trace bener (/opt/test/gui-test.js)
// stackBkfsPath = BKFS path untuk stack trace (/opt/test/gui-test.js)
const stackBkfsPath = (data as any).stackBkfsPath;
const stackFilename = stackBkfsPath
  ? stackBkfsPath.replace(/\.ts$/, '.js')
  : moduleFilename;
// sourcefile untuk esbuild sourcemap – cukup nama file aja (tanpa path)
const sourceFileName = (stackBkfsPath || moduleFilename).split(/[\\\/]/).pop()!.replace(/\.js$/, '.ts');

// Jika content adalah TypeScript, transpile dulu ke JavaScript
if (isTypeScript) {
  try {
    const esbuild = hostRequire!("esbuild");
    const result = esbuild.transformSync(content, {
      loader: "ts",
      format: "cjs",
      target: "node18",
      sourcemap: "inline",
      sourcefile: sourceFileName,
    });
    content = result.code;
  } catch (transpileErr: any) {
    console.error(`[Worker ${pid}] TS Transpile Error: ${transpileErr.message}`);
    throw transpileErr;
  }
}

// Create a new module instance with physical path (for node_modules resolution)
const appModule = new Module(moduleFilename, module.parent);
appModule.filename = stackFilename; // __filename shows BKFS path
appModule.paths = Module._nodeModulePaths(path!.dirname(moduleFilename));

// _compile dengan stackFilename agar stack trace nunjuk BKFS path
(appModule as any)._compile(content, stackFilename);

AppClass = appModule.exports.main || appModule.exports.Main || appModule.exports.default || appModule.exports;
```

Jadi:

- **moduleFilename** (fisik: `<cwd>/<appName>.js`) dipakai untuk `new Module(...)` dan `Module._nodeModulePaths(...)` — sehingga `require("node_modules/...")` internal app bisa resolve.
- **stackFilename** (BKFS: `/opt/test/gui-test.js`) ditetapkan sebagai `appModule.filename` dan diteruskan ke `_compile` — sehingga `__filename` dan **stack trace** menunjuk lokasi yang benar di VFS.

Inilah jawaban latihan #4: satu modul punya **dua identitas** karena dua kebutuhan berbeda (resolve modul vs error reporting).

Latihan / Praktik

1. Baca `rebuildVFSCache()` di `src/kernel/Kernel.ts`. Telusuri bagaimana `/lib` dibaca rekursif, file `.ts` di-transpile, dan hasilnya disimpan di `vfsCache`. Apa kunci dan nilai cache-nya?
2. Tambahkan log di `Module._load` (`WorkerEntry.ts`) saat `vfsPath` ketemu. Jalankan app yang `import ... from "@tsix/UserLib"`. Amati bahwa tidak ada hit filesystem — dan log kedua kalinya datang dari `moduleCache` (bukan `_compile` ulang).
3. Buat app yang melempar error. Periksa stack trace-nya — apakah menunjuk path BKFS (`/opt/...`) atau path fisik (`<cwd>/...`)? Cocokkan dengan `stackBkfsPath`.
4. Tambahkan log `basename(parent.filename)` di cabang `rewrite` relatif. Lihat nilai `@tsix_*` saat framework internal memakai `./x`. Jelaskan hubungannya dengan dummy filename.
5. Jelaskan mengapa dua identitas filename diperlukan (`require` vs stack trace), dan mengapa awalan `@tsix_ / @common_` menentukan `rewrite` relatif.
6. Bandingkan jalur TS-transpile vs JS-Direct: kapan `-r esbuild-register` dipakai di `execArgv`, dan kapan `esbuild` dipanggil manual di worker? (Lihat [Modul 11](#).)

Referensi

- [wiki/course/00-overview.md §1, §6.2](#) — DME sebagai prinsip inti & ringkasan bootstrap
- [wiki/course/11-worker-thread-sandbox.md](#) — `WorkerEntry` sebagai bootloader + `restrictHostAPI` ([Modul 11](#))
- [wiki/course/05-syscall-ipc.md](#) — `syscall` & `IPC`: mengapa worker berkomunikasi lewat `syscall`, bukan `require` langsung ([Modul 05](#))
- `src/kernel/Kernel.ts` — `rebuildVFSCache()`, `setVFSProvider()`
- `src/kernel/Scheduler.ts` — `spawnWorker()`, `workerData.vfsCache`
- `src/userland/WorkerEntry.ts` — hijack `Module._load`, DME, dummy filename
- `src/common/IPCTypes.ts` — tipe `WorkerInitData` (kontrak `workerData`)

Modul 12 — selesai. Bagian IV tuntas. Lanjut ke [Modul 13 — TTY & Virtual Console](#).

RFC-TSIX-EDU-002 | Modul ketiga belas kurikulum TSIX. Memahami TTY sebagai emulasi PTY lengkap: buffer layar, ANSI subset, raw/cooked mode, dan master-side ioctl.

TTY TSIX bukan sekadar "console teks" — ia adalah **emulasi PTY lengkap**. Aplikasi remote (mis. `pixelterm`) bisa mengendalikan TTY dari sisi master melalui `ioctl 0x2001 / 0x2002` (inject input / read output).

Tujuan Pembelajaran

- ☐ Menjelaskan peran TTYManager dan alokasi konsol 1-32
- ☐ Menjelaskan komponen TTY (buffer, cursor, parser ANSI)
- ☐ Menjelaskan ioctl penting TTYDevice (`clear`, `raw`, `0x2001`, `0x2002`, `winsize`)
- ☐ Menjelaskan alokasi TTY ke proses via `ttyId` di EXEC
- ☐ Menjelaskan perilaku keyboard (`Ctrl+C`, `Alt+F1-6`)

Konsep Inti

Komponen

Komponen	File	Peran
TTYManager	<code>src/kernel/tty/TTYManager.ts</code>	Kelola 32 konsol virtual (<code>Map<number, TTY></code>), <code>activeId</code> , <code>switch TTY</code> , <code>callback onSwitch/onInterrupt</code>
TTY	<code>src/kernel/tty/TTY.ts</code>	Buffer <code>char[y][x]</code> , <code>cursor</code> , <code>parser ANSI subset</code> , <code>raw/cooked mode</code> , <code>render ulang</code>
TTYDevice	<code>src/kernel/devices/TTYDevice.ts</code>	<code>/dev/ttyN</code> — bungkus TTY jadi <code>IDevice</code> (<code>read/write/ioctl</code>)

TTY dipakai sebagai tiga device sekaligus untuk proses yang "menempel" di konsol: `FD 0` (`stdin`), `FD 1` (`stdout`), `FD 2` (`stderr`) semuanya menunjuk ke `TTYDevice` yang sama. `init` (`PID 1`) dibuat dengan `fds: [tty1, tty1, tty1]` dan `ttyId: 1`.

Buffer & Mode

- **Buffer layar**: `charBuffer: string[][]` berisi karakter, `attrBuffer: string[][]` berisi gaya ANSI (mis. `"31;1"`). Keduanya diindeks `[y][x]` (baris dulu, kolom kedua). Ukuran default `80×24`, tapi saat boot `TTYManager` memakai ukuran terminal host (`process.stdout.rows / columns`).
- **Cooked mode** (default): input di-line-buffer. `Echo` dilakukan oleh TTY, `Enter` memindahkan baris ke `inputLines[]`, `Backspace` menghapus karakter terakhir, dan `Ctrl+C` menghasilkan sinyal (tidak masuk buffer).
- **Raw mode** (`setRawMode(true)`): tiap karakter langsung masuk `inputBuffer[]` tanpa echo dan tanpa line editing. Cocok untuk line editor / shell.
- **Output buffer**: setiap `write()` juga merekam data ke `outputBuffer` — inilah yang dibaca master lewat `ioctl 0x2002` (`getOutput()` sekaligus mengosongkannya).

ioctl TTYDevice

cmd	Nama	Makna	Implementasi
1	<code>CLEAR_SCREEN</code>	bersihkan layar TTY	<code>tty.clear()</code> ; jika aktif, reset <code>stdout</code> host (<code>\x1bc</code>)
2	<code>SWITCH_TTY</code>	pindah konsol	<code>return -1</code> — ditangani di <code>Syscalls.ts</code> via <code>Kernel.ttyManager.switch()</code>
10	<code>SET_RAW_MODE</code>	set mode raw/cooked	<code>tty.setRawMode(!arg)</code>
<code>0x2001</code>	<code>INJECT_INPUT</code>	master mengetik ke <code>stdin</code> slave	<code>tty.pushInput(arg)</code>

cmd	Nama	Makna	Implementasi
0x2002	READ_OUTPUT	master membaca stdout slave	<code>tty.getOutput()</code> (dan kosongkan buffer)
4	TIOCGWINSZ	ukuran window	<code>{ lines: tty.height, columns: tty.width }</code>

[!NOTE] Kode `0x2001/0x2002` juga dipakai `MySQLDevice` (`MYSQL_IOCTL_CONNECT/DISCONNECT`). Makna `ioctl` bergantung pada driver — di `TTYDevice` keduanya adalah injeksi input / baca output PTY.

Subset ANSI yang Ditangani

Parser ANSI TSIX sederhana: urutan `ESC [ ...` di-parse sampai ditemukan huruf perintah (`cmd`), lalu argumen dipisah dengan `;`. Berikut subset yang diimplementasikan di `TTY.handleANSI()`:

Urutan	Nama	Perilaku
<code>ESC[nA</code>	Cursor Up	kursor naik <code>n</code> baris (clamp 0)
<code>ESC[nB</code>	Cursor Down	kursor turun <code>n</code> baris (clamp <code>height-1</code>)
<code>ESC[nC</code>	Cursor Forward	kursor maju <code>n</code> kolom (clamp <code>width-1</code>)
<code>ESC[nD</code>	Cursor Backward	kursor mundur <code>n</code> kolom (clamp 0)
<code>ESC[r;ch / ESC[r;cf</code>	Cursor Position	pindah kursor absolut (1-based)
<code>ESC[...m</code>	SGR (warna/atribut)	simpan ke <code>currentAttr</code> (default <code>"0"</code>)
<code>ESC[2J</code>	Erase Display	<code>clear()</code> seluruh layar
<code>ESC[0J</code>	Erase Display (partial)	hapus dari kursor ke bawah
<code>ESC[K</code>	Erase Line	hapus dari kursor ke akhir baris
<code>ESC[nS</code>	Scroll Up (SU)	geser baris ke atas <code>n</code> baris
<code>ESC[nT</code>	Scroll Down (SD)	geser baris ke bawah <code>n</code> baris
<code>ESC[nL</code>	Insert Line (IL)	sisip <code>n</code> baris kosong di baris kursor
<code>ESC[nM</code>	Delete Line (DL)	hapus <code>n</code> baris di baris kursor
<code>ESC[s</code>	Save Cursor (DECSC)	simpan posisi kursor
<code>ESC[u</code>	Restore Cursor (DECRC)	pulihkan posisi kursor

Selain itu `write()` menangani kontrol karakter dasar: `\n` (baris baru), `\r` (`cursorX = 0`), `\b` (mundur satu kolom). `render()` menggambar ulang seluruh layar memakai kursor absolut per baris (`ESC[r;ch + ESC[2K`) agar tidak rusak oleh wrapping terminal host.

Alokasi TTY ke proses

Berbeda dari `PortManager` (jaringan), alokasi TTY ke proses dilakukan **via `ttyId` di `syscall EXEC`** — kernel override `FD 0/1/2` ke `tty{n}`:

- Argumen `ttyId` valid hanya untuk rentang **1-12**.
- Jika valid, `stdin/stdout/stderr` di-set ke device `tty{ttyId}`.
- Jika `ttyId` tidak diberikan, proses **mewarisi** `ttyId` dari parent (`pcb.ttyId`).
- `createProcess()` menyimpan `ttyId` di PCB; `runInit()` membuat PID 1 dengan `ttyId: 1` lalu `setForegroundProcess(pid, 1)`.

Scheduler memelihara pemetaan `ttyForegroundPids: Map<ttyId, pid>` — satu foreground process per TTY. Saat proses **detach** (daemonize) atau **EXIT**, ia dilepas dari pemetaan foreground dan `ttyId`-nya dibersihkan. Contoh pemakaian di userland: `init` (PID 1) di `tty1`, lalu `login` di-spawn untuk TTY lain dengan `ttyId` tujuan.

Keyboard

- **Hotkey** Alt+F1-F6 → `TTYManager.switch` (juga Alt+1-6 untuk macOS).
- **Ctrl+C** → `SIGINT` ke foreground process TTY aktif.
- **Switch konsol** → `SIGWINCH` ke foreground process TTY yang baru aktif.
- **Resize jendela host** → update `LINES/COLUMNS` di env semua proses + `SIGWINCH` broadcast + `TTYManager.handleResize()`.

Alur / Cara Kerja

Input: keystroke → proses foreground

```

Host keyboard (process.stdin)
  | data mentah (utf8)
  ▼
KeyboardDevice (data handler kernel)
  | hotkey? (Alt+F1..6) → TTYManager.switch(...) → STOP
  ▼
Kernel: ttyManager.getActiveTTY().pushInput(data)
  |
  ▼
TTY.pushInput(data)
  ├── cooked : echo + lineBuffer → Enter → inputLines[]
  └── raw      : tiap char → inputBuffer[]
      |
      ▼
Proses foreground → syscall READ(fd 0) → TTYDevice.read()
  | raw → tty.read() → inputBuffer.shift()
  └── cooked → tty.read() → inputLines.shift()

```

Output: aplikasi → TTY → render

```

Aplikasi → syscall WRITE/PRINT(fd 1) → TTYDevice.write(data)
  |
  ▼
TTY.write(data)
  1. outputBuffer += data          (untuk master ioctl 0x2002)
  2. onWrite(data)                 (master remote ikut menerima)
  3. parse char: \n \r \b, ESC[...] (subset ANSI), karakter biasa
      | karakter biasa → putChar() → charBuffer[y][x] + attrBuffer[y][x]
      ▼
TTY aktif? → tty.onWrite → process.stdout.write(data) (layar host)
Master?    → syscall READ(pid) → ioctl 0x2002 → getOutput() (pixelterm)

```

Wiring saat boot (Kernel.boot)

```

new TTYManager(32)          → 32× TTY (ukuran host)
  └─ 32× TTYDevice "tty1..tty32" → /dev/ttyN (isActive = aktifId === N)
devices: stdin=KeyboardDevice, fb0/stdout/stderr → tty1, null, tty1..32
kbd.setDataHandler          → getActiveTTY().pushInput(data)
kbd.setHotkeyHandler        → handleKeyboardHotkey (Alt+F1..6)
kbd.setInterruptHandler     → handleHostInterrupt (Ctrl+C)
ttyManager.setOnSwitchCallback → SIGWINCH ke foreground TTY baru
ttyManager.setOnInterruptCallback → SIGINT ke foreground TTY
runInit() → createProcess("init", fds:[tty1x3], ttyId:1)
          → setForegroundProcess(pid, 1)

```

Switch konsol (Alt+F2)

```

Host kirim seq "\x1b00" (atau "\x1b[1;3Q")
  → KeyboardDevice.onHotkey → handleKeyboardHotkey(seq) → true (stop)
  → ttyManager.switch(2)
    1. activeId = 2
    2. reset host terminal: "\x1bc\x1b[3J"
    3. banner "TSIX VIRTUAL CONSOLE [ TTY 2 ]" 500ms (jika visualIdentity ada)

```


- 4. render() buffer TTY2 → process.stdout
- 5. onSwitchCallback(2) → SIGWINCH ke foreground PID TTY2

Ctrl+C → SIGINT

```
Host/kbd deteksi "\u0003"
→ pushInput: cooked → onInterrupt() (tidak masuk buffer); raw → onInterrupt() + tetap masuk buffer
→ TTYManager.onInterruptCallback(activeId)
→ Kernel → SIGINT ke foreground process TTY aktif
→ Scheduler.sendInterruptSignal(ttyId) → kill(fgPid, 2)
  → event "signal" SIGINT → grace 100ms → worker.terminate() jika tak ditangani
```

Kode Sumber

File	Peran
src/kernel/tty/TTY.ts	Emulasi TTY: buffer, cursor, subset ANSI, raw/cooked, render
src/kernel/tty/TTYManager.ts	Alokasi 32 konsol, activeId, switch, callback
src/kernel/devices/TTYDevice.ts	Device wrapper /dev/ttyN + ioctl
src/kernel/devices/KeyboardDevice.ts	Driver keyboard + deteksi hotkey/interrupt
src/kernel/Scheduler.ts	Foreground per TTY (ttyForegroundPids), SIGINT/SIGWINCH
src/kernel/Syscalls.ts	EXEC ttyId, ioctl 0x2001/0x2002, switch/resize
src/kernel/Kernel.ts	Boot wiring, hotkey handler, resize listener, runInit

Snippet (level kode)

Buffer input TTY — push/pop (src/kernel/tty/TTY.ts)

```
public pushInput(data: string) {
  if (this.rawMode) {
    // RAW MODE: langsung ke inputBuffer (tiap karakter)
    for (const char of data) {
      if (char === "\u0003") { // Ctrl+C
        if (this.onInterrupt) {
          this.onInterrupt();
        }
        // Tetap push untuk app yang mau menangani sendiri
        this.inputBuffer.push(char);
      } else {
        this.inputBuffer.push(char);
      }
    }
  } else {
    // COOKED MODE: echo + backspace + line buffering
    for (const char of data) {
      const code = char.charCodeAt(0);
      if (char === "\u0003") { // Ctrl+C → interrupt, jangan masuk buffer
        if (this.onInterrupt) this.onInterrupt();
        continue;
      }
      if (char === "\r" || char === "\n") {
        this.inputLines.push(this.lineBuffer + "\n");
        this.lineBuffer = "";
        this.write("\n"); // echo newline
        continue;
      }
      if (code === 127 || code === 8) { // Backspace
        if (this.lineBuffer.length > 0) {
          this.lineBuffer = this.lineBuffer.slice(0, -1);
        }
      }
    }
  }
}
```

```

        this.write("\b \b");    // echo hapus visual
    }
    continue;
}
this.lineBuffer += char;
// ... filter escape sequence untuk echo (NORMAL/ESC/CSI),
//     lalu echo hanya char printable (0x20-0x7E) + \n \r \t
}
}
}

public read(): string | null {
    if (this.rawMode) {
        if (this.inputBuffer.length === 0) return null;
        return this.inputBuffer.shift() || null;
    } else {
        if (this.inputLines.length === 0) return null;
        return this.inputLines.shift() || null;
    }
}

public setRawMode(enabled: boolean) {
    this.logger.debug(`setRawMode(${enabled}) - previously ${this.rawMode}`);
    this.rawMode = enabled;
    // Apps yang pindah ke raw mode biasanya tidak ingin input cooked tertunda
    if (enabled) {
        this.lineBuffer = "";
    }
}

```

ioctl TTYDevice (src/kernel/devices/ttyDevice.ts)

```

public ioctl(cmd: number, arg: any): any {
    if (cmd === 1) { // 1 = CLEAR_SCREEN
        this.tty.clear();
        if (this.isActive()) {
            process.stdout.write("\x1bc");
        }
        return 0;
    }
    if (cmd === 2) { // 2 = SWITCH_TTY
        return -1; // Handled in Syscalls.ts via Kernel.ttyManager
    }
    if (cmd === 10) { // 10 = SET_RAW_MODE
        this.tty.setRawMode(!arg);
        return 0;
    }
    if (cmd === 0x2001) { // INJECT_INPUT (Master typing into slave stdin)
        this.tty.pushInput(arg as string);
        return true;
    }
    if (cmd === 0x2002) { // READ_OUTPUT (Master reading slave stdout)
        return this.tty.getOutput();
    }
    if (cmd === 4) { // 4 = TIOCGWINSZ (Get Window Size)
        return { lines: this.tty.height, columns: this.tty.width };
    }
    return -1;
}

```

TTYManager.switch (src/kernel/tty/ttyManager.ts)

```

public async switch(id: number, forceRedraw: boolean = false) {
    if (!this.ttys.has(id)) return;
    if (id === this.activeId && !forceRedraw) return;

    this.logger.info(`Switching from TTY${this.activeId} to TTY${id}`);
    this.activeId = id;

    // Reset total terminal host agar tidak ada sisa dari TTY lama

```

```

process.stdout.write("\x1bc\x1b[3J");

// Banner visual terpusat (500ms) jika ada visual identity
if (this.visualIdentity && !forceRedraw) {
  const rows = process.stdout.rows || 24;
  const cols = process.stdout.columns || 80;
  const text = `TSIX VIRTUAL CONSOLE [ TTY ${id} ]`;
  const barLines = this.visualIdentity.split("\n");
  const bannerWidth = 32;
  const bannerHeight = 1 + barLines.length;
  const startRow = Math.max(1, Math.floor((rows - bannerHeight) / 2));
  const startCol = Math.max(1, Math.floor((cols - bannerWidth) / 2));
  process.stdout.write("\x1b[?25l");
  process.stdout.write(`\x1b[${startRow};${startCol}H\x1b[97m  ${text}\x1b[0m`);
  barLines.forEach((line, index) => {
    process.stdout.write(`\x1b[${startRow + 1 + index};${startCol}H${line}`);
  });
  await new Promise(resolve => setTimeout(resolve, 500));
  process.stdout.write("\x1bc\x1b[3J\x1b[?25h");
}

const content = this.getActiveTTY().render();
process.stdout.write(content);

// Notify foreground process di TTY yang baru aktif
if (this.onSwitchCallback) {
  this.onSwitchCallback(id);
}
}

```

Alokasi ttyId di EXEC (src/kernel/Syscalls.ts)

```

// --- TTY REDIRECTION SUPPORT ---
// Jika TTY tertentu diminta, override I/O default ke TTY itu
if (ttyId !== undefined && ttyId >= 1 && ttyId <= 12) {
  const targetTtyDevice = this.kernel.devices[`tty${ttyId}`];
  if (targetTtyDevice) {
    stdinDevice = targetTtyDevice;
    stdoutDevice = targetTtyDevice;
    stderrDevice = targetTtyDevice;
  }
} else {
  // Warisan normal jika tidak ada TTY khusus
  if (stdoutFd !== undefined && pcb.fdTable[stdoutFd]) {
    stdoutDevice = pcb.fdTable[stdoutFd]!.device;
  }
  if (stdinFd !== undefined && pcb.fdTable[stdinFd]) {
    stdinDevice = pcb.fdTable[stdinFd]!.device;
  }
}

const newPcb = this.scheduler.createProcess(binaryName, {
  fds: [stdinDevice, stdoutDevice, stderrDevice],
  // ... appName, args, env, uid/gid, dst
  ttyId: ttyId !== undefined ? ttyId : pcb.ttyId, // target TTY atau warisi dari parent
  ppid: pid,
});

```

Hotkey Alt+F1..6 (src/kernel/Kernel.ts)

```

private handleKeyboardHotkey(seq: string): boolean {
  const hotkeys: Record<string, number> = {
    // Alt+F1..F6 (Xterm, iTerm2, Terminal.app)
    "\x1b\x1b0P": 1,  "\x1b[1;3P": 1,  "\x1b[11;3~": 1,
    "\x1b\x1b0Q": 2,  "\x1b[1;3Q": 2,  "\x1b[12;3~": 2,
    "\x1b\x1b0R": 3,  "\x1b[1;3R": 3,  "\x1b[13;3~": 3,
    "\x1b\x1b0S": 4,  "\x1b[1;3S": 4,  "\x1b[14;3~": 4,
    "\x1b\x1b[15~": 5, "\x1b[15;3~": 5,  "\x1b[1;3;15~": 5,
    "\x1b\x1b[17~": 6, "\x1b[17;3~": 6,  "\x1b[1;3;17~": 6,
    // Alt+1..6 (macOS saat Option bertindak sebagai Meta)

```

```

        "\x1b1": 1, "\x1b2": 2, "\x1b3": 3,
        "\x1b4": 4, "\x1b5": 5, "\x1b6": 6,
    };
    if (hotkeys[seq]) {
        // FIRE AND FORGET: jangan await agar tidak memblokir input keyboard
        this.ttyManager?.switch(hotkeys[seq]);
        return true; // handled
    }
    return false;
}

```

Wiring callback & foreground (src/kernel/Kernel.ts + src/kernel/Scheduler.ts)

```

// Kernel: daftarkan callback TTY
this.ttyManager.setOnSwitchCallback((ttyId: number) => {
    const fgPid = this.scheduler?.getForegroundProcess(ttyId);
    if (fgPid) {
        this.logger.debug(`Sending SIGWINCH to PID ${fgPid} (TTY${ttyId} activated)`);
        this.scheduler?.sendEvent(fgPid, "signal", "SIGWINCH");
    }
});

this.ttyManager.setOnInterruptCallback((ttyId: number) => {
    const fgPid = this.scheduler?.getForegroundProcess(ttyId);
    if (fgPid) {
        this.logger.info(`Sending SIGINT to PID ${fgPid} (TTY${ttyId} Ctrl+C)`);
        this.scheduler?.sendEvent(fgPid, "signal", "SIGINT");
    }
});

// Scheduler: satu foreground process per TTY
public setForegroundProcess(pid: number | null, ttyId?: number) {
    if (pid !== null && !ttyId) {
        const pcb = this.getProcess(pid);
        ttyId = pcb?.ttyId || 1;
    }
    const targetTty = ttyId || 1;
    if (pid === null) {
        this.ttyForegroundPids.delete(targetTty);
    } else {
        this.ttyForegroundPids.set(targetTty, pid);
    }
}

```

Latihan / Praktik

1. Baca `src/kernel/tty/TTY.ts` — jelaskan struktur buffer `char[y][x] + attr[y][x]` dan subset ANSI di `handleANSI()`.
2. Jalankan `pixelterm` (jika tersedia) — amati bagaimana ia memakai `ioctl 0x2001` (inject input) dan `0x2002` (read output) dari sisi master.
3. Dari TTY lain, tekan `Alt+F2` — amati switch konsol dan banner visual.
4. Baca `src/kernel/devices/TTYDevice.ts` — jelaskan tiap `ioctl (1, 2, 10, 0x2001, 0x2002, 4)`.
5. Telusuri `syscall EXEC` di `src/kernel/Syscalls.ts` — apa bedanya saat `ttyId` diisi vs tidak? Kapan proses mewarisi `ttyId` parent?
6. Tekan `Ctrl+C` pada proses foreground — jelaskan jalur sinyal dari `pushInput` → `onInterrupt` → `SIGINT`.

Referensi

- `wiki/Kernel-dan-Scheduler.md §TTY`
- `wiki/course/00-overview.md §8`
- `src/kernel/tty/TTY.ts`, `src/kernel/tty/TTYManager.ts`, `src/kernel/devices/TTYDevice.ts`
- `src/kernel/Scheduler.ts` (`§ ttyForegroundPids`, `setForegroundProcess`, `sendInterruptSignal`)
- `src/kernel/Syscalls.ts` (`syscall EXEC`, `SEND/READ` via `ioctl 0x2001/0x2002`, `SWITCH_TTY`, `WINSIZE`)

Modul 13 — selesai. Lanjut ke [Modul 14 — Userland](#).

RFC-TSIX-EDU-002 | Modul keempat belas kurikulum TSIX. Memahami lapisan aplikasi: PID 1 (init), login, shell, dan dua gaya penulisan aplikasi TSIX.

Semua userland berjalan di Worker Thread (Ring 4) — termasuk PID 1. Tidak ada inittab: logika init **hardcoded** di `init.ts`.

Tujuan Pembelajaran

- ☐ Menjelaskan peran init (PID 1) tanpa inittab
- ☐ Menjelaskan alur login: `passwd+shadow(bcrypt)` → `setgroups`→`setgid`→`setuid`
- ☐ Menjelaskan dua gaya aplikasi: `IProgram` vs `Program()` wrapper
- ☐ Menjelaskan konsep "error = window" (`std.error`)
- ☐ Menjelaskan peran `monitorProcess` (respawn login)

Konsep Inti

Init (PID 1)

Tidak ada `/etc/inittab` — seluruh keputusan init di-hardcode di `src/mirror/bin/init.ts` (class `Init`). Urutan eksekusi:

1. **Enforce setuid** — `chmod("/bin/passwd.js", 2541)` dan `chmod("/bin/sudo.js", 2541)`. Nilai 2541 (desimal) sama dengan `0o4755` (setuid root + `rw-r-x-r-x`). Targetnya `.js`, bukan `.ts`, karena runtime mengeksekusi sidcar `.js`.
2. **RSA identity** — cek `/etc/keys/rsa/id_rsa.pub`. Jika tidak ada, `SecurityAgent.generateKeyPair()` membuat kunci baru (RSA 2048-bit) lalu disimpan ke `/etc/keys/rsa/`. Jika sudah ada, kunci diverifikasi. Fingerprint SHA256 diambil via `shell.getFingerprint()`, dan bar warna identitas (`SecurityAgent.generateVisualIdentity`) dikirim ke TTY via `std.ioctl(1, 33, colorBar)` — `ioctl 33 = SET_VISUAL_IDENTITY`.
3. **Exec rc.local** — jalankan `/etc/rc.local.js` lalu `waitpid`. Exit code `0` = startup berhasil; selain itu dicatat sebagai peringatan.
4. **Spawn login TTY2-6** — `for (let i = 2; i <= 6; i++) spawnLogin(i)`. Setiap proses login di-*monitor* oleh `monitorProcess` (respawn + delay 1s anti-crash-loop).
5. **Spawn login TTY1** (foreground) — dilakukan setelah banner dan fingerprint.
6. **Loop selamanya** — `while (true) { await new Promise(r => setTimeout(r, 10000)); }`. Init tidak boleh exit; jika PID 1 mati, kernel melakukan `process.exit(exitCode)` (`keepAlive, 1 = reboot`).

Login

`src/mirror/bin/login.ts` — satu proses login per TTY, di-spawn oleh init. Alur inti:

1. Baca `/etc/passwd` → cari entri user → ambil `uid` (field 3), `gid` (field 4), `home` (field 6), `shell` (field 7).
2. Baca `/etc/shadow` → ambil hash `bcrypt` (field 2) → `bcrypt.compareSync(password, hash)`.
3. Baca `/etc/group` → kumpulkan *supplementary groups* (grup yang memuat username, selain gid primer).
4. `setgroups` → `setgid` → `setuid` — **urutan POSIX wajib**. GID harus diganti sebelum UID; setelah UID menjadi non-root, proses kehilangan privilege untuk mengubah GID.
5. Set env `USER/HOME`, `chdir(home)`, lalu `exec shell` — shell diambil dari `/etc/passwd` field 7 (umumnya `tsh`).

Login WM (Asteracea) — mode GUI

TTY login (`login.ts`) di-spawn ulang sebagai proses root per TTY, jadi selalu bisa baca `/etc/shadow` & `setuid`. Berbeda dengan itu, **WM (Asteracea) adalah SATU proses yang merangkap login manager + desktop session**:

- WM start sebagai root (di-spawn init), lalu setelah login pertama **drop privilege permanen** ke user yang login.
- Akibatnya saat logout → login ulang (mis. sebagai root), WM sudah non-root → **dua masalah**:

1. Tidak bisa baca `/etc/shadow` (0640 root) → verifikasi password gagal.
 2. Kernel menolak `setgroups`/`setgid`/`setuid` untuk non-root → tidak bisa ganti user.
- Solusinya (lihat Snippet):
 1. **Verifikasi via helper SetUID root** — `login.js --verify <user> <pass> <file>` (login.js sudah SetUID root) → hasil ditulis ke file temp (`OK/FAIL:...`), dibaca WM. Kanal file dipakai karena *exit code* anak tidak andal (WorkerEntry selalu menuntaskan dengan `exit(0)`).
 2. **Saved UID (kernel)** — `pcb.suid`. Saat WM drop dari root, kernel menyimpan `suid=0`; saat re-login, WM boleh `setuid(0)` untuk **restore** ke root, lalu drop ke user target. Ini mekanisme Saved UID ala Unix (`seteuid/setresuid`).
 3. **Urutan set identitas di WM** — jika WM belum root → `setuid(0)` dulu (restore), baru `setgroups` → `setgid` → `setuid(user target)` (urutan POSIX untuk drop).

[!NOTE] **Keamanan Saved UID** — `suid` default di `createProcess` = UID proses itu sendiri, jadi **app biasa tidak mewarisi saved root** dan tidak bisa escalate. Hanya proses yang turun dari root (yaitu WM) yang punya "tiket kembali ke root".

Shell (tsh)

`src/mirror/bin/tsh.ts` — shell interaktif, ditulis dengan gaya `IProgram` (`export class main implements IProgram`).

Fitur utama:

- Prompt `user@hostname:cwd#/$` yang bisa dikustomisasi via env `PROMPT_FORMAT` (`&username`, `&hostname`, `&cwd`, `&usertype`).
- Line editor: raw mode, riwayat (panah atas/bawah), Tab completion, Ctrl+U, Home/End.
- Sebelum menjalankan perintah, shell beralih ke cooked mode agar perintah foreground dapat menerima SIGINT.
- `tsh <username>` — delegasi ke `/bin/login.js` (login adalah SetUID root, sehingga bisa autentikasi & ganti user).
- Resize (`SIGWINCH` / event `RESIZE`) memperbarui env `LINES/COLUMNS` tanpa menimpa layar aplikasi foreground.

Dua gaya aplikasi

Ada dua gaya menulis aplikasi TSIX:

1. Class IProgram (legacy mayoritas) — `export class main` yang mengimplementasikan `IProgram`, dengan method `execute(lib, args)`. Semua API diakses lewat parameter `lib`:

```
export class main implements IProgram {
  async execute(lib: OSContext, args: string[]) {
    // ...
  }
}
```

Contoh: `ls`, `cat`, `ps`, `kill`, `tsh`.

2. Program() wrapper + proxy singletons (baru) — `import singleton std, fs, shell, net, db` dari `@tsix/Application`, lalu bungkus fungsi utama dengan `Program(fn)`:

```
import { Program, std, fs, shell, net } from "@tsix/Application";

export default Program(async (args) => {
  // ...
});
```

Contoh: `esp-send`, `dome`, `iot-dashboard`.

[!NOTE] **Cara kerja Program()** — `Program(fn)` mengembalikan class anonim yang implements `IProgram`. Method `execute` menyimpan `OSContext` ke `global._tsix0sc`, memanggil `fn(args)`, dan jika `fn` melempar error,

memanggil `std.error(error.stack, appName)` lalu me-re-throw. Singleton `std/fs/shell/net/db` adalah Proxy yang meneruskan akses ke `_tsixLib` pada thread saat ini. `os.pid` dan `os.rand` juga tersedia.

Perbandingan kedua gaya:

Aspek	Class IProgram (legacy)	Program() wrapper (baru)
Bentuk	Class <code>main</code> + method <code>execute()</code>	Fungsi anonim dibungkus <code>Program(fn)</code>
Signature	<code>execute(lib: OSContext, args: string[])</code>	<code>fn(args: string[])</code>
Akses API	Lewat parameter <code>lib</code> (<code>lib.std</code> , <code>lib.fs</code> , ...)	Import proxy singleton (<code>std</code> , <code>fs</code> , <code>shell</code> , <code>net</code> , <code>db</code>)
Argumen	Parameter kedua	Parameter tunggal fungsi
Error handling	Manual (try/catch sendiri)	Otomatis → <code>std.error()</code> lalu re-throw
Implementasi	<code>implements IProgram</code> langsung	<code>Program()</code> mengembalikan class <code>implements IProgram</code>
Contoh	<code>ls</code> , <code>cat</code> , <code>ps</code> , <code>kill</code> , <code>tsh</code>	<code>esp-send</code> , <code>dome</code> , <code>iot-dashboard</code>

Error = "Window"

`std.error()` melakukan 4 hal:

1. Tulis ke `/var/log/syslog`.
2. Cari `fileHint` dari stack trace (melewati `UserLib`, `Application`, `emerald`).
3. Broadcast ke parent via IPC — pesan `{ type: "GUI_WINDOW_ERROR", wid, pid, file, error, context, timestamp }`.
4. Print ke TTY dengan warna merah `[ERROR]`.

Jadi ketika aplikasi crash, **window error muncul di desktop** — bukan hanya teks di terminal.

Alur / Cara Kerja

Boot: Kernel → init (PID 1)

```
Kernel (main.ts) - kernel.runInit()
  resolve /bin/init.js
  createProcess("init", fds:[tty1×3]) → PID 1
  setForegroundProcess(1, 1)

[init.ts - Worker Thread, Ring 4]
1. Enforce setuid
  chmod /bin/passwd.js 2541 (0o4755)
  chmod /bin/sudo.js 2541
2. RSA identity
  cek /etc/keys/rsa/id_rsa.pub
  ada → verify + fingerprint
  tidak ada → generateKeyPair() → simpan id_rsa + id_rsa.pub
  fingerprint + color bar → ioctl(1, 33, colorBar) (SET_VISUAL_IDENTITY)
3. Exec /etc/rc.local.js + waitpid
4. for (i = 2; i <= 6; i++) spawnLogin(i)
  exec /bin/login.js (ttyId=i)
  monitorProcess → respawn saat login mati (delay 1s anti-crash-loop)
5. Banner ASCII + fingerprint color bar
6. spawnLogin(1) ← foreground TTY1
7. while (true) { sleep 10s } ← init tidak boleh exit
```

Login: TTY → shell

```
login.ts (satu proses per TTY)
1. username = args[0] || readLine("Username: ")
2. password = readPassword("Password: 🐙") ← masked, raw mode
3. /etc/passwd → uid(3), gid(4), home(6), shell(7)
4. /etc/shadow → hash bcrypt (field 2)
```



```

5. bcrypt.compareSync(password, hash)
   gagal → "Login: Invalid username or password" → ulangi dari langkah 1
6. MOTD: /etc/motd + frasa acak /etc/motd.json
7. /etc/group → supplementary gids (grup yang memuat username)
8. setgroups(gids) → setgid(gid) → setuid(uid) ← urutan POSIX
9. setenv USER / HOME → chdir(home)
10. exec(shell dari /etc/passwd) → waitpid → break (satu sesi)

```

Login WM: GUI → desktop

```

asteracea.ts (satu proses = login manager + desktop)
1. WM start sebagai root (di-spawn init)
2. Tampil login screen (GUI)
3. Verifikasi: spawn /bin/login.js --verify <user> <pass> <file> ← SetUID root
   baca hasil file: "OK" / "FAIL:..."
4. /etc/passwd → uid(3), gid(4), home(6) ← world-readable, baca langsung
5. /etc/group → supplementary gids
6. Kalau WM belum root (sudah pernah login non-root) → setuid(0) ← restore via Saved UID
7. setgroups(gids) → setgid(gid) → setuid(uid) ← urutan POSIX (drop)
8. setenv USER / HOME → chdir(home)
9. Tampil desktop – WM JADI user tsb (taskbar, launcher, wallpaper)
10. Logout → kembali ke langkah 2 (WM tetap hidup; identitas di-reset via Saved UID)

```

Kode Sumber

File	Peran
src/mirror/bin/init.ts	PID 1 — init hardcoded (setuid, RSA, rc.local, spawn login)
src/mirror/bin/login.ts	Autentikasi passwd+shadow (bcrypt) → setgroups/setgid/setuid → exec shell
src/mirror/bin/tsh.ts	Shell interaktif (gaya IPprogram)
src/mirror/opt/asteracea/asteracea.ts	WM + login manager GUI — verifikasi via login.js --verify, re-elevate via Saved UID
src/mirror/lib/UserLib.ts	Framework: std, fs, shell, net, db (sub-library)
src/mirror/lib/Application.ts	Program() wrapper + proxy singleton
src/mirror/lib/IProgram.ts	Interface IPprogram & OSContext

Snippet (level kode)

Semua snippet di bawah disalin dari sumber — *kode adalah kebenaran*.

Init — enforce setuid

```

// Runtime mengeksekusi sidecar .js (bukan source .ts),
// jadi chmod harus di .js. Nilai 2541 = 0o4755 (setuid root).
await lib.fs.chmod("/bin/passwd.js", 2541);
await lib.std.print(`${ok} [INIT] SetUID bit applied to: /bin/passwd.js\n`);
await lib.fs.chmod("/bin/sudo.js", 2541);
await lib.std.print(`${ok} [INIT] SetUID bit applied to: /bin/sudo.js\n`);

```

ok adalah prefix berwarna hijau [OK] yang didefinisikan di awal execute() di init.ts.

Init — spawnLogin (TTY2-6)

```

const spawnLogin = async (ttyId: number) => {
  try {
    if (ttyId > 1)
      await lib.std.print(`${ok} Initializing session on TTY${ttyId}...\n`);
  }
}

```

```

    const result = await lib.shell.exec("/bin/login.js", [], undefined, undefined, ttyId);
    if (result && result.pid) {
        terminals.set(ttyId, result.pid);
        await lib.std.log(`Login service spawned on TTY${ttyId} (PID ${result.pid})`, "init");
        // Kita nungguin di background (thread terpisah di Worker)
        this.monitorProcess(lib, ttyId, result.pid, spawnLogin);
    }
} catch (e: any) {
    await lib.std.print(`Init: Error spawning login on TTY${ttyId}: ${e.message}\n`);
}
};

// Start login on all TTYS (TTY2-6)
for (let i = 2; i <= 6; i++) {
    await spawnLogin(i);
}

```

Init — monitorProcess (respawn anti-crash-loop)

```

private async monitorProcess(
    lib: UserLib, ttyId: number, pid: number,
    respawn: (tty: number) => Promise<void>,
) {
    const exitCode = await lib.shell.waitpid(pid);
    await lib.std.print(
        `Init: Process on TTY${ttyId} (PID ${pid}) exited with code ${exitCode}. Respawning...\n`,
    );
    // Delay sedikit biar nggak spam kalau crash loop
    await new Promise(r => setTimeout(r, 1000));
    await respawn(ttyId);
}

```

Login — verifikasi bcrypt

```

// 1. Cari user di /etc/passwd → uid, gid, home, shell
const parts = userEntry.split(":");
const uid = parseInt(parts[2]);
const gid = parseInt(parts[3]);
const home = parts[5];
const userShell = parts[6]; // shell dari /etc/passwd field 7

// 2. Ambil hash dari /etc/shadow → bcrypt.compareSync
const hash = shadowEntry.split(":")[1];
const match = bcrypt.compareSync(password, hash);
if (!match) {
    await lib.std.print("Login: Invalid username or password\n");
    continue; // kembali ke loop → prompt ulang
}

```

Login — setgroups → setgid → setuid (urutan POSIX)

```

// 3.5. Resolve Supplementary Groups dari /etc/group
const supplementaryGids: number[] = [gid]; // dimulai dari primary gid
try {
    const groupContent = await lib.fs.readFile("/etc/group") || "";
    const groupLines = groupContent.split("\n")
        .map(l => l.trim()).filter(l => l.length > 0);
    for (const gLine of groupLines) {
        const gParts = gLine.split(":");
        const groupGid = parseInt(gParts[2]);
        const groupUsers = gParts[3] ? gParts[3].split(",") : [];
        if (groupUsers.includes(username.trim()) && groupGid !== gid) {
            supplementaryGids.push(groupGid);
        }
    }
} catch (e) {
    // Ignore group errors
}

```

```
// Set Identity – GID harus SEBELUM UID.
// Setelah UID menjadi non-root, proses kehilangan privilege untuk mengubah GID.
await lib.shell.setgroups(supplementaryGids);
await lib.shell.setgid(gid);
await lib.shell.setuid(uid);

// Set Env + exec shell
await lib.shell.setenv("USER", username.trim());
await lib.shell.setenv("HOME", home);
await lib.shell.chdir(home);
const result = await lib.shell.exec(userShell); // shell dari /etc/passwd
if (result && result.pid) {
  await lib.shell.waitpid(result.pid);
}
```

Login WM — verifikasi via login.js --verify (kanal file)

```
// asteracea.ts – verifikasi password lewat helper SetUID root.
// WM non-root tidak bisa baca /etc/shadow (0640 root) → delegasikan ke login.js.
const verifyOut = `/tmp/verify-${Date.now()}-${Math.random().toString(36).slice(2, 8)}.txt`;
let authOk = false;
const vp = await shell.exec("/bin/login.js", ["--verify", u, password, verifyOut]);
if (vp && vp.pid) {
  await shell.waitpid(vp.pid); // tunggu selesai
  const res = (await fs.readFile(verifyOut)) || "";
  authOk = String(res).trim() === "OK"; // "OK" / "FAIL:..."
}
await fs.unlink(verifyOut); // bersihkan temp
```

```
// login.ts – sisi helper (dijalankan sebagai SetUID root, jadi bisa baca shadow)
if (args[0] === "--verify") {
  const vUser = (args[1] || "").trim();
  const vPass = args[2] || "";
  const vOut = args[3] || "/tmp/verify-result.txt";
  let ok = false, errMsg = "";
  try {
    const shadow = (await lib.fs.readFile("/etc/shadow")) || "";
    const entry = shadow.split("\n").map(l => l.trim()).filter(l => l)
      .find(l => l.split(":")[0] === vUser);
    const hash = entry ? entry.split(":")[1] : "";
    ok = !!hash && bcrypt.compareSync(vPass, hash);
    if (!hash) errMsg = "account not found or no password";
  } catch (e) { errMsg = e.message || "verify error"; }
  await lib.fs.writeFile(vOut, ok ? "OK" : "FAIL:" + errMsg);
  return; // selesai – WorkerEntry yang menuntaskan proses
}
```

Saved UID (kernel) — setuid boleh restore ke suid

```
// Syscalls.ts – SETUID dengan Saved UID (mirip setuid/seteuid Unix)
if (this.isRoot(pcb)) {
  pcb.suid = pcb.uid; // root bebas ganti UID; simpan hak kembali (0 utk root)
  pcb.uid = newUid;
  pcb.ruid = newUid;
} else if (newUid === pcb.suid) {
  pcb.uid = newUid; // non-root hanya boleh RESTORE ke Saved UID (mis. balik ke root)
  pcb.ruid = newUid;
} else {
  throw new Error("Permission Denied: Only root or root group members can change UID");
}

// Scheduler.ts – createProcess: suid default = uid proses itu sendiri
suid: options.suid ?? options.uid ?? 0, // app biasa TIDAK bisa escalate
```

App — gaya IProgram (legacy)

```
// greet.ts — gaya klasik: class main implements IProgram
import { IProgram, OSContext } from "@tsix/IProgram";

export class main implements IProgram {
  async execute(lib: OSContext, args: string[]): Promise<string | void> {
    const name = args[0] || "dunia";
    await lib.std.print(`Halo, ${name}!\n`);
    await lib.std.print(`CWD: ${await lib.shell.getcwd()}\n`);
  }
}
```

App — gaya Program() wrapper (baru)

```
// halo.ts — gaya baru: Program() wrapper + proxy singleton
import { Program, std, os } from "@tsix/Application";

export default Program(async (args) => {
  const name = args[0] || "dunia";
  await std.print(`Halo, ${name}!\n`);
  await std.print(`PID: ${os.pid}\n`);
  return "Selesai.";
});
```

Latihan / Praktik

1. Login sebagai `root`, lalu jalankan `ps` — identifikasi proses login per TTY (`/bin/login.js`).
2. Dari shell `root`, jalankan `tsh tamu` — amati delegasi login (`setuid root`) dan shell baru dengan user berbeda.
3. Baca `src/mirror/bin/login.ts` — trace urutan `setgroups/setgid/setuid` dan jelaskan kenapa urutan ini wajib.
4. Baca `src/mirror/bin/init.ts` — temukan `monitorProcess`; bunuh proses login di sebuah TTY (`kill <pid>` dari `root`) lalu amati `init` me-respawn dalam ± 1 detik.
5. Tulis app gaya `IProgram` dan app gaya `Program()` — bandingkan struktur keduanya (lihat Snippet).
6. Buat app yang memanggil `std.error()` — amati munculnya window error di desktop (pesan `GUI_WINDOW_ERROR`).

Referensi

- [wiki/Memulai.md](#), [wiki/Perintah-Sistem.md](#), [wiki/Panduan-Developer.md](#)
- [wiki/course/00-overview.md §9](#) (Userland: `init`, `shell`, `aplikasi`)
- [src/mirror/bin/init.ts](#), [src/mirror/bin/login.ts](#), [src/mirror/bin/tsh.ts](#)
- [src/mirror/lib/UserLib.ts](#), [src/mirror/lib/Application.ts](#), [src/mirror/lib/IProgram.ts](#)

Modul 14 — selesai. Bagian V tuntas. Lanjut ke [Modul 15 — Networking MQTNL](#).

RFC-TSIX-EDU-002 | Modul kelima belas kurikulum TSIX. Memahami MQTNL (MQTT Network Layer) — protokol networking TSIX yang memakai MQTT pub/sub sebagai wire, bukan IP/routing.

Alih-alih TCP/IP, MQTNL memakai **MQTT pub/sub sebagai wire**. Alamat = **nama node** (string), port = **endpoint aplikasi** (PortManager). Koneksi bersifat connectionless/UDP-like — tidak ada listen/accept sungguhan, hanya emulasi polling.

Tujuan Pembelajaran

- ☐ Menjelaskan konsep alamat = nama node, port = endpoint aplikasi
- ☐ Menjelaskan alur socket: bind → sendto → recvfrom
- ☐ Menjelaskan peran PortManager dan releasePortsByPid
- ☐ Menjelaskan mengapa tidak ada listen/accept (emulasi polling)
- ☐ Menyebutkan syscall networking (30–34)

Konsep Inti

Model: MQTT pub/sub adalah "wire"-nya

MQTNL tidak memakai TCP/IP. Tidak ada alamat IP, tidak ada routing, tidak ada koneksi stateful. Sebagai gantinya, TSIX memakai **MQTT pub/sub sebagai media transport (wire)**:

- **Setiap node** terhubung ke **MQTT broker** yang sama.
- **Alamat = nama node** (string): "tsix", "tsix-node-2", "esp3253". Nama ini unik per perangkat.
- **Topic** = `mqtnl@1.0/<dstAddress>` (protokol JSON) / `mqtnl@1.1/<dstAddress>` (protokol Binary). Isi topic adalah **paket MQTNL** (header + payload).
- **Semua node subscribe** `mqtnl@1.x/#`, lalu **menyaring** paket berdasarkan `dstAddress` (atau "*" untuk broadcast). Paket untuk node lain di-drop.

Pengiriman bersifat fire-and-forget tanpa sesi — karena itu model ini **connectionless / UDP-like**.

Alamat & Port

Konsep	Nilai	Analog TCP/IP
Alamat	Nama node (string)	IP address
Port	Endpoint aplikasi (0–65535)	Port TCP/UDP
Wire	Topic MQTT pub/sub	Paket IP
Koneksi	Connectionless (fire-and-forget)	UDP

Setiap `sendto()` membuat paket mandiri. Header paket membawa alamat & port tujuan, jadi tidak perlu koneksi yang dijaga.

Tiga Komponen Utama

1. **SimpleMQTNLDriver** — *network interface* (`/dev/smqtnl0`, `/dev/smqtnl1`). Menjaga koneksi ke broker MQTT, publish paket keluar, menerima & menyaring paket masuk, fragmentasi 32KB, reassembly, dan dekripsi. Tiap instance punya `localAddress` (nama node) sendiri.
2. **SocketDevice** — abstraksi satu socket: buffer data masuk + daftar waiter. Dibuat per syscall `SOCKET` dan disimpan di FD table. Bersifat pasif — data masuk di-push oleh driver, aplikasi membacanya lewat `recvfrom`.

3. **PortManager** — penjaga port virtual 0-65535. `allocatePort()` untuk bind eksplisit, `allocateRandomPort(10000-20000)` untuk `bind(0)`, `releasePortsByPid()` saat proses exit.

Per-port handler

Ketika aplikasi `bind(port)`, kernel memanggil `driver.registerHandler(port, cb)`. Handler ini menyalurkan data masuk ke socket milik aplikasi:

```
bind(port)
  → PortManager.allocatePort(port, pid)
  → socket.setPort(port); socket.driver = targetDriver
  → driver.registerHandler(port, (data) => socket.push(data))
```

Saat paket tiba, driver memanggil handler pada `dstPort`. Inilah **dispatch per-port**: satu driver melayani banyak port sekaligus.

Alur socket (ringkas)

```
App → socket() → bind(port) → driver.registerHandler(port)
  → sendto(addr, port, data) → driver.send → publish topic
  → MQTT broker → semua node subscribe 'mqtnl@1.x/#'
  → filter dstAddress → reassembly → decrypt → socket.push()
  → recvfrom() → buffer.shift()
```

Syscall

```
SOCKET=30, BIND=31, SENDTO=32, RECVFROM=33, NETSTAT=34
```

Tidak ada `listen/accept` di level syscall. UserLib meng-emulasi: `listen() = socket() + bind()`; `accept() = polling recv()` sampai ada paket pertama.

Alur / Cara Kerja

Alur pengiriman dari **App A** di node "tsix" (port N) ke **App B** di node "esp32S3" (port M) melalui MQTT broker:



Langkah detail:

1. **SOCKET** — App A dan App B memanggil `socket()`. Kernel membuat `SocketDevice` baru dan menyisipkannya ke FD table (context: "socket").
2. **BIND** — App A memanggil `bind(fd, N)`. Kernel menentukan driver target (default `smqtnl0`), memesan port N lewat `PortManager.allocatePort(N, pid)`, lalu `registerHandler(N, cb)` yang menyalurkan data ke socket A.

- 3. **SENDTO** — App A memanggil `sendto(fd, "esp32S3", M, data)`. Kernel memanggil `driver.send("esp32S3", M, data, FLAG_DATA, N)`.
- 4. **Fragmentasi** — `send()` memecah payload menjadi chunk 32KB bila perlu, lalu membungkus tiap chunk dalam paket ber-header 9 field.
- 5. **Publish** — driver publish tiap chunk ke topic `mqtnl@1.0/esp32S3` (`qos: 0, retain: false`).
- 6. **Broker & filter** — semua node subscribe `mqtnl@1.x/#` menerima paket. Tiap driver menyaring: paket hanya diproses bila `dstAddress` sama dengan `localAddress` (atau `"*"`). Paket lain di-drop. Paket yang dikirim sendiri (`src = local`) juga diabaikan.
- 7. **Reassembly & dekripsi** — driver tujuan merakit chunk (sesi per `srcAddr:srcPort → dstAddr:dstPort`, TTL 30s), lalu mendekripsi payload via `SecurityAgent` (port dapat di-upgrade ke ChaCha20-Poly1305 via `ioctl 0x1001`).
- 8. **Dispatch** — driver memanggil handler pada `dstPort → socket.push({ src, port, localPort, data, isBinary, ts }) → data` masuk buffer socket.
- 9. **RECVFROM** — App B memanggil `recv(fd)`. Kernel membaca buffer (`read() = buffer.shift()`). Jika kosong, kernel menunggu event-driven (`waitForData`) sampai data di-push.

[!NOTE] **Loopback lokal.** Jika `dstAddress` sama dengan alamat node pengirim (atau `"localhost"`), paket tidak keluar ke broker — driver meneruskannya langsung ke instance driver lokal (`SimpleMQTNLDriver.findLocal()`), mirip `localhost` di OS sungguhan.

Kode Sumber

File	Peran
<code>src/kernel/devices/SimpleMQTNLDriver.ts</code>	Network interface MQTNL
<code>src/kernel/devices/SocketDevice.ts</code>	Socket = device (everything is a file)
<code>src/kernel/PortManager.ts</code>	Alokasi port virtual
<code>src/mirror/lib/NetworkLib.ts</code>	API <code>lib.net</code> (legacy, <code>OSContext</code> -based)
<code>src/mirror/lib/UserLib.ts</code>	<code>lib.net</code> inline (baru, <code>this.dispatch</code>)
<code>src/kernel/Syscalls.ts</code>	Implementasi syscall <code>SOCKET-NETSTAT</code> (30–34)
<code>src/sysconfig.json</code>	Konfigurasi interface network default

[!WARNING] **Dual NetworkLib.** Ada dua implementasi `lib.net`: inline di `UserLib.ts` (baru) dan `NetworkLib.ts` (legacy, `OSContext`-based). Keduanya menuju syscall yang sama.

Interface network default (`sysconfig.json`)

Saat boot, kernel membaca `cfg.network.interfaces` dan membuat satu `SimpleMQTNLDriver` per entri (`Kernel.ts → "Initialize Network Interfaces"`). Tiap interface punya **deviceName** (nama `/dev`), **address** (nama node), dan **broker**:

deviceName	address (nama node)	broker	defaultPort
<code>smqtnl0</code>	<code>tsix</code>	<code>mqtt://192.168.1.204</code>	1883
<code>smqtnl1</code>	<code>tsix-node-2</code>	<code>mqtt://192.168.1.204</code>	1883

`cfg.network.defaultDevice = smqtnl0`. Jika `bind()` tanpa argumen `address`, kernel memakai interface default ini.

Snippet (level kode)

Semua snippet di bawah **disalin dari sumber** — "kode adalah kebenaran". Bagian yang dipangkas ditandai `// ...`.

NetworkLib — pemakaian dari sisi app (UserLib.ts)

`lib.net` adalah instance `NetworkLib` pada `UserLib` (aplikasi baru) dan pada `NetworkLib.ts` (aplikasi legacy berbasis `OSContext`). Socket dikembalikan sebagai **fd** (angka), bukan objek:

```
export class NetworkLib {
  constructor(
    private dispatch: (code: SyscallCode, args: any) => Promise<any>,
  ) { }

  public async socket(): Promise<number> {
    return await this.dispatch(SyscallCode.SOCKET, null);
  }

  public async bind(
    fd: number,
    port: number,
    address?: string,
  ): Promise<boolean> {
    return await this.dispatch(SyscallCode.BIND, { fd, port, address });
  }

  // listen() = socket + bind — emulasi, tidak ada syscall LISTEN
  public async listen(port: number): Promise<number> {
    const fd = await this.socket();
    const ok = await this.bind(fd, port);
    return ok ? fd : -1;
  }

  // accept() = polling recv() sampai ada paket pertama
  public async accept(serverFd: number): Promise<any> {
    while (true) {
      const pkt = await this.recv(serverFd);
      if (pkt) {
        return {
          fd: serverFd,
          src: pkt.src,
          port: pkt.port,
          localPort: pkt.localPort || 0,
          firstPkt: pkt,
        };
      }
      await new Promise((r) => setTimeout(r, 100));
    }
  }

  public async sendto(
    fd: number,
    address: string,
    port: number,
    data: any,
    flag: number = 0,
    srcPort: number = 0,
  ): Promise<boolean> {
    return await this.dispatch(SyscallCode.SENDTO, {
      fd,
      address,
      port,
      data,
      flag,
      srcPort,
    });
  }

  public async recv(fd: number): Promise<any> {
    return await this.dispatch(SyscallCode.RECVFROM, fd);
  }
}
```

Contoh pemakaian (pola server-client):


```
// Server di node "tsix"
const fd = await lib.net.listen(8080);           // socket + bind(8080)
const client = await lib.net.accept(fd);         // polling recv → { src, port, firstPkt }
const data = await lib.net.recv(fd);             // baca data

// Client mengirim ke node "esp32S3", port 5000
await lib.net.sendto(fd, "esp32S3", 5000, JSON.stringify({ cmd: "ping" }));
```

PortManager — bind & release (PortManager.ts)

```
public allocatePort(port: number, pid?: number): boolean {
  if (port < 0 || port > 65535) return false;
  if (this.usedPorts.has(port)) return false;

  this.usedPorts.add(port);
  if (pid !== undefined) this.portOwner.set(port, pid);
  return true;
}

public allocateRandomPort(min: number = 10000, max: number = 20000): number | null {
  for (let i = 0; i < 100; i++) {
    const port = Math.floor(Math.random() * (max - min + 1)) + min;
    if (!this.usedPorts.has(port)) {
      this.usedPorts.add(port);
      return port;
    }
  }
  return null;
}

public releasePortsByPid(pid: number): void {
  const toRelease: number[] = [];
  this.portOwner.forEach((owner, port) => {
    if (owner === pid) toRelease.push(port);
  });
  for (const port of toRelease) {
    this.usedPorts.delete(port);
    this.portOwner.delete(port);
  }
}
```

[!TIP] `releasePortsByPid()` dipanggil saat proses exit — **jaring pengaman** jika aplikasi lupa melepas port.

SocketDevice — buffer & handler dispatch (SocketDevice.ts)

```
public push(data: any) {
  this.buffer.push(data);
  // Bangunkan semua reader yang sedang nunggu data (event-driven)
  while (this.waiters.length > 0) {
    const w = this.waiters.shift();
    w();
  }
}

public read(): any {
  return this.buffer.shift() || null;
}

public async waitForData(timeoutMs: number): Promise<boolean> {
  if (this.buffer.length > 0) return true;
  return new Promise<boolean>((resolve) => {
    let timer: ReturnType<typeof setTimeout> | undefined;
    const onData = () => {
      if (timer) clearTimeout(timer);
      const idx = this.waiters.indexOf(onData);
      if (idx >= 0) this.waiters.splice(idx, 1);
      resolve(true);
    };
  });
}
```

```

    });
    timer = setTimeout(() => {
        const idx = this.waiters.indexOf(onData);
        if (idx >= 0) this.waiters.splice(idx, 1);
        resolve(false);
    }, timeoutMs);
    this.waiters.push(onData);
});
}

```

SimpleMQTNLDriver — init & send (SimpleMQTNLDriver.ts)

```

public init(ctx: KContext): void {
    this.client = mqtt.connect(this.brokerUrl);

    this.client.on("connect", () => {
        for (const proto of this.protocols) {
            const prefix = proto.getTopicPrefix();
            this.client?.subscribe(`${prefix}#`); // subscribe "mqtnl@1.0/#" & "mqtnl@1.1/#"
        }
    });

    this.client.on("message", (topic, message) => {
        this.handleIncomingMessage(topic, message);
    });
}

```

```

public async send(address: string, port: number, data: any, flag: PacketFlags = PacketFlags.FLAG_DATA, srcPort: number = 0) {
    // [LOOPBACK] Reserved alias "localhost" selalu menunjuk ke interface
    // pengirim sendiri – mirip localhost di OS sungguhan.
    if (address === "localhost") {
        address = this.localAddress;
    }

    // [LOOPBACK] Kalau tujuan adalah alamat lokal (milik node ini), kirim
    // langsung ke receiver tanpa keluar ke broker MQTT.
    const localTarget = SimpleMQTNLDriver.findLocal(address);

    // Cek koneksi hanya untuk paket yang benar-benar keluar ke broker.
    if (!localTarget && (!this.client || !this.client.connected)) return false;

    // ... (normalisasi payload, pemilihan protokol JSON/Binary, enkripsi,
    //      fragmentasi 32KB, header 9 field) – lihat file sumber

    const topic = `${prefix}${address}`; // mis. "mqtnl@1.0/esp32S3"

    for (let i = 0; i < packetCount; i++) {
        // ... (bungkus packet, protocol.pack)
        if (localTarget) {
            localTarget.handleIncomingMessage(topic, payloadBuffer); // LOOPBACK lokal
        } else {
            await new Promise((resolve) => {
                this.client!.publish(topic, payloadBuffer, { qos: 0, retain: false }, (err) => {
                    if (err) this.logger.error(`MQTT Publish error: ${err.message}`);
                    resolve(true);
                });
            });
        }
    }
    return true;
}

```

BIND — menghubungkan socket ke driver & port (Syscalls.ts)

```

case SyscallCode.BIND: {
    const { fd, port, address } = args as {
        fd: number;
        port: number;
        address?: string;
    };
}

```

```

};
const entry = pcb.fdTable[fd];
if (!entry || !(entry.device instanceof SocketDevice))
    throw new Error("Invalid Socket FD");

const socket = entry.device as SocketDevice;
if (socket.bound) throw new Error("Socket already bound");

// ... pilih targetDriver (dari `address`, atau cfg.network.defaultDevice)

const portMgr = this.kernel.getPortManager();
let actualPort = port;

if (port === 0) {
    const randomPort = portMgr.allocateRandomPort();
    if (randomPort === null) throw new Error("No random ports available");
    actualPort = randomPort;
} else {
    if (!portMgr.allocatePort(port, pcb.pid))
        throw new Error(`Port ${port} already in use`);
}

socket.setPort(actualPort);
socket.driver = targetDriver;

// Register handler
targetDriver.registerHandler(actualPort, (data) => {
    socket.push(data);
});

return true;
}

```

Latihan / Praktik

1. Baca `wiki/Networking-MQTNL.md` — alur lengkap dan topik.
2. Baca `src/kernel/PortManager.ts` — pahami `allocateRandomPort` dan `releasePortsByPid`.
3. Jalankan dua node (mis. `tsix` dan `esp32S3` di MQTT broker) — kirim pesan antar keduanya.
4. Baca `src/kernel/devices/SocketDevice.ts` — jelaskan bagaimana socket menjadi `IDevice`.
5. Dari aplikasi, jalankan `lib.net.netstat()` — bandingkan hasilnya dengan tabel interface di `sysconfig.json`.

Referensi

- `wiki/Networking-MQTNL.md` — dokumentasi lengkap MQTNL
- `wiki/course/00-overview.md §7` — Networking MQTNL
- `src/kernel/devices/SimpleMQTNLDriver.ts` — network interface
- `src/kernel/devices/SocketDevice.ts` — socket = device
- `src/kernel/PortManager.ts` — alokasi port virtual
- `src/kernel/Syscalls.ts` — implementasi syscall SOCKET-NETSTAT (30-34)
- `src/sysconfig.json` — interface network default

Modul 15 — selesai. Lanjut ke [Modul 16 — Wire Protocol MQTNL](#).

RFC-TSIX-EDU-002 | Modul keenam belas kurikulum TSIX. Memahami format data di atas MQTT: dual protocol JSON vs Binary, deteksi magic byte, dan fragmentasi.

MQTNL punya **dua protokol wire**: JSON v1.0 (readable) dan Binary v1.1 (kompak, tanpa enkripsi — untuk OTA byte-exact). Driver mendeteksi protokol dari **magic byte pertama** per srcAddress.

Tujuan Pembelajaran

- ☐ Menjelaskan kontrak `IMQTNLProtocol`
- ☐ Membedakan protocol JSON v1.0 vs Binary v1.1
- ☐ Menjelaskan deteksi magic byte
- ☐ Menjelaskan fragmentasi 32KB dan reassembly TTL 30s
- ☐ Menjelaskan mengapa userland tidak menyentuh protokol langsung

Konsep Inti

Kontrak `IMQTNLProtocol`

Semua protokol wire MQTNL mengimplementasi satu kontrak. Interface ini adalah "satu-satunya pintu" antara driver dan wire format — menambah protokol baru (mis. v2) cukup dengan mengimplementasi class baru, tanpa mengubah driver.

```
export interface IMQTNLProtocol {
    /** Nama protokol (misal: "JSON", "Binary") */
    getName(): string;
    /** Byte pertama penanda protokol dalam byte stream.
     *  JSON: '[' (0x5B). Binary: 'B' (0x42). */
    getMagicChars(): number[];
    /** MQTT Topic Prefix (misal: "mqtnl@1.0/", "mqtnl@1.1/") */
    getTopicPrefix(): string;
    /** Objek packet internal → Buffer/String untuk dikirim ke broker MQTT */
    pack(packet: any): Buffer | string;
    /** Raw data yang diterima → objek packet internal */
    unpack(data: Buffer | string): any;
}
```

Header paket (9 field + payload)

Struktur paket internal (objek yang di-pack/unpack) identik untuk kedua protokol. Bedanya hanya cara field ditulis ke wire.

Field	JSON (index array)	Binary (offset)	Tipe
srcAddress	0	S_ADDR_LEN + S_ADDR	string
srcPort	1	S_PORT	u16 LE
dstAddress	2	D_ADDR_LEN + D_ADDR	string
dstPort	3	D_PORT	u16 LE
packetCount	4	PACKET_COUNT	u16 LE
packetIndex	5	PACKET_INDEX	u16 LE
dataSize	6	DATA_SIZE	u32 LE

Field	JSON (index array)	Binary (offset)	Type
packetHeaderFlag	7	FLAG	u8
forwarded	8	FORWARDED	u8
payload	9	PAYLOAD (sisanya buffer)	string / Buffer

[!NOTE] packetCount dan packetIndex adalah bagian dari header (bukan fitur terpisah). packetCount = jumlah total chunk, packetIndex = nomor chunk (0-based). dataSize = ukuran payload **total** sebelum dipecah (bukan ukuran per chunk).

Dual protocol JSON vs Binary

Aspek	JSON v1.0	Binary v1.1
Class	MQTNLProtocolJSON	MQTNLProtocolBinary
Topic prefix	mqtnl@1.0/<addr>	mqtnl@1.1/<addr>
Magic byte	[(0x5B)	B (0x42) + ver 0x01
Bentuk wire	JSON array (teks, readable)	Buffer biner kompak, byte-exact
Keterbacaan	mudah dibaca manusia	sulit dibaca (biner)
Ukuran	lebih besar (overhead teks)	lebih kecil & presisi
Enkripsi	RSA handshake + ChaCha20-Poly1305	tanpa enkripsi (plain, untuk OTA)
Pakai untuk	komunikasi umum (default legacy)	transfer firmware OTA byte-exact

 Handshake RSA: SYN → SYN-ACK → ACK → DATA terenkripsi Sumber: wiki/diagram/Networking-MQTNL-2.mmd

Format frame Binary v1.1

Diagram byte-per-byte dari frame biner (N = panjang srcAddress, M = panjang dstAddress, L = panjang payload):

```

| 0x42 | 0x01 | len | <src addr> | srcPort | len | <dst addr> | dstPort |
| MAGIC| VER | sAL=N | srcAddr (N b) | u16LE | dAL=M | dstAddr (M b) | u16LE |

| pktCount | pktIndex | dataSize | flag | fwd | <payload> |
| u16LE | u16LE | u32LE | u8 | u8 | payload (L b) |

```

- Ukuran header tetap = 18 byte + N + M (2 + 1 + N + 2 + 1 + M + 2 + 2 + 2 + 4 + 1 + 1).
- Contoh: srcAddress="tsix" (N=4), dstAddress="esp32S3" (M=7) → header = 29 byte.
- Urutan pembacaan seluruh field deterministik (tanpa delimiter), jadi panjang alamat wajib dikirim (S_ADDR_LEN / D_ADDR_LEN).
- **Tidak ada CRC/checksum** di dalam frame. Integritas OTA dijaga oleh dataSize (u32) dan jalur raw yang byte-exact.

Deteksi magic byte

Driver tidak menebak protokol dari topic. Ia membaca **byte pertama** payload MQTT lalu mencocokkannya ke getMagicChars() tiap protokol terdaftar:

```

// Detect protocol by Magic Byte (di SimpleMQTNLDriver.handleIncomingMessage)
const magicByte = Buffer.isBuffer(message) ? message[0] : (message as string).charCodeAt(0);
const proto = this.protocols.find(p => p.getMagicChars().includes(magicByte));
if (!proto) {
  this.logger.error(`Protocol mismatch on ${topic}. Magic: 0x${message[0].toString(16)}`);
  return;
}
packet = proto.unpack(message);

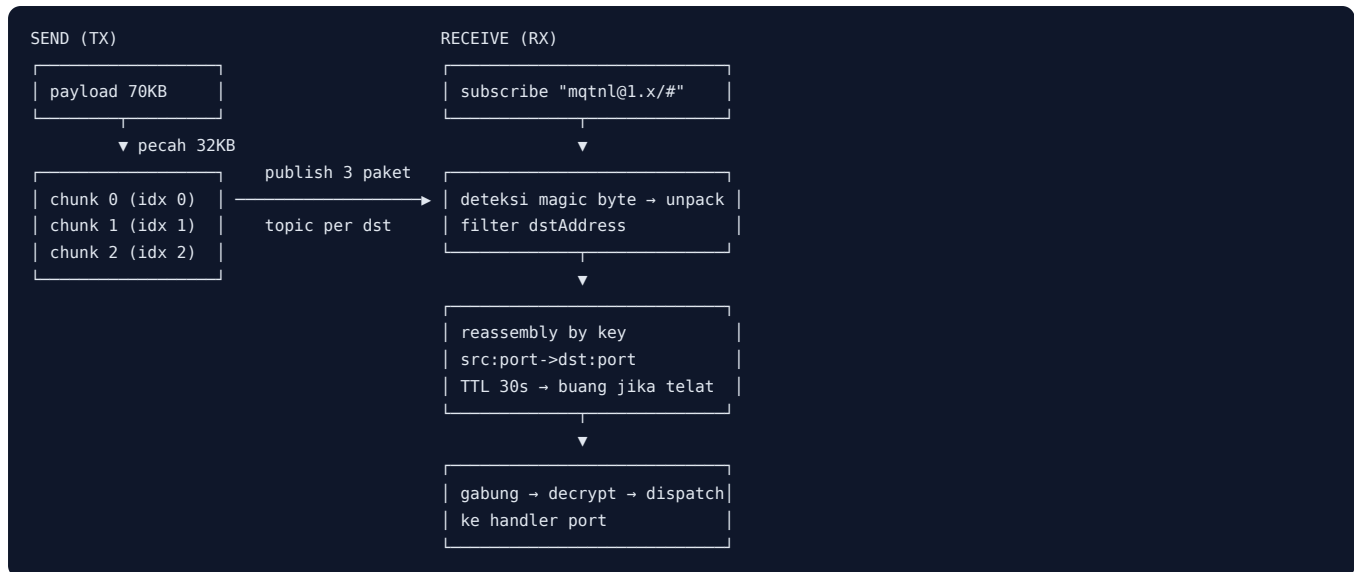
```

```
// [MULTI-PROTO] Register protocol untuk pengirim ini (per srcAddress)
this.protocolRegistry.set(packet.header.srcAddress, proto);
```

Hasilnya disimpan di `protocolRegistry` per `srcAddress`. Saat mengirim balik ke alamat itu, driver memakai protokol yang sama (default: JSON).

Fragmentasi & Reassembly

- **Fragmentasi:** payload dipecah menjadi potongan 32KB (`packetSize = 32768`). Setiap potongan dikirim sebagai paket MQTT terpisah dengan `packetCount/packetIndex` yang diisi.
- **Reassembly:** penerima mengumpulkan chunk dalam sesi per `src:port->dst:port`. Sesi dianggap lengkap saat jumlah chunk unik = `packetCount`, lalu digabung berurutan (`Buffer.concat` untuk Buffer, `join("")` untuk string).
- **TTL 30 detik:** sesi yang tidak lengkap dalam 30 detik dibuang (`cleanupExpiredPackets`, dijalankan tiap 60 detik).



 Streaming firmware OTA via MQTTN Binary v1.1 Sumber: wiki/diagram/mqtnl-binary-ota-1.mmd

Jalur kirim (TX) — `SimpleMQTNLDriver.send()`

1. Tentukan protokol: `protocolRegistry.get(address) || activeProtocol` (default JSON).
2. `useRaw = (protocol.getName() === "Binary")` — jika Binary, **lewati enkripsi** (`securedPayload = payload`), supaya panjang byte pas untuk OTA.
3. Jika JSON, enkripsi payload via `SecurityAgent.securePacketOut()` (RSA handshake → ChaCha20-Poly1305).
4. **Fragmentasi:** pecah `securedPayload` menjadi chunk 32KB (`this.packetSize`).
5. Untuk tiap chunk `i`: buat objek packet (`packetCount = chunks.length`, `packetIndex = i`, `dataSize = securedPayload.length`), `protocol.pack(packet) → Buffer`, lalu `publish` ke topic `${prefix}${address}` (atau loopback lokal jika alamatnya node ini).

Jalur terima (RX) — `SimpleMQTNLDriver.handleIncomingMessage()`

1. Deteksi protokol dari **magic byte pertama** payload MQTT.
2. `proto.unpack(message) → objek packet`. Simpan protokol di `protocolRegistry` per `srcAddress`.
3. **Filter paket:** buang paket yang `dstAddress` bukan alamat lokal (kecuali `"*"`).
4. **Reassembly:** simpan `packet.payload` di sesi `src:port->dst:port`. Jika belum lengkap → tunggu chunk berikutnya.
5. Jika lengkap: gabung chunk berurutan → **decrypt** (Binary: `securePacketInRaw` + fallback plain; JSON: `securePacketIn`).
6. **Dispatch:** kirim `{src, port, localPort, data, isBinary, ts}` ke handler port tujuan (atau layani PING/broadcast).

Snippet (level kode)

Semua snippet di bawah **VERIFIED** — disalin langsung dari sumber.

JSON v1.0 — pack/unpack (MQTNLProtocolJSON.ts)

```
export class MQTNLProtocolJSON implements IMQTNLProtocol {
  getName(): string { return "JSON"; }
  getMagicChars(): number[] { return [0x5B]; } // '['
  getTopicPrefix(): string { return "mqtnl@1.0/"; }

  pack(packet: any): string {
    return JSON.stringify([
      packet.header.srcAddress, packet.header.srcPort,
      packet.header.dstAddress, packet.header.dstPort,
      packet.header.packetCount, packet.header.packetIndex,
      packet.header.dataSize, packet.header.packetHeaderFlag,
      packet.header.forwarded, packet.payload,
    ]);
  }

  unpack(data: Buffer | string): any {
    const str = Buffer.isBuffer(data) ? data.toString("utf8") : data;
    const packed = JSON.parse(str);
    return {
      header: {
        srcAddress: packed[0], srcPort: packed[1],
        dstAddress: packed[2], dstPort: packed[3],
        packetCount: packed[4], packetIndex: packed[5],
        dataSize: packed[6], packetHeaderFlag: packed[7],
        forwarded: packed[8],
      },
      payload: packed[9],
    };
  }
}
```

Binary v1.1 — pack/unpack (MQTNLProtocolBinary.ts)

```
export class MQTNLProtocolBinary implements IMQTNLProtocol {
  getName(): string { return "Binary"; }
  getMagicChars(): number[] { return [0x42]; } // 'B'
  getTopicPrefix(): string { return "mqtnl@1.1/"; }

  pack(packet: any): Buffer {
    const srcAddr = Buffer.from(packet.header.srcAddress || "", "utf8");
    const dstAddr = Buffer.from(packet.header.dstAddress || "", "utf8");
    const payload = Buffer.isBuffer(packet.payload)
      ? packet.payload
      : Buffer.from(packet.payload || "", "utf8");

    const headerSize = 2 + 1 + srcAddr.length + 2 + 1 + dstAddr.length
      + 2 + 2 + 2 + 4 + 1 + 1;
    const buf = Buffer.alloc(headerSize + payload.length);
    let offset = 0;
    buf.writeUInt8(0x42, offset++); // Magic 'B'
    buf.writeUInt8(0x01, offset++); // Proto Ver 1
    buf.writeUInt8(srcAddr.length, offset++);
    srcAddr.copy(buf, offset); offset += srcAddr.length;
    buf.writeUInt16LE(packet.header.srcPort, offset); offset += 2;
    buf.writeUInt8(dstAddr.length, offset++);
    dstAddr.copy(buf, offset); offset += dstAddr.length;
    buf.writeUInt16LE(packet.header.dstPort, offset); offset += 2;
    buf.writeUInt16LE(packet.header.packetCount, offset); offset += 2;
    buf.writeUInt16LE(packet.header.packetIndex, offset); offset += 2;
    buf.writeUInt32LE(packet.header.dataSize, offset); offset += 4;
    buf.writeUInt8(packet.header.packetHeaderFlag, offset++);
    buf.writeUInt8(packet.header.forwarded, offset++);
    payload.copy(buf, offset);
    return buf;
  }
}
```

```

}

unpack(data: Buffer | string): any {
  const buffer = Buffer.isBuffer(data) ? data : Buffer.from(data, "utf8");
  let offset = 0;
  if (buffer.readUInt8(offset++) !== 0x42) throw new Error("Invalid Magic Byte");
  if (buffer.readUInt8(offset++) !== 0x01) throw new Error("Invalid Protocol Version");

  const srcAddrLen = buffer.readUInt8(offset++);
  const srcAddress = buffer.subarray(offset, offset + srcAddrLen).toString("utf8");
  offset += srcAddrLen;
  const srcPort = buffer.readUInt16LE(offset);
  offset += 2;

  const dstAddrLen = buffer.readUInt8(offset++);
  const dstAddress = buffer.subarray(offset, offset + dstAddrLen).toString("utf8");
  offset += dstAddrLen;
  const dstPort = buffer.readUInt16LE(offset);
  offset += 2;

  const packetCount = buffer.readUInt16LE(offset);
  offset += 2;
  const packetIndex = buffer.readUInt16LE(offset);
  offset += 2;
  const dataSize = buffer.readUInt32LE(offset);
  offset += 4;
  const packetHeaderFlag = buffer.readUInt8(offset++);
  const forwarded = buffer.readUInt8(offset++);
  const payload = buffer.subarray(offset);

  return {
    header: { srcAddress, srcPort, dstAddress, dstPort,
              packetCount, packetIndex, dataSize,
              packetHeaderFlag, forwarded },
    payload: payload,
  };
}
}

```

Fragmentasi — loop 32KB (SimpleMQTNLDriver.send())

```

// --- FRAGMENTATION LAYER ---
const chunks: (string | Buffer)[] = [];
if (securedPayload.length === 0) {
  chunks.push(useRaw ? Buffer.alloc(0) : "");
} else {
  for (let i = 0; i < securedPayload.length; i += this.packetSize) {
    if (typeof securedPayload === "string") {
      chunks.push(securedPayload.substring(i, i + this.packetSize));
    } else {
      chunks.push(securedPayload.subarray(i, i + this.packetSize));
    }
  }
}

const packetCount = chunks.length;
const topic = `${prefix}${address}`;

for (let i = 0; i < packetCount; i++) {
  const packet = {
    header: {
      srcAddress: this.localAddress,
      srcPort: srcPort,
      dstAddress: address,
      dstPort: port,
      packetCount: packetCount, // jumlah total chunk
      packetIndex: i,           // chunk ke-i (0-based)
      dataSize: securedPayload.length, // ukuran TOTAL payload
      packetHeaderFlag: flag,
      forwarded: 0,
    },
    payload: chunks[i],
  };
}

```



```

    };
    const payloadBuffer = protocol.pack(packet);
    this.txBytes += payloadBuffer.length;
    // publish ke topic (atau loopback lokal jika alamat = node ini)
  }

```

Reassembly — gabung chunk (SimpleMQTNLDriver.handleIncomingMessage())

```

// --- REASSEMBLY LAYER ---
const key = `${packet.header.srcAddress}:${packet.header.srcPort}->${packet.header.dstAddress}:${packet.header.dstPort}`;

if (!this.receivedPackets.has(key)) {
  this.receivedPackets.set(key, {
    total: packet.header.packetCount,
    received: new Map(),
    timestamp: Date.now(),
  });
}

const session = this.receivedPackets.get(key)!;
session.received.set(packet.header.packetIndex, packet.payload);
session.timestamp = Date.now();

if (session.received.size < session.total) {
  return; // belum lengkap — tunggu chunk berikutnya
}

// Semua chunk tiba!
this.receivedPackets.delete(key);

const entries = Array.from(session.received.entries()).sort((a, b) => a[0] - b[0]);
const assembledPayload = Buffer.isBuffer(entries[0][1])
  ? Buffer.concat(entries.map(e => e[1] as Buffer))
  : entries.map(e => e[1] as string).join("");

```

TTL 30 detik — pembersihan sesi kadaluarsa

```

private cleanupExpiredPackets() {
  const now = Date.now();
  const ttl = 30000; // 30 detik untuk chunk yang ditinggalkan
  for (const [key, session] of this.receivedPackets.entries()) {
    if (now - session.timestamp > ttl) {
      this.logger.warn(`[REASSEMBLY] Dropped incomplete packet from ${key} (TTL Expired)`);
      this.receivedPackets.delete(key);
    }
  }
}

```

Latihan / Praktik

1. Baca `src/common/protocols/MQTNLProtocolJSON.ts` — pahami struktur packet JSON (array 10 elemen).
2. Baca `src/common/protocols/MQTNLProtocolBinary.ts` — hitung ukuran header binary untuk `srcAddress="tsix"` dan `dstAddress="esp32S3"` (jawaban: 29 byte).
3. Baca `src/kernel/devices/SimpleMQTNLDriver.ts` — telusuri jalur TX (send) dan RX (handleIncomingMessage), lalu cari `packetSize` dan `ttl`.
4. Baca `wiki/mqtnl_binary_ota.md` — bagaimana Binary dipakai untuk OTA byte-exact.
5. Jelaskan mengapa Binary v1.1 sengaja tanpa enkripsi.
6. Eksperimen: kirim payload > 32KB dari aplikasi, amati log `[FRAGMENTATION]` dan `[REASSEMBLY]`.

Referensi

- `wiki/mqtnl-ota.md`, `wiki/mqtnl_binary_ota.md` — OTA via MQTNL

- `wiki/course/00-overview.md §7`
 - `src/common/protocols/IMQTNLProtocol.ts, MQTNLProtocolJSON.ts, MQTNLProtocolBinary.ts`
-

Modul 16 — selesai. Bagian VI tuntas. Lanjut ke [Modul 17 — PixelSpace Protocol](#).

RFC-TSIX-EDU-002 | Modul ketujuh belas kurikulum TSIX. Memahami kontrak data GUI: alur data Worker→Kernel→DOME→Browser, GUIRegistry sebagai otoritas kepemilikan window, dan sanksi keamanan.

PixelSpace adalah **konstitusi GUI TSIX**. Kontraknya (`GUITypes.ts`) tidak boleh diubah sembarangan — semua lapisan di atasnya (DOME, Emerald, Cashew, Asteracea) bergantung pada bentuk data ini.

Tujuan Pembelajaran

- ☐ Menjelaskan alur data mounting UI dan event
- ☐ Menjelaskan peran GUIRegistry (auth wid↔pid)
- ☐ Menyebutkan GUIAction utama
- ☐ Menjelaskan sanksi keamanan (SIGKILL / SIGSEGV)
- ☐ Menjelaskan mengapa `payload.pid` di-override kernel

Konsep Inti

Alur data

```
Worker (app) → GUI_REQ (61) → Kernel (GUIRegistry auth pid↔wid)
→ DOME daemon (WS broadcast) → Browser (DOM)
Browser → event → DOME → Kernel (SEND_MSG) → Worker (callback)
```

Alur mounting UI

```
Worker App → GUI_REQ (MOUNT_NODE) → Kernel → gui_request event → DOME → WS → Browser (createElement)
```

Alur event (klik/input)

```
Browser → WS → DOME → SEND_MSG ke pid → Kernel → ipc_message event → Worker → callback → updateProps
```

GUIAction (operasi GUI)

Action	Kegunaan
CREATE_WINDOW / DESTROY_WINDOW	Bikin / hancurkan jendela
MOUNT_NODE / UNMOUNT_NODE	Pasang / lepas elemen
UPDATE_PROPS	Ubah properti elemen
MINIMIZE_WINDOW / RESTORE_WINDOW	Sembunyikan / kembalikan
MAXIMIZE_WINDOW / UNMAXIMIZE_WINDOW	Layar penuh / normal

GUIRegistry (kernel)

Otoritas **tunggal** kepemilikan window:

- `wid → pid` (primary map) + `pid → Set<wid>` (reverse map, lookup cepat saat exit)
- Z-index auto-increment (mulai 100)
- `registerDaemon(pid)` — hanya "gued" yang boleh menerima forward `GUI_REQ`
- Cleanup otomatis saat proses mati (`destroyAllForPid`)

Keamanan

Pelanggaran	Sanksi
Payload format rusak	SIGKILL — proses ditembak mati
Akses window milik PID lain	SIGSEGV — segmentation fault

[!IMPORTANT] `payload.pid` **selalu di-override kernel**. Jangan pernah percaya `pid` yang dikirim aplikasi — kernel menetapkan identitas asli dari konteks proses.

Kode Sumber

File	Peran
<code>src/common/GUITypes.ts</code>	Kontrak data (IDOMNode, IGUIPayload, GUIAction, IBrowserEvent, IGUIEventIPC)
<code>src/kernel/GUIRegistry.ts</code>	Otoritas window + auth
<code>src/kernel/Syscalls.ts</code>	Handler <code>GUI_REQ</code> + security
<code>src/mirror/root/ps-sample1.ts</code>	Praktik raw protocol

Snippet (level kode)

GUIRegistry.createWindow

```
public createWindow(wid: string, pid: number, title: string = "Untitled"): IWindowEntry {
  if (this.windows.has(wid)) {
    throw new Error(`GUIRegistry: Window '${wid}' already exists.`);
  }
  const entry: IWindowEntry = { wid, pid, title,
    zIndex: this.nextZIndex++, focused: true, createdAt: Date.now() };
  this.windows.set(wid, entry);
  // Update reverse map pid → Set<wid>
  if (!this.pidToWids.has(pid)) this.pidToWids.set(pid, new Set());
  this.pidToWids.get(pid)!.add(wid);
  // Defocus window lain
  return entry;
}
```

Latihan / Praktik

- Baca `src/common/GUITypes.ts` — pahami seluruh interface "konstitusi".
- Baca `src/mirror/root/ps-sample1.ts` — praktik raw protocol tanpa toolkit.
- Jalankan app GUI lalu `ps` — cari PID `gued/DOME`. Baca `wiki/identity_guid_ipc_walkthrough.md` untuk alur identitas.
- Modifikasi window milik PID lain lewat `GUI_REQ` — amati `SIGSEGV`.

Referensi

- `wiki/PIXELSPACE_DEVELOPER_GUIDE.md` §1-2, 10
- `wiki/course/00-overview.md` §10
- `src/common/GUITypes.ts`, `src/kernel/GUIRegistry.ts`, `src/kernel/Syscalls.ts`

Modul 17 — selesai. Lanjut ke [Modul 18 — DOME Engine](#).

RFC-TSIX-EDU-002 | Modul kedelapan belas kurikulum TSIX. Memahami DOME: relay WebSocket + primitive DOM producer + kompositor (titlebar, drag, resize, focus, replay).

DOME adalah display server TSIX — analog X11/Wayland, tapi dirender sebagai **DOM di browser host** via WebSocket :8080. Server-nya monolitik (satu daemon, relay + kompositor) karena pertimbangan latency drag/resize (kompositor terpisah = overhead ~2-4ms). Client browser justru modular: `dome.ts` menyajikan modul statis `dome-client-*.js` via `/dome/*.js`.

Tujuan Pembelajaran

- ☐ Menjelaskan peran DOME sebagai relay + kompositor
- ☐ Menjelaskan spesifikasi & fitur DOME
- ☐ Menjelaskan tradeoff desain monolitik vs kompositor terpisah
- ☐ Menjelaskan peran `windowStates` dan replay
- ☐ Menjelaskan enforcement flag `maximizable` di semua jalur maximize
- ☐ Menjelaskan proteksi navigasi, per-app traffic, modul DDC, dan boot readiness
- ☐ Menjelaskan hubungan DOME ↔ Kernel ↔ Browser

Konsep Inti

Spesifikasi

Aspek	Deskripsi
Lokasi	<code>src/mirror/opt/dome/dome.ts</code> (server) + <code>dome-client.html</code> tipis + modul <code>dome-client-*.js</code> (browser)
Port	WebSocket :8080
Peran	Display server (Ring 4 daemon)
Model	Relay WS + primitive DOM producer + kompositor

Fitur

- Window registry, Z-index, relay event
- **State Replay** (`windowStates`): semua `MOUNT_NODE` & `UPDATE_PROPS` disimpan, dikirim ulang saat browser reconnect (F5)
- **State Pruning** (`pruneWindowState`): saat node di-unmount, DOME membersihkan state — mencegah orphan state leak ke replay
- **Orphan Discard** (browser-side): `MOUNT_NODE` dengan parent yang tidak ditemukan di-discard — mencegah residu dialog/modal setelah F5
- **Overlay Layer Search**: `findElementById` mencari di 3 level — `win.el` → `__global_start_menu__` → `__tsix_overlay_layer__`
- **Client modular**: logika browser dipisah ke modul statis `dome-client-core.js`, `-term.js`, `-codemirror.js`, `-chart.js`, `-dom.js`, `-windows.js`, `-ui.js`, `-ddc.js`, `-res.js`; `dome.ts` membacanya dari VFS `/opt/dome/*.js` saat startup dan menyajikannya via `/dome/*.js`
- **Flag `maximizable`**: dihormati di semua jalur maximize — double-click titlebar, context menu "Maximize", guard server `maximize_window`, dan guard syscall `MAXIMIZE_WINDOW`
- **Maximize saat minimized**: window ditampilkan dulu di posisi terakhir sebelum rect diukur; restore memakai `_origRect` jika `_savedRect` nol
- **Proteksi navigasi**: refresh (F5/Ctrl+R/Cmd+R) dan back/forward butuh konfirmasi (`protectNavigation()` + `beforeunload`)

- **Per-app traffic accounting:** `TRAFFIC_QUERY` meng-exclude traffic app penanya sendiri dan melaporkan `selfTxBytes / selfTxPkts`
- **Modul DDC:** `dome-client-ddc.js` menghost aplikasi Native-JS (Fabric.js / Three.js via CDN) di Shadow DOM; DOME merelay `DDC_MSG / DDC_RESIZE / DDC_STOP`; `destroyDDCByWid(wid)` dipanggil saat window ditutup
- **Boot readiness:** DOME menulis `/var/run/dome.ready`; `rc.local` mem-poll penanda (timeout 10s, interval 200ms) sebelum start Asteracea — menggantikan `sleep 1s`
- **`ensureListener()`** : memasang listener sekali per elemen per event — menggantikan pola `cloneNode` yang menghapus listener lama

Tradeoff desain: kenapa monolitik?

	Kompositor terpisah (X11-style)	DOME monolitik
Overhead	~2-4ms per operasi drag/resize	langsung, tanpa IPC ekstra
Keuntungan	modularitas	latency rendah

Karena target utama adalah **interaksi drag/resize/focus** yang sensitif latency, DOME memilih monolitik. Ada catatan desain & rencana refactor di wiki.

Alur Replay

```
Browser F5 → WebSocket reconnect
→ DOME kirim CREATE_WINDOW (semua window aktif)
→ DOME kirim semua stored states (MOUNT_NODE + UPDATE_PROPS)
→ Jika window sedang maximized, DOME kirim MAXIMIZE_WINDOW ulang
→ Browser build ulang UI + terima event dari user
```

Snippet (level kode)

State pruning (ringkas)

```
const allIds = new Set<string>([targetId]);
for (const state of states) {
  if (state?.type === "MOUNT_NODE" && state.node) {
    const nodeIds = new Set<string>();
    collectNodeIds(state.node, nodeIds);
    if (nodeIds.has(targetId)) for (const id of nodeIds) allIds.add(id);
  }
}
// Hapus MOUNT_NODE (parent atau node di dalam tree) & UPDATE_PROPS (target exact match)
```

Browser orphan discard

```
if (parent) {
  parent.appendChild(domEl);
} else {
  console.log('[DEBUG] Orphan discarded:', node.id, targetId); // DISCARD
}
```

Maximize guard — semua jalur hormati flag `maximizable`

```
// dome-client-windows.js — double-click titlebar & handleMaximizeWindow
if (!w._isMaximized && w.maximizable === false) return; // titlebar dbl-click
if (win.maximizable === false) return; // handleMaximizeWindow
```

```
// dome.ts – guard otoritatif sisi server (event browser + syscall)
if (event.eventType === "maximize_window") {
  if (entry.maximizable === false) return;
}
// syscall GUIAction.MAXIMIZE_WINDOW
if (windows.get(wid)?.maximizable === false) break;
```

Restore aman — fallback saat rect tersimpan nol

```
// dome-client-windows.js – handleRestoreWindow
let saved = win._savedRect;
if (saved && (saved.width <= 0 || saved.height <= 0)) {
  saved = win.el._origRect || null; // jangan restore ke (0,0,0,0)
}
```

ensureListener — listener tidak dobel & tidak hilang

```
// dome-client-dom.js – dipakai buildDOM() & handleUpdateProps()
function ensureListener(el, event, listener) {
  if (!el.__tsixL) el.__tsixL = Object.create(null);
  if (!el.__tsixL[event]) {
    el.addEventListener(event, listener);
    el.__tsixL[event] = true;
  }
}
```

Per-app traffic — observer tidak menghitung dirinya sendiri

```
// dome.ts – TRAFFIC_QUERY meng-exclude traffic app penanya
const self = appTraffic.get(payload.pid) || { txBytes: 0, txPkts: 0 };
const stats = {
  ...wsTraffic,
  txBytes: Math.max(0, wsTraffic.txBytes - self.txBytes),
  txPkts: Math.max(0, wsTraffic.txPkts - self.txPkts),
  selfTxBytes: self.txBytes, // traffic milik app penanya
  selfTxPkts: self.txPkts,
};
```

DDC cleanup — hentikan RAF loop saat window ditutup

```
// dome-client-windows.js – handleDestroyWindow (safety net)
if (typeof TSIX.destroyDDCByWid === "function") {
  TSIX.destroyDDCByWid(wid); // stop semua runtime DDC window ini
}
```

Latihan / Praktik

1. Buka `http://localhost:8080` — amati DOME client di browser.
2. Tekan F5 saat beberapa window terbuka — amati prompt konfirmasi navigasi dan state replay.
3. Baca `src/mirror/opt/dome/dome.ts` — temukan `windowStates`, `pruneWindowState`, `guard maximizable`, `relay DDC_*`, dan `TRAFFIC_QUERY`.
4. Baca `src/mirror/opt/dome/dome-client-windows.js` — cari `handleMaximizeWindow` (`maximize-while-minimized`) dan `handleRestoreWindow`.
5. Baca `src/mirror/opt/dome/dome-client-core.js` — cari `protectNavigation()`.
6. Baca `src/mirror/opt/dome/dome-client-ddc.js` — cari `initDDC` dan `destroyDDCByWid`.
7. Baca `src/mirror/opt/dome/dome-client-dom.js` — cari `ensureListener`.
8. Baca `src/mirror/etc/rc.local.ts` — cari polling `/var/run/dome.ready` sebelum start Asteracea.

Referensi

- [wiki/PIXELSPACE_DEVELOPER_GUIDE.md](#) §2, §11
 - [wiki/course/00-overview.md](#) §10
 - [wiki/changelogs/dome.md](#) — riwayat perubahan DOME
 - `src/mirror/opt/dome/dome.ts` + `src/mirror/opt/dome/dome-client-*.js`
 - `src/mirror/etc/rc.local.ts` — polling `/var/run/dome.ready`
-

Modul 18 — selesai. Lanjut ke [Modul 19 — Emerald Widget Toolkit](#).

RFC-TSIX-EDU-002 | Modul kesembilan belas kurikulum TSIX. Memahami layer toolkit GUI di atas protocol stabil: Screen wrapper, factory functions, dan connected widgets self-rendering.

Emerald (`@tsix/emerald`) adalah GTK/Qt-nya TSIX. Semua UI dibangun sebagai **Virtual DOM Tree** (`IDOMNode`) dan dikirim via syscall — aplikasi **tidak punya akses** ke `document`, `window`, atau API browser apa pun.

Tujuan Pembelajaran

- ☐ Menjelaskan posisi Emerald dalam tumpukan GUI
- ☐ Menjelaskan factory functions (`div`, `button`, `input`, ...)
- ☐ Menjelaskan Screen wrapper & pola mount → bind → loop
- ☐ Menjelaskan connected widgets (self-rendering), termasuk `ConnectedDataGrid`
- ☐ Menjelaskan `ensureListener()` — listener props persisten lintas `UPDATE_PROPS` (bug `cloneNode` diperbaiki)

Konsep Inti

Posisi dalam arsitektur

BROWSER → DOME (display server) → KERNEL (auth) → EMERALD (kamu di sini)

Struktur dasar app

```
import { Program } from "@tsix/Application";
import { Screen, div, h1, paragraph } from "@tsix/emerald";

export const main = Program(async (args: string[]) => {
  // 1. Buat Screen (jendela)
  // 2. Bangun Virtual DOM tree dengan factory functions
  // 3. Mount, bind event, loop
});
```

Factory functions

Factory menghasilkan `IDOMNode`. Daftar lengkap ada di `src/mirror/lib/emerald.ts`:

- Dasar: `text()`, `div()`, `span()`, `h1()/h2()/h3()`, `paragraph()`, `button()`, `input()`, `textarea()`, `selectBox()`, `image()`
- IoT: `lineChart()`, `radialGauge()`, `verticalGauge()`, `sevenSegment()`, `indicatorLamp()`, `toggleSwitch()`, `sensorCard()`, `relayCard()`, `badge()`, `taskbarButton()`
- Tabel: `dataGrid()` (statis), `slider()`

`alert()`, `confirm()`, `question()`, `openFileDialog()`, `saveFileDialog()` **bukan factory** — itu method pada `Window/Screen` yang menampilkan modal dialog overlay, bukan menghasilkan `IDOMNode`.

Pattern dasar: Mount → Bind → Loop

```
const screen = new Screen({ title: "App", width: 400, height: 300 });
await screen.mount(
  div({ id: "root", style: { padding: "16px" } },
    h1({ text: "Hello" }),
    button({ id: "btn", text: "Klik" }),
  ),
);
// bind event → updateProps (di-batch & auto-flush) → loop
```

```
await screen.on("btn", "click", async () => {
  await screen.update("btn", { text: "✅ Diklik!" });
});
```

Connected widgets (self-rendering)

Widget "connected" me-render dirinya sendiri dan memperbarui layar secara mandiri — pola yang lebih tinggi dari factory biasa. Pola umum: `build()` → `mount(screen)` → `setData()/setValue()`, memakai **targeted update** (bukan `setContent` penuh).

Kelas `Connected*` di `emerald.ts`:

- `ConnectedLineChart`, `ConnectedRadialGauge`, `ConnectedSevenSegment`, `ConnectedIndicatorLamp`
- `ConnectedVerticalGauge`, `ConnectedToggle`, `ConnectedSensorCard`, `ConnectedRelayCard`
- `ConnectedDataGrid` — tabel data interaktif: sort asc/desc, resize kolom (drag native), seleksi row berbasis kunci stabil, `appendData()` inkremental & opsi `maxRows`. Satu scroll container (header `th sticky`) — scroll horizontal & vertikal otomatis sinkron.

[!IMPORTANT] **Listener props & ensureListener() (bug `cloneNode` sudah diperbaiki)**. Empat listener props — `onClickId`, `onContextMenuId`, `onInputId`, `onKeyDownId` — dipasang oleh DOME client lewat helper `ensureListener()` di `src/mirror/opt/dome/dome-client-dom.js` (dipakai di `buildDOM()` dan `handleUpdateProps()`), **sekali per elemen per event type** (dilacak via `el.__tsixL`). Dulu `handleUpdateProps` meng-clone node untuk "membersihkan listener lama" — `cloneNode` tidak menyalin listener, jadi jika satu batch `UPDATE_PROPS` membawa beberapa listener props sekaligus (mis. `onInputId` + `onKeyDownId` di field password), listener yang baru dipasang bisa hilang. Dengan `ensureListener`, listener **persisten lintas `UPDATE_PROPS`** — tidak hilang dan tidak dobel. Best practice tetap berlaku: set listener props (`onClickId`, `onInputId`, dst) **saat mount-time** (di `build()/mount()`), lalu daftarkan callback via `screen.on()` / `win.bindHandler()`.

Kode Sumber

File	Peran
<code>src/mirror/lib/emerald.ts</code>	Widget toolkit @tsix/emerald
<code>src/mirror/lib/theme.ts</code>	Tema
<code>src/mirror/lib/cashew.ts</code>	Layer di atas Emerald (Modul 20)
<code>src/mirror/opt/dome/dome-client-dom.js</code>	DOME client DOM engine — <code>buildDOM()</code> , <code>handleUpdateProps()</code> , <code>ensureListener()</code>
<code>src/mirror/root/ps-sample2.ts</code> , <code>ps-sample3.ts</code>	Praktik

Latihan / Praktik

1. Baca `wiki/emerald-in-a-nutshell.md` — kerjakan Hello World sampai studi kasus lengkap.
2. Bangun form dengan input + button; bind event klik yang mengubah teks.
3. Gunakan `alert()/confirm()` — amati bagaimana ia tampil sebagai window overlay.
4. Baca `src/mirror/lib/emerald.ts` — cari implementasi `Screen` dan `setContent`.
5. Bangun `ConnectedDataGrid` dengan sort & resize kolom; pakai `appendData()` untuk data real-time dan `maxRows` untuk membatasi baris.

Referensi

- `wiki/emerald-in-a-nutshell.md`, `wiki/cashew-in-a-nutshell.md`
- `wiki/PIXELSPACE_DEVELOPER_GUIDE.md` §5-7
- `src/mirror/lib/emerald.ts`, `src/mirror/lib/theme.ts`

Modul 19 — selesai. Lanjut ke [Modul 20 — Cashew Component Framework](#).

RFC-TSIX-EDU-002 | Modul kedua puluh kurikulum TSIX. Memahami layer OOP/Delphi-style di atas Emerald: `TForm`, `TButton`, `TEdit`, auto-bind lifecycle, dan widget kompleks.

Cashew (`@tsix/cashew`) mengadopsi pola komponen ala **Delphi / Turbo Pascal** — komponen adalah **class ber-state** dengan properti & event, bukan fungsi bersarang. Ini analog VCL di atas GTK.

Tujuan Pembelajaran

- Menjelaskan filosofi OOP/Delphi-style Cashew
- Menjelaskan lifecycle `TForm.run()` (auto-bind)
- Menjelaskan komponen dasar: `TForm`, `TPanel`, `TLabel`, `TButton`, `TEdit`
- Menjelaskan kenapa `onClickId`/`onInputId` diset saat build (mount-time)
- Menyebutkan widget kompleks (`TChart`, `TSevenSegment`, `TSensorCard`, dll)

Konsep Inti

Filosofi

Di Emerald kamu menulis UI dengan fungsi bersarang (`div([button(...)])`). Di Cashew, kamu membuat **objek class**:

```
const form = new TForm("My App", 400, 300);
const btn = new TButton("btn-click");
btn.caption = "Klik";
btn.onClick = () => { count++; lblCounter.caption = "Count: " + count; };
form.add(btn);
await form.run();
```

Lifecycle `TForm.run()`

`run()` punya **auto-bind lifecycle**: `bindEventHandler` + `refresh` per komponen. Setelah `run()`, perubahan properti (`caption`, `text`, dll) otomatis sinkron ke layar.

Urutan di `TForm.run()` (`src/mirror/lib/cashew.ts`):

1. **Theme** — muat tema aktif sebelum build agar warna CSS variable benar.
2. **Build & mount** — bangun pohon `IDOMNode` via `this.build()` lalu `_screen.mount(...)`.
3. **Auto-bind** — DFS traversal semua children, panggil `comp.bindEventHandler(screen)` (registrasi event).
4. **Auto-refresh** — DFS traversal semua children, panggil `await comp.refresh(screen)` (render data dinamis, mis. `TListBox`, `TDataGrid`).
5. **Setup** — panggil `await onSetup(screen)` bila di-set (binding tambahan).
6. **Event loop** — `_screen.loopUntilClose()` menunggu sampai window ditutup.

```
sequenceDiagram
    participant App as Aplikasi (main)
    participant Form as TForm.run()
    participant Comp as Komponen (TButton, TEdit, TDataGrid, ...)
    participant Screen as Screen (Emerald)

    App->>Form: await run()
    Form->>Form: loadCurrent() - theme
    Form->>Screen: mount(build())
    loop auto-bind (DFS)
        Form->>Comp: bindEventHandler(screen)
        Comp->>Screen: screen.on(id, "click"/"input", cb)
    end
```

```

loop auto-refresh (DFS)
  Form->>Comp: await refresh(screen)
  Comp->>Screen: setContent / setData
end
Form->>App: await onSetup(screen)
Form->>Screen: loopUntilClose()
loop event loop
  Screen->>Comp: event (click / input / timer)
  Comp->>Screen: screen.update(id, props)
end

```

Hirarki Komponen

Semua komponen berpangkal dari base class `TComponent` di `src/mirror/lib/cashew.ts`:

```

graph TD
  TC[TComponent] --> TForm
  TC --> TPanel
  TC --> TLabel
  TC --> TButton
  TC --> TEdit
  TC --> TMemo
  TC --> TCheckBox
  TC --> TListBox
  TC --> TRadioButton
  TC --> TComboBox
  TC --> TStatusBar
  TC --> TLineChart
  TC --> TRadialGauge
  TC --> TSevenSegment
  TC --> TIndicatorLamp
  TC --> TToggleSwitch
  TC --> TVerticalGauge
  TC --> TSensorCard
  TC --> TRelayCard
  TC --> TSlider
  TC --> TTimer
  TC --> TChart
  TC --> TDataGrid

```

[!NOTE] **Bukan class.** `TDialogs` adalah utility **statis** (bukan `TComponent`). `HStack`, `VStack`, `Spacer`, `TScrollBar`, `TFlowPanel`, `TGridPanel`, `TSplitHorizontal`, `TSplitVertical`, `TGroupBox` adalah **fungsi** yang mengembalikan `TComponent` — layout cepat tanpa subclass baru. Konstanta `align (alTop, alClient, dst.)` tersedia untuk `style.position`.

Kenapa event diset saat build (mount-time)

`onClickId/onInputId` ditulis langsung ke `props` di constructor atau `build()` (contoh: `TButton` men-set `this.props.onClickId = id`). Saat DOME men-mount node, listener terpasang sekali. Menambah listener lewat `app.on()` setelah mount berisiko bentrok dengan pola `cloneNode` DOME yang **tidak menyalin listener** — lihat Modul 19.

Komponen dasar

Komponen	Fungsi
<code>TForm</code>	Window utama (<code>add()</code> , <code>alert()</code> , <code>confirm()</code> , <code>screen</code> , <code>onSetup</code> ; opsi <code>maximizable/resizable/fullscreen/frameless</code>)
<code>TPanel</code>	Container (dengan/ tanpa style)
<code>TLabel</code>	Teks (<code>caption</code>)
<code>TButton</code>	Tombol (<code>onClick</code> , <code>caption</code> , <code>enabled</code>)

Komponen	Fungsi
TEdit	Input teks (<code>onInput</code> , <code>text</code> , <code>placeholder</code>)
TMemo , TCheckBox , TListBox , TStatusBar , TRadioButton , TComboBox	Lainnya
TScrollBox , TFlowPanel , TGridPanel , TSplitHorizontal/Vertical , TGroupBox	Layout
HStack , VStack , Spacer	Layout cepat

Widget kompleks & IoT

Komponen	Fungsi
TDialogs	Utility dialog statis: <code>alert</code> , <code>confirm</code> , <code>input</code> , <code>openFile</code> , <code>saveFile</code>
TTimer	Interval timer ala Delphi; auto-cleanup saat form ditutup (<code>onTimer</code>)
TChart	Chart real-time multi-series (Lightweight Charts via IPC DOME)
TLineChart	Line chart SVG (spline, fill, scroll animation)
TRadialGauge	Gauge melingkar ala speedometer
TSevenSegment	Display LED 7-segment (opsi <code>scale</code> , <code>height</code>)
TIndicatorLamp	Lampu indikator ON/OFF dengan glow
TToggleSwitch	Switch ON/OFF yang bisa diklik
TVerticalGauge	Tabung kaca vertikal; value text dual-layer masking
TSensorCard	Kartu sensor IoT dengan progress bar
TRelayCard	Kartu relay ON/OFF
TSlider	Slider range horizontal

TDataGrid — satu scroll container

TDataGrid membungkus `ConnectedDataGrid` Emerald (`src/mirror/lib/emerald.ts`). Perubahan 2026-08-03 membuat grid memakai **satu scroll container**:

- **Satu container scroll** — header table + `<style>` + body table berada dalam satu `body-scroll` (`overflow: auto`). Header di-pin via **th sticky** (`position: sticky; top: 0; z-index: 2`), pola yang sama dengan `dataGrid()` statis.
- **Tanpa kompensasi manual** — wrapper `thead-scroll` , relay `scrollLeft` , `scrollbar-gutter` , dan `calc(100% - scrollbarWidth)` dihapus. Scroll horizontal & vertikal otomatis sinkron; lebar header == body == container.
- **Resize kolom** — scope resize adalah wrapper grid `table.closest(".tsix-dgrid")` (bukan `table.parentElement`). Drag handle mengubah lebar `<col>` di **semua colgroup** (header + body) sekaligus.
- **col_resized** — saat drag selesai, browser mengirim `col_resized` dengan `targetId = id grid` (wrapper `.tsix-dgrid`) → `ConnectedDataGrid` menyimpan lebar dan re-apply di setiap render (sort/refresh/setData).

[!IMPORTANT] **Mount-time event.** Set `onClickId` / `onInputId` saat build (mount-time) — bukan di `app.on()` — untuk menghindari bug `cloneNode` (lihat Modul 19).

Alur / Cara Kerja

1. Buat objek `TForm` — dua bentuk: sequential `new TForm(title, w, h, ...)` atau object literal `new TForm({ title, ... })`.
2. Tambahkan komponen via `form.add(...)` — membentuk pohon parent-child.

3. Set properti (`caption`, `text`, `value`) dan event handler (`onClick`, `onInput`, `onChange`, `onTimer`).
4. Panggil `await form.run()` — build → mount → auto-bind → auto-refresh → `onSetup` → loop.
5. Ubah properti kapan saja; komponen yang sudah bind otomatis `screen.update(...)` ke layar.
6. Window ditutup → `loopUntilClose()` selesai; timer managed di-clean up otomatis.

Kode Sumber

- `src/mirror/lib/cashew.ts` — seluruh framework (TComponent, TForm, TDialogs, komponen dasar, layout, widget IoT, TDataGrid)
 - `src/mirror/lib/emerald.ts` — `ConnectedDataGrid` (render grid & state lebar kolom)
 - `src/mirror/opt/dome/dome-client-dom.js` — build DOM, `data-col-resize`, event `col_resized`
 - `wiki/cashew-in-a-nutshell.md`, `wiki/emerald-in-a-nutshell.md` — ringkasan
-

Snippet (level kode)

Quick start Cashew

```
import { Program } from "@tsix/Application";
import { TForm, TLabel, TButton, TStatusBar } from "@tsix/cashew";

export const main = Program(async () => {
  const form = new TForm("My App", 400, 300);
  let count = 0;

  const lblCounter = new TLabel("counter");
  lblCounter.caption = "Count: 0";
  form.add(lblCounter);

  const btnClick = new TButton("btn-click");
  btnClick.caption = "Klik";
  btnClick.onClick = () => { count++; lblCounter.caption = "Count: " + count; };
  form.add(btnClick);

  const status = new TStatusBar("status");
  status.text = "🟢 Siap";
  form.add(status);

  // Tidak perlu bind manual — TForm.run() memanggil bindEventHandler()
  // untuk setiap komponen secara otomatis (auto-bind lifecycle).
  await form.run();
});
```

TForm.run() — auto-bind lifecycle (dari cashew.ts)

```
async run(): Promise<void> {
  // 1. Theme — load dulu biar komponen pake warna yang benar
  try {
    const t = await ensureTheme();
    await t.loadCurrent();
    t.watch();
  } catch (_) { /* theme opsional — skip jika gagal */ }

  // 2. Build & mount ke Screen
  this._screen = new Screen(opts); // title, width, height, maximizable, ...
  await this._screen.mount(this.build());
  // (opsional) kirim WINDOW_THEME ke DOME untuk CSS variables

  // 3. Auto-bind: DFS — panggil bindEventHandler() untuk semua komponen
  const autoBind = (comp: TComponent) => {
    comp.bindEventHandler(this._screen);
    for (const child of comp.children) autoBind(child);
  };
  for (const child of this.children) autoBind(child);
}
```



```
// 4. Auto-refresh: DFS – panggil refresh() (misal TListBox, TDataGrid)
const autoRefresh = async (comp: TComponent) => {
  await comp.refresh(this._screen);
  for (const child of comp.children) await autoRefresh(child);
};
for (const child of this.children) await autoRefresh(child);

// 5. Setup custom – setelah bind & refresh
if (this.onSetup) await this.onSetup(this._screen);

// 6. Event loop sampai window ditutup
await this._screen.loopUntilClose();
}
```

TComponent — bindEventHandler & refresh (base no-op)

```
export class TComponent {
  // Daftarkan event handler ke Screen. Otomatis dipanggil oleh TForm.run().
  bindEventHandler(screen: Screen): void {
    // Base: no-op – subclass override untuk register event
  }

  // Update tampilan setelah mount. Otomatis dipanggil oleh TForm.run().
  async refresh(screen: Screen): Promise<void> {
    // Base: no-op – subclass override untuk render dinamis
  }

  build(): IDOMNode {
    return {
      id: this.id,
      tag: this.tag,
      props: { ...this.props, style: { ...this.style } },
      children: this._children.map((c) => c.build()),
    };
  }
}
```

Contoh widget — TButton & TEdit (dari cashew.ts)

```
// TButton – bind onClick saat auto-bind
// constructor: this.tag = "button"; this.props.onClickId = id;
export class TButton extends TComponent {
  public onClick: (() => void) | null = null;
  bindEventHandler(screen: Screen): void {
    this._screen = screen;
    if (this.onClick) screen.on(this.id, "click", this.onClick);
  }
}

// TEdit – bind onInput saat auto-bind
// constructor: this.tag = "input"; this.props.onInputId = id;
export class TEdit extends TComponent {
  public onInput: ((value: string) => void) | null = null;
  bindEventHandler(screen: Screen): void {
    this._screen = screen;
    if (this.onInput) {
      screen.on(this.id, "input", (ev: any) => {
        this.onInput!(ev?.value || "");
      });
    }
  }
}
```

Contoh widget — TDataGrid (dari cashew.ts)

```
// Penggunaan di app
const grid = new TDataGrid(
```

```

"sensor",
[
  { key: "node_id", label: "Node", width: 140 },
  { key: "value", label: "Nilai", width: 80, align: "right" },
  { key: "timestamp", label: "Waktu", width: "40%" },
],
[],
{ height: 300 },
);
form.add(grid);
grid.onSort = (key, dir) => { reloadSorted(key, dir); };
grid.onRowClick = (index, record) => { openDetail(record); }; // index = row-key stabil

// Data inkremental – hanya baris baru yang dikirim ke browser (hemat WS)
await grid.appendData(newRows);

```

```

// (dari cashew.ts – TDataGrid) auto-bind oleh TForm.run(): mount
// ConnectedDataGrid + daftarkan handler sort & klik row.
bindEventHandler(screen: Screen): void {
  this._screen = screen;
  void this.grid
    .mount(
      screen,
      (key, dir) => { if (this.onSort) this.onSort(key, dir); },
      (index, record) => { if (this.onRowClick) this.onRowClick(index, record); },
    )
    .catch(() => {});
}

// (dari cashew.ts – TDataGrid) auto-refresh: render data awal
async refresh(screen: Screen): Promise<void> {
  await this.grid.setData(this._data);
}

```

TTimer — interval ala Delphi

```

const timer = new TTimer("tmr-update", 1000);
timer.onTimer = () => { chart.pushData(Date.now(), sensor.value); };
timer.enabled = true;
form.add(timer); // bindEventHandler auto-start saat form.run()

```

Layout helpers

```

form.add(
  HStack(
    new TButton("btn-a"),
    new TButton("btn-b"),
    Spacer(), // flex: 1 – mendorong tombol berikutnya ke kanan
    new TButton("btn-c"),
  ),
);

// Dua panel bersebelahan dengan divider yang bisa di-drag
form.add(TSplitHorizontal(leftPanel, rightPanel, "lfr"));

```

Latihan / Praktik

1. Baca `wiki/cashew-in-a-nutshell.md` — ikuti quick start dan komponen layout.
2. Bangun app dengan `TForm` + `TPanel` + `TEdit` + `TListBox`: ketik di `TEdit`, klik tombol, hasilnya masuk ke `TListBox`.
3. Gunakan `TDialogs.confirm()` untuk konfirmasi sebelum menghapus item.
4. Baca `src/mirror/lib/cashew.ts` — cari implementasi `TForm.run()`, `bindEventHandler()`, dan `refresh()`.
5. Buat `TDataGrid` dengan `appendData()` untuk data yang terus bertambah (log / telemetry).

Referensi

- `wiki/cashew-in-a-nutshell.md`, `wiki/emerald-in-a-nutshell.md`
 - `wiki/course/00-overview.md` §10
 - `src/mirror/lib/cashew.ts`, `src/mirror/lib/emerald.ts`
 - `src/mirror/opt/dome/dome-client-dom.js`
 - Changelog: `wiki/changelogs/cashew.md`, `wiki/changelogs/dome.md`
-

Modul 20 — selesai. Lanjut ke [Modul 21 — Asteracea & TDE](#).

RFC-TSIX-EDU-002 | Modul kedua puluh satu kurikulum TSIX. Memahami Window Manager TSIX: aplikasi PixelSpace biasa (fullscreen frameless) yang mengelola lifecycle aplikasi GUI lain.

Poin penting: **Asteracea bukan bagian kernel/DOME** — ia aplikasi PixelSpace biasa (Ring 4, sandboxed worker) yang berjalan fullscreen. Ia me-launch, mengontrol, dan mengelola lifecycle aplikasi GUI lain.

Tujuan Pembelajaran

- ☐ Menjelaskan posisi Asteracea dalam arsitektur
- ☐ Menjelaskan filosofi queue-based IPC (MessageBus)
- ☐ Menjelaskan taskbar (pinned/running/foreign)
- ☐ Menjelaskan launcher & fuzzy search
- ☐ Menjelaskan lifecycle events via `/opt/asteracea/wm-pid`

Konsep Inti

Spesifikasi

Aspek	Deskripsi
Lokasi	<code>src/mirror/opt/asteracea/asteracea.ts</code>
Mode	Fullscreen, frameless
Model	Queue-based IPC (MessageBus)
Proto	PixelSpace Protocol via DOME
Privilege	Ring 4 (sandboxed worker)
Auto-start	<code>/etc/rc.local.ts</code> (setelah DOME siap — poll <code>/var/run/dome.ready</code>)

Filosofi queue-based

Semua event masuk ke antrian FIFO (MessageBus) dan diproses satu per satu — mencegah race condition dan starvation:

```
App A —(shell.send)→ Kernel —(ipc_message)→ MessageBus Queue → handlers
App B —(GUI_REQ)→ DOME —(ipc_message)→ MessageBus Queue → handlers
Browser —(click)→ DOME —(shell.send)→ MessageBus Queue → handlers
```

 Alur notifikasi desktop: app → kernel → Asteracea → browser *Sumber:* wiki/diagram/ASTERACEA_WM-1.mmd

Komponen

- **Taskbar:** pinned / running / foreign windows
- **Launcher:** start menu + **fuzzy search** aplikasi
- **Login & wallpaper**
- **Desktop Context Menu (DCM)**
- **App State Manager**

IPC lifecycle events

WM mendengarkan `GUI_WINDOW_*` lifecycle events: `GUI_WINDOW_CREATED`, `GUI_WINDOW_MINIMIZED`, `GUI_WINDOW_RESTORED`, `GUI_WINDOW_MAXIMIZED`, `GUI_WINDOW_UNMAXIMIZED`, `GUI_WINDOW_CLOSED`, dan `GUI_WINDOW_ERROR`. Aplikasi GUI lain

berkomunikasi dengan WM melalui `/opt/asteracea/wm-pid` (PID file) — Emerald broadcast event ke Asteracea.

Perbaikan & Fitur Terbaru

Agar sinkron dengan kode nyata, beberapa perilaku berikut ditambahkan:

- **Daemonize** — `main` diawali `shell.daemonize("Asteracea Window Manager")`. Proses lepas dari console `tty1`; stdio dialihkan ke `/dev/null`. Log tetap masuk `/var/log/syslog` via VFS (`std.log/std.error`), dan render GUI tetap normal karena lewat DOME.
- **Icon & tooltip taskbar untuk foreign app** — handler `GUI_WINDOW_CREATED` meneruskan `payload.icon || "■"` ke `registerForeignApp()`. Icon dipakai untuk tombol taskbar; `title` (judul window) menjadi tooltip via `prop title` (diterjemahkan ke `data-tt` di DOME). Foreign app kini bisa menampilkan icon custom.
- **Handler `GUI_WINDOW_MAXIMIZED` / `GUI_WINDOW_UNMAXIMIZED`** — keduanya memanggil `transitionTo(appId, "RUNNING")` dan menetapkan style taskbar aktif (sama seperti `GUI_WINDOW_RESTORED`). State WM tetap sinkron setelah maximize lewat context menu taskbar; klik taskbar berikutnya langsung menjadi minimize toggle.
- **Login tanpa prefill password** — field password tidak lagi diisi `value: "1"` (inisialisasi `loginPass = ""`). Prefill lama akan "menempel" di depan password baru dan membuat login selalu gagal setelah `passwd` mengganti password.
- **Persistensi wallpaper** — sebelum menulis `/opt/asteracea/wallpaper/current-wp.b64`, `showWallpaperDialog` membuat folder `/opt/asteracea/wallpaper` (di-try-catch). Sebelumnya folder tidak ada → `fs.writeFile` gagal diam-diam → wallpaper blank setelah reboot.

Kode Sumber

File	Peran
<code>src/mirror/opt/asteracea/asteracea.ts</code>	Window manager
<code>src/mirror/opt/asteracea/menu/*.menu</code>	Konfigurasi menu launcher
<code>src/mirror/etc/rc.local.ts</code>	Auto-start DOME + Asteracea (poll <code>/var/run/dome.ready</code>)

Latihan / Praktik

1. Login ke TSIX — amati desktop: taskbar, launcher, wallpaper.
2. Buka launcher dan ketik sebagian nama app — amati fuzzy search.
3. Baca `wiki/ASTERACEA_WM.md` — pelajari MessageBus dan AppState Manager.
4. Baca `src/mirror/opt/asteracea/asteracea.ts` — cari handler lifecycle `GUI_WINDOW_*`.

Referensi

- `wiki/ASTERACEA_WM.md` — dokumentasi lengkap WM
- `wiki/PIXELSPACE_DEVELOPER_GUIDE.md` §3, §9
- `wiki/course/00-overview.md` §10
- `src/mirror/opt/asteracea/asteracea.ts`, `src/mirror/opt/asteracea/menu/*.menu`

Modul 21 — selesai. Lanjut ke [Modul 22 — State Replay & Persistence](#).

RFC-TSIX-EDU-002 | Modul kedua puluh dua kurikulum TSIX. Memahami bagaimana UI dipulihkan saat browser reconnect (F5): `windowStates`, `pruneWindowState`, orphan discard, `findElementById` di 3 layer, dan proteksi navigasi (`allowReload`).

F5 di browser seharusnya **tidak menghapus desktop**. Sejak proteksi navigasi (DOME 2026-08-06), refresh butuh konfirmasi dulu. DOME menyimpan semua `MOUNT_NODE` & `UPDATE_PROPS` di `windowStates` dan mengirim ulang ke browser baru setelah reload dikonfirmasi. Pruning memastikan tidak ada state orphan yang bocor ke replay.

Tujuan Pembelajaran

- ☐ Menjelaskan mekanisme State Replay
- ☐ Menjelaskan `pruneWindowState` dan mengapa diperlukan
- ☐ Menjelaskan browser-side orphan discard
- ☐ Menjelaskan `findElementById` di 3 layer
- ☐ Menjelaskan proteksi navigasi (`allowReload`) dan replay setelah reload
- ☐ Menjelaskan fallback `_savedRect` → `_origRect` saat restore
- ☐ Menjelaskan skenario bug yang dicegah (dialog residue setelah F5)

Konsep Inti

State Replay

`windowStates` = `Map<wid, state[]>` — menyimpan semua `MOUNT_NODE` & `UPDATE_PROPS` sebagai "replay payloads".

Lifecycle (di `dome.ts`, sisi server):

- `CREATE_WINDOW` → `windowStates.set(wid, [])` — inisialisasi riwayat state.
- `MOUNT_NODE` → push `{ type, wid, node, targetId }` ke array.
- `UPDATE_PROPS` → push `{ type, wid, targetId, props }` ke array.
- `UNMOUNT_NODE` → `pruneWindowState(wid, targetId)` lalu broadcast.
- `DESTROY_WINDOW` → `windowStates.delete(wid)` — riwayat dibersihkan.

Replay saat reconnect (F5):

```
Browser F5 → WebSocket reconnect
→ DOME kirim CREATE_WINDOW (semua window aktif)
→ DOME kirim semua stored states (MOUNT_NODE + UPDATE_PROPS)
→ jika entry.isMaximized → kirim MAXIMIZE_WINDOW (status maximize dipulihkan)
→ DOME kirim tema terakhir (lastThemeColors)
→ Browser build ulang UI + terima event dari user
```

Sisi browser, saat socket `onclose` semua window sesi lama dibuang (`state.windows.clear()`), lalu `onopen` siap menerima replay dari server.

State Pruning (`pruneWindowState`)

Saat `UNMOUNT_NODE` dipanggil, DOME membersihkan state terkait:

- Collect:** untuk tiap state `MOUNT_NODE`, kumpulkan SEMUA `nodeId` dari tree-nya (termasuk child) via `collectNodeIds`.
- Filter:** hapus `MOUNT_NODE` jika `state.targetId === targetId` atau ada `nodeId` di tree-nya yang sama dengan `targetId`; `UPDATE_PROPS` hanya dihapus jika `state.targetId === targetId` (exact match).

Ini mencegah **child states** (dari `setContent()`) tersisa setelah parent di-unmount → mencegah residu DOM saat F5.

Browser-side orphan discard

Jika `handleMountNode` menerima `MOUNT_NODE` dengan `targetId` yang tidak ditemukan di DOM, node **di-discard** (dibuang) — bukan fallback ke `win.content`. Mencegah:

- Wallpaper dialog child states muncul di desktop setelah F5
- Login overlay residue
- Modal/dialog orphan elements

Overlay layer search

`findElementById(wid, nodeId)` mencari di 3 level:

1. `win.el.querySelector([data-tsix-id=...])` — di dalam window
2. `__global_start_menu__` — start menu global
3. `__tsix_overlay_layer__` — overlay layer (launcher, dialog)

Ini memungkinkan elemen yang di-extract ke overlay (mis. `launcher-grid`) tetap ditemukan oleh mount/replay.

Proteksi Navigasi (konfirmasi refresh)

Sejak DOME 2026-08-06, refresh tidak lagi langsung jalan. IIFE `protectNavigation()` di `dome-client-core.js` memakai flag `allowReload`:

- **F5 / Ctrl+R / Cmd+R** di-intercept di fase capture → `preventDefault()` → dialog `window.confirm("Yakin mau refresh TDE? ...")`.
 - **OK** → set `allowReload = true`, lalu `location.reload()`.
 - **Cancel** → halaman tetap utuh, tidak ada reload.
- **back/forward, close tab, navigasi lain** → event `beforeunload` memunculkan dialog native browser; di-skip jika `allowReload` (agar refresh yang sudah dikonfirmasi tidak prompt ganda).
- `pointerdown` (sekali) menandai interaksi — mencegah Chrome 91+ mematikan `beforeunload` pada halaman yang belum disentuh.

Setelah reload yang dikonfirmasi, **state replay tetap bekerja**: DOME mengirim ulang `CREATE_WINDOW` + stored states ke koneksi WebSocket baru, sehingga desktop ter-restore penuh.

Restore aman (`_savedRect` → `_origRect`)

Saat window di-restore (mis. setelah minimize), `handleRestoreWindow` memakai `_savedRect`. Jika `_savedRect` berdimensi nol (`width <= 0 || height <= 0`) — mis. sisa maximize saat window masih `display:none` — kode **fallback ke `win.el._origRect`**. Ini mencegah window ter-restore ke `(0,0,0,0)` (pojok kiri atas dengan ukuran aneh).

Snippet (level kode)

Pruning (`pruneWindowState`)

```
const collectNodeIds = (node: any, ids: Set<string>): void => {
  if (!node || typeof node !== "object") return;
  if (typeof node.id === "string" && node.id) ids.add(node.id);
  if (Array.isArray(node.children)) {
    for (const child of node.children) collectNodeIds(child, ids);
  }
};

const pruneWindowState = (wid: string, targetId: string): void => {
  const states = windowStates.get(wid) || [];
  const filtered = states.filter((state: any) => {
    if (state?.type === "MOUNT_NODE") {
      const nodeIds = new Set<string>();
      collectNodeIds(state.node, nodeIds);
      // hapus MOUNT_NODE jika targetId-nya = targetId, atau
      // nodeId di dalam tree-nya ada yang = targetId (termasuk child)
    }
  });
};
```

```

    if (state.targetId === targetId || nodeIds.has(targetId))
      return false;
  }
  if (state?.type === "UPDATE_PROPS") {
    if (state.targetId === targetId) return false; // exact match saja
  }
  return true;
});
windowStates.set(wid, filtered);
};

```

Browser orphan discard

```

if (targetId) {
  const parent = TSIX.findElementById(wid, targetId);
  if (parent) {
    parent.appendChild(domEl);
  } else {
    // Parent not found – discard (jangan fallback ke win.content)
  }
} else {
  win.content.appendChild(domEl);
}

```

Pencarian lintas layer (findElementById)

```

function findElementById(wid, nodeId) {
  // 1) di dalam window milik app
  const win = state.windows.get(wid);
  if (win) {
    const el = win.el.querySelector(
      '[data-tsix-id="' + CSS.escape(nodeId) + '"]',
    );
    if (el) return el;
  }
  // 2) start menu global
  const gm = document.getElementById("__global_start_menu__");
  if (gm) {
    if (gm.getAttribute("data-tsix-id") === nodeId) return gm;
    const child = gm.querySelector(
      '[data-tsix-id="' + CSS.escape(nodeId) + '"]',
    );
    if (child) return child;
  }
  // 3) overlay layer (launcher, dialog)
  const overlay = document.getElementById("__tsix_overlay_layer__");
  if (overlay) {
    if (overlay.getAttribute("data-tsix-id") === nodeId) return overlay;
    const child = overlay.querySelector(
      '[data-tsix-id="' + CSS.escape(nodeId) + '"]',
    );
    if (child) return child;
  }
  return null;
}

```

Proteksi navigasi (protectNavigation)

```

(function protectNavigation() {
  let allowReload = false; // true saat user sudah konfirmasi refresh

  // Tandai interaksi user – Chrome 91+ mematikan beforeunload
  // jika halaman belum pernah disentuh.
  document.addEventListener("pointerdown", function () { }, { once: true });

  const isRefreshKey = (e) =>
    e.key === "F5" ||
    ((e.key === "r" || e.key === "R") && (e.ctrlKey || e.metaKey));

```



```
document.addEventListener(
  "keydown",
  function (e) {
    if (!isRefreshKey(e) || e.repeat) return;
    e.preventDefault();
    const ok = window.confirm(
      "Yakin mau refresh TDE?\n\n" +
      "Semua window yang berjalan akan ditutup lalu di-restore " +
      "ulang dari server.\n\n" +
      "OK = refresh, Cancel = batal",
    );
    if (ok) {
      allowReload = true;
      window.location.reload();
    }
  },
  true, // fase capture
);

window.addEventListener("beforeunload", function (e) {
  if (allowReload) return; // refresh yang sudah dikonfirmasi
  e.preventDefault();
  e.returnValue = "";
});
})();
```

Fallback restore (`_savedRect` → `_origRect`)

```
let saved = win._savedRect;
// Safety: rect nol (mis. sisa maximize saat window masih hidden)
// → jangan restore ke (0,0,0,0), pakai posisi/ukuran asli.
if (saved && (saved.width <= 0 || saved.height <= 0)) {
  saved = win.el._origRect || null;
}
```

Latihan / Praktik

1. Buka beberapa window (termasuk dialog/modal), lalu tekan F5 — muncul dialog konfirmasi; pilih **OK** — amati UI pulih via state replay. Pilih **Cancel** — desktop tidak berubah.
2. Tutup sebuah window yang punya child dialog, lalu F5 (OK) — amati tidak ada residu dialog.
3. Baca `src/mirror/opt/dome/dome.ts` — cari `windowStates`, `pruneWindowState`, dan blok replay saat reconnect.
4. Baca `src/mirror/opt/dome/dome-client-dom.js` — cari orphan discard di `handleMountNode`.
5. Baca `src/mirror/opt/dome/dome-client-core.js` — cari `findElementById` (3 layer) dan `protectNavigation` (`allowReload`).
6. Baca `src/mirror/opt/dome/dome-client-windows.js` — cari fallback `_savedRect` → `_origRect` di `handleRestoreWindow`.

Referensi

- [wiki/PIXELSPACE_DEVELOPER_GUIDE.md §11](#)
- [wiki/course/00-overview.md §10](#)
- `src/mirror/opt/dome/dome.ts` (`windowStates`, `pruneWindowState`, `replay`)
- `src/mirror/opt/dome/dome-client-dom.js` (`orphan discard`)
- `src/mirror/opt/dome/dome-client-core.js` (`findElementById`, `protectNavigation`)
- `src/mirror/opt/dome/dome-client-windows.js` (`_savedRect` → `_origRect`)

Modul 22 — selesai. Bagian VII tuntas. Lanjut ke [Modul 23 — Development Workflow](#).

RFC-TSIX-EDU-002 | Modul kedua puluh tiga kurikulum TSIX. Memahami siklus host↔VFS: install, vfs-bootstrap, sync-vfs, vfs-pull, SYNC_TO_HOST, userlib-update, dan SDK mirroring.

TSIX punya dua dunia: **host** (folder `src/mirror` + `src/common` di repo) dan **VFS** (file database, path-nya dibaca dari `kernel.database` di `src/sysconfig.json`, default `system.db`). Developer menulis di host, lalu menyinkronkan ke VFS. Ada juga jalur sebaliknya — aplikasi di dalam VFS menulis kembali ke host (root-only).

Tujuan Pembelajaran

- ☐ Menjelaskan siklus host ↔ VFS
- ☐ Membedakan `install`, `vfs-bootstrap`, `sync-vfs`, `vfs-pull`
- ☐ Menjelaskan `SYNC_TO_HOST` (syscall)
- ☐ Menjelaskan `userlib-update` (SDK mirroring)
- ☐ Menjelaskan peran `tsconfig paths (@tsix/*)`

Konsep Inti

Dua dunia: host vs VFS

- Host** — tempat menulis kode: `src/mirror/` (rootfs: `bin`, `lib`, `etc`, `sbin`, ...) dan `src/common/` (framework: `SyscallCode`, `IPCTypes`, `GUITypes`).
- VFS** — filesystem yang di-boot kernel: file database `system.db` (BKFS/SQLite). Kernel membaca path-nya dari `kernel.database`.
- src/root/** — hasil `vfs-pull`: salinan VFS di host untuk inspeksi / perubahan permanen.

Semua script host mengambil path DB lewat `scripts/lib/db-path.ts`, agar nilainya selalu sinkron dengan path hasil instalasi.

[!IMPORTANT] Sejak `install.ts` ada, fresh install **tidak perlu** menjalankan `vfs:bootstrap` terpisah — installer sudah melakukan seluruh sync rootfs sendiri.

Tabel script & arah sinkronisasi

Perintah	File	Arah	Kegunaan
<code>npm run install</code>	<code>scripts/install.ts</code>	host → DB	Fresh install interaktif: buat DB baru + tulis <code>sysconfig.json</code> + sync rootfs penuh
<code>npm run vfs:bootstrap</code>	<code>scripts/vfs-bootstrap.ts</code>	host → DB	Bulk sync <code>src/mirror</code> → / dan <code>src/common</code> → <code>/lib/common</code>
<code>npx ts-node scripts/sync-vfs.ts <path></code>	<code>scripts/sync-vfs.ts</code>	host → DB	Sync satu file (dipasang di VS Code "run on save")
<code>npm run vfs:pull</code>	<code>scripts/vfs-pull.ts</code>	DB → host	Tarik seluruh VFS → <code>src/root</code> (kecuali folder runtime)
<code>syscall SYNC_TO_HOST (53)</code>	<code>src/kernel/Syscalls.ts</code>	DB → host	Root-only: tulis satu file VFS ke host (terbatas dalam project root)
<code>sudo userlib-update</code>	<code>src/mirror/sbin/userlib-update.ts</code>	DB → host	Di dalam TSIX: sync <code>/lib</code> → <code>src/.tsix_sdk/lib</code>
<code>npm run bkfs:create</code>	<code>scripts/create-bkfs.ts</code>	—	Buat DB kosong (opsional skeleton direktori standar)

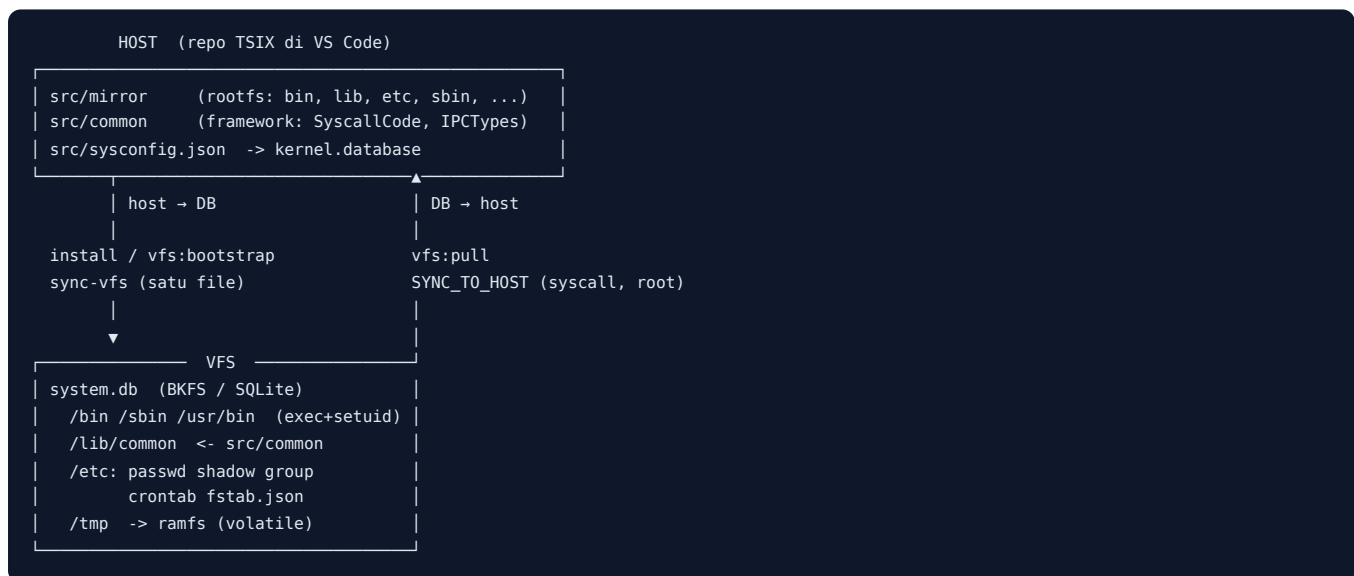
Konfigurasi bersama

- `src/sysconfig.json` — `kernel.database` (path DB), `kernel.rootHostPath`, `scheduler.workerEntryPath`, `scheduler.bootEntry`, `network.interfaces`.
- `tsconfig.json` — `@tsix/*` → `src/.tsix_sdk/lib/*`, `src/root/lib/*`, `src/mirror/lib/*`; `@common/*` → `src/common/*`; `@bin/*` → `src/root/bin/*`, `src/.tsix_sdk/bin/*`.

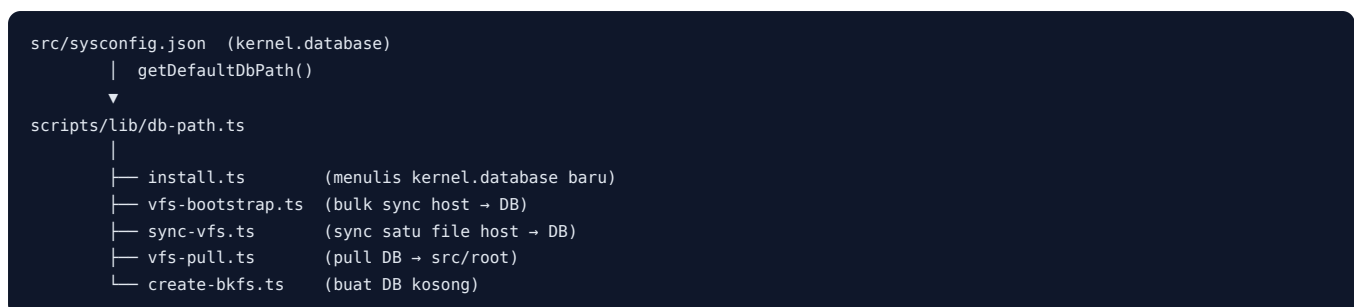
[!NOTE] Tidak ada sinkronisasi otomatis host → VFS saat boot. Host adalah "permukaan penulisan"; VFS adalah sumber kebenaran runtime. Perubahan dipindahkan lewat script/syscall di atas.

Alur / Cara Kerja

Diagram siklus host ↔ VFS



sysconfig → db-path → scripts



1. Fresh install (npm run install)

1. Tanya konfigurasi interaktif: hostname, default user login, broker MQTT, address per-interface, port MQTT default, verbose kernel, path DB baru, (opsional) password root.
2. Tulis hasil ke `src/sysconfig.json` (menyimpan kernel.database baru).
3. Buat file `.db` baru. Jika file sudah ada dan tanpa `--force` → berhenti; dengan `--force` → file lama di-backup ke `*.bak-<timestamp>`.
4. Sync rootfs: `src/mirror` → `/`, lalu `src/common` → `/lib/common`. Setiap `.ts` di-transpile jadi sidecar `.js`; direktori eksekusi diberi mode eksekusi (`/sbin` = `0744`, lainnya `0755`); `login`, `passwd`, `sudo` diberi SetUID (`4755` root).
5. Sync eksplisit file `/etc` tanpa ekstensi: `passwd`, `shadow` (mode `0640`), `group`, `crontab`, `profile`, `motd`, `fstab.md`, `pkg-demo.conf`.
6. Tulis `fstab.json` fresh — hanya `/tmp` sebagai ramfs (mode `1777` sticky); mount dev-spesifik tidak dibawa.

7. Kosongkan `/etc/crontab`.
8. Jika password root diisi → di-hash bcrypt dan ditulis ke `/etc/shadow`.
9. Verifikasi: total node, jumlah akun `passwd`, daftar group, entri `shadow`.

```
npm run install                # interaktif, path dari sysconfig
npm run install -- --path data/tsix.db    # path database tertentu
npm run install -- --path data/tsix.db --force # timpa (auto-backup)
npm run install -- --defaults            # non-interaktif, semua default
npm run install -- --no-config           # skip tulis sysconfig.json
```

2. Bulk sync (npm run vfs:bootstrap)

Sinkronisasi massal `src/mirror` → `/` dan `src/common` → `/lib/common`. Berguna untuk instalasi awal perangkat atau setelah banyak edit sekaligus.

```
npm run vfs:bootstrap          # DB default dari sysconfig
npm run vfs:bootstrap -- data/test.db # path lain (posisi argumen)
```

3. Sync satu file (sync-vfs)

Untuk satu file cepat (mis. saat save di editor). Pemetaan path: `src/mirror/*` → `/*`, `src/common/*` → `/common/*`. File `.ts` ikut di-transpile ke sidecar `.js`.

```
npx ts-node scripts/sync-vfs.ts src/mirror/bin/hello.ts
```

4. Pull balik (npm run vfs:pull)

Menarik seluruh VFS → `src/root`, dengan pemetaan: `/etc/*` → `src/root/etc/*`, `/root/*` → `src/root/home/root/*`, lainnya → `src/root/<path>`. Folder runtime dikecualikan: `dev`, `tmp`, `proc`, `logs`, `var`.

```
npm run vfs:pull
```

5. Syscall SYNC_TO_HOST (di dalam TSIX)

Aplikasi di dalam VFS (root) bisa menulis file ke host lewat syscall `SYNC_TO_HOST`. `userlib-update` dan `vfs-pull` (versi `sbin`) menggunakannya.

```
# dari shell TSIX, sebagai root
sudo userlib-update # sync /lib → src/.tsix_sdk/lib
vfs-pull            # tarik seluruh VFS → host root
```

Kode Sumber

- `scripts/install.ts` — fresh install agent (host → DB).
- `scripts/vfs-bootstrap.ts` — bulk sync host → DB.
- `scripts/sync-vfs.ts` — sync satu file host → DB.
- `scripts/vfs-pull.ts` — tarik DB → `src/root`.
- `scripts/create-bkfs.ts` — buat DB kosong.
- `scripts/lib/db-path.ts` — path DB bersama dari `sysconfig`.
- `src/kernel/Syscalls.ts` — implementasi `SYNC_TO_HOST` / `SYNC_FROM_HOST`.
- `src/mirror/lib/UserLib.ts` — `syncToHost()` / `syncFromHost()`.
- `src/mirror/sbin/userlib-update.ts`, `src/mirror/sbin/vfs-pull.ts` — perintah di dalam TSIX.

Snippet (level kode)

Path DB bersama (scripts/lib/db-path.ts)

```
export function getDefaultDbPath(): string {
  const configPath = path.resolve(__dirname, "../../src/sysconfig.json");
  try {
    const raw = fs.readFileSync(configPath, "utf8");
    const cfg = JSON.parse(raw);
    if (cfg && typeof cfg.kernel === "object" && cfg.kernel.database) {
      return String(cfg.kernel.database);
    }
  } catch (_) {
    /* abaikan - pakai fallback */
  }
  return "system.db";
}
```

Mode eksekusi & SetUID (isSetuidBinary / applyBinaryMode)

```
const EXEC_DIRS = ["/bin", "/sbin", "/usr/bin", "/usr/local/bin"];

function isSetuidBinary(vfsPath: string): boolean {
  return /\bin\/(login|passwd|sudo)\.(ts|js)$/.test(vfsPath);
}

function isExecutableBinary(vfsPath: string): boolean {
  return EXEC_DIRS.some(d => vfsPath.startsWith(d + "/"));
}

function applyBinaryMode(bkfs: BKFS, vfsPath: string, label = "INSTALL"): void {
  if (isSetuidBinary(vfsPath)) {
    bkfs.chmod(vfsPath, 0o4755); // setuid root
    bkfs.chown(vfsPath, 0, 0);
  } else if (isExecutableBinary(vfsPath)) {
    // /sbin = root-only (0o744), lainnya 0o755
    bkfs.chmod(vfsPath, vfsPath.startsWith("/sbin/") ? 0o744 : 0o755);
  }
}
```

syncDir rekursif (vfs-bootstrap.ts, identik di install.ts)

```
const syncDir = (hostDir: string, vfsDir: string) => {
  if (!bkfs.exists(vfsDir)) bkfs.mkdir(vfsDir, 0, 0, 0o755);
  const items = fs.readdirSync(hostDir);
  for (const item of items) {
    const fullHostPath = path.join(hostDir, item);
    const fullVfsPath = path.join(vfsDir, item).replace(/\\/g, "/");
    const stats = fs.statSync(fullHostPath);

    if (stats.isDirectory()) {
      syncDir(fullHostPath, fullVfsPath);
      continue;
    }

    const isTarget =
      item.endsWith(".ts") || item.endsWith(".js") ||
      item.endsWith(".json") || item.endsWith(".html") ||
      item.endsWith(".css") || item.endsWith(".menu") ||
      item.endsWith(".mp3") || item.endsWith(".wav");
    if (!isTarget) continue;

    // Binary (mp3/wav) disimpan sebagai latin1
    const isBinary = item.endsWith(".mp3") || item.endsWith(".wav");
    const content = isBinary
      ? fs.readFileSync(fullHostPath).toString("latin1")
      : fs.readFileSync(fullHostPath, "utf8");
    bkfs.touch(fullVfsPath, content);

    // Transpile TS → JS sidecar (yang dieksekusi runtime)
    if (fullVfsPath.endsWith(".ts")) {

```

```

const result = esbuild.transformSync(content, {
  loader: "ts", format: "cjs", target: "node18", sourcemap: "inline",
});
if (result.code) {
  const jsPath = fullVfsPath.substring(0, fullVfsPath.length - 3) + ".js";
  bkfs.touch(jsPath, result.code);
  if (isSetuidBinary(jsPath) || isExecutableBinary(jsPath)) {
    applyBinaryMode(bkfs, jsPath);
  }
}
}

if ((isSetuidBinary(fullVfsPath) || isExecutableBinary(fullVfsPath)) &&
  (fullVfsPath.endsWith(".ts") || fullVfsPath.endsWith(".js"))) {
  applyBinaryMode(bkfs, fullVfsPath);
}
}
};

```

Alur `install.ts` (diringkas dari sumber)

```

async function main() {
  const opts = parseArgs(process.argv.slice(2));
  const cfg = loadConfig();
  const dbRel = opts.dbPath || cfg.kernel.database || "system.db";
  let rootPassword = "";

  // 1. Konfigurasi interaktif → cfg (hostname, user, broker, port, verbose, db path, root password)
  // 2. Tulis src/sysconfig.json
  if (opts.writeConfig) {
    cfg.kernel.database = path.relative(PROJECT_ROOT, dbPath).replace(/\\/g, "/");
    saveConfig(cfg);
  }

  // 3. Buat DB baru (file lama di-backup bila --force)
  const bkfs = new BKFS(dbPath);
  // 4. Sync rootfs
  syncDir(bkfs, MIRROR_ROOT, "/"); // src/mirror → /
  syncDir(bkfs, COMMON_ROOT, "/lib/common"); // src/common → /lib/common
  // 5. fstab fresh (hanya /tmp ramfs), crontab kosong, password root opsional
  // 6. Verifikasi & tutup DB
}

```

Syscall `SYNC_TO_HOST` (`src/kernel/Syscalls.ts`)

```

case SyscallCode.SYNC_TO_HOST: {
  if (!this.isRoot(pcb))
    throw new Error("Permission Denied: Only root or root group members can perform physical sync.");
  const { vfsPath, hostPath } = args;

  // Keamanan: hostPath tidak boleh keluar dari project root
  const projectRoot = process.cwd();
  const absoluteHostPath = path.resolve(projectRoot, hostPath);
  if (!absoluteHostPath.startsWith(projectRoot)) {
    throw new Error("Security Violation: Target path is outside project root.");
  }

  const { vfs, relativePath } = this.mountManager.resolve(vfsPath);
  const content = vfs.read(relativePath);
  if (content === null) throw new Error(`Source file not found in VFS: ${vfsPath}`);

  const parentDir = path.dirname(absoluteHostPath);
  if (!fs.existsSync(parentDir)) fs.mkdirSync(parentDir, { recursive: true });
  fs.writeFileSync(absoluteHostPath, content);
  this.logger.info(`[SYNC] Applied VFS:${vfsPath} -> HOST:${hostPath}`);
  return true;
}

```

UserLib.`syncToHost` (`src/mirror/lib/UserLib.ts`)

```
public async syncToHost(vfsPath: string, hostPath: string): Promise<boolean> {  
  return await this.dispatch(SyscallCode.SYNC_TO_HOST, { vfsPath, hostPath });  
}
```

Latihan / Praktik

1. Jalankan `npm run install -- --defaults` — periksa `src/sysconfig.json` tertulis dan DB baru ter-buat, lalu `npm start`.
 2. Jalankan `npm run install` (interaktif) — isi hostname & user login, set password root, amati sync per file.
 3. Tulis app baru di `src/mirror/bin/`, lalu `npm run vfs:bootstrap` — jalankan app di shell TSIX.
 4. Ubah satu file lalu `npx ts-node scripts/sync-vfs.ts src/mirror/bin/<file>.ts` — bandingkan kecepatannya vs bootstrap penuh.
 5. Ubah file di VFS (dari shell TSIX), lalu `npm run vfs:pull` — cek perubahan muncul di `src/root`.
 6. Baca `wiki/Memulai.md` — ikuti alur mulai dari nol.
-

Referensi

- `wiki/Memulai.md`, `wiki/Panduan-Developer.md`
 - `wiki/course/00-overview.md` §11 (Alur Pengembangan / Sync VFS)
 - `wiki/Virtual-File-System.md` — Sinkronisasi VFS ↔ Host
 - `scripts/install.ts`, `scripts/vfs-bootstrap.ts`, `scripts/sync-vfs.ts`, `scripts/vfs-pull.ts`, `scripts/create-bkfs.ts`, `scripts/lib/db-path.ts`
 - `src/kernel/Syscalls.ts`, `src/mirror/lib/UserLib.ts`, `src/mirror/sbin/userlib-update.ts`, `src/mirror/sbin/vfs-pull.ts`
-

Modul 23 — selesai. Lanjut ke [Modul 24 — Best Practices & Penulisan App](#).

RFC-TSIX-EDU-002 | Modul kedua puluh empat kurikulum TSIX. Menutup kurikulum dengan praktik terbaik: dua gaya app, error = window, tema, batching UPDATE_PROPS, dan Piagam Antigonon.

Kurikulum selesai di sini. Modul ini adalah "kitab praktik" — kumpulan aturan yang menjaga aplikasi TSIX tetap konsisten, aman, dan mudah dipelihara.

Tujuan Pembelajaran

- ☐ Membedakan dua gaya app (`IProgram` vs `Program()`)
- ☐ Menjelaskan "error = window" (std.error 4 langkah)
- ☐ Menjelaskan batching `UPDATE_PROPS`
- ☐ Menjelaskan Piagam Antigonon (4 aturan GUI)
- ☐ Menerapkan praktik terbaik pada app baru

Konsep Inti

Dua gaya aplikasi

1. Class `main` implements `IProgram` (gaya klasik, mayoritas di `/bin`):

Kontrak ada di `src/mirror/lib/IProgram.ts`:

```
export interface OSContext {
  std: StdLib;
  fs: FsLib;
  shell: ShellLib;
  aux: AuxLib;
}

export interface IProgram {
  execute(os: OSContext, args: string[]): Promise<string | void>;
}
```

Contoh minimal (pola `cat.ts` / `echo.ts`):

```
import { IProgram, OSContext } from "../lib/IProgram";

export class main implements IProgram {
  async execute(os: OSContext, args: string[]): Promise<string | void> {
    const { std } = os;
    await std.print("Hello from TSIX!\n");
    // void → WorkerEntry memanggil shell.exit(0)
  }
}
```

2. `Program()` wrapper + proxy singletons (baru, disarankan untuk app baru & GUI):

```
import { Program, std } from "@tsix/Application";

export const main = Program(async (args: string[]) => {
  await std.print("Hello from TSIX!\n");
});
```


Keuntungan alias: **path independence** — import tetap sama di mana pun file diletakkan (/bin/ , /root/test/ , dst). Menghilangkan error "Module not found" saat pindah folder.

Perbandingan dua gaya:

Aspek	main implements IProgram	Program() wrapper
Import	<code>import { IProgram, OSContext } from "../lib/IProgram"</code>	<code>import { Program, std, fs, shell, net, db } from "@tsix/Application"</code>
Entry point	<code>async execute(os: OSContext, args)</code>	fungsi callback <code>async (args) => ...</code>
Akses library	<code>os.std, os.fs, os.shell, os.aux</code>	proxy singleton <code>std, fs, shell, net, db</code>
Path	import relatif ke lokasi file	import absolut <code>@tsix/*</code> — path independent
Error handling	manual (WorkerEntry menangkap error lain → <code>realExit(1)</code>)	otomatis: wrapper tangkap error → <code>std.error()</code> → rethrow
Cocok untuk	utilitas CLI di /bin	app baru, GUI (Emerald), daemon

Cara kerja wrapper (src/mirror/lib/Application.ts): `Program(fn)` mengembalikan class yang implements `IProgram`. Saat `execute` dipanggil, ia menyimpan `OSContext` ke `global._tsix0sc` lalu memanggil `fn(args)`. Jika `fn` melempar error, wrapper mengirim error itu ke parent via `std.error()` (menjadi window error di desktop), lalu melempar ulang agar `WorkerEntry` turut menanganinya. Proxy `std/fs/shell/net/db` membaca `UserLib` dari `global._tsixLib` secara lazy — itulah kenapa import-nya path-independent.

Error = "Window"

Jangan biarkan app crash diam-diam. `std.error(message, context?, wid?)` menampilkan error sebagai **window popup di desktop** (diproses WM/Asteracea). Alur 4 langkah (lihat `StdLib.error()` di `src/mirror/lib/UserLib.ts`):

1. **Log ke syslog** — tulis baris `[ERROR]` + timestamp ke `/var/log/syslog`.
2. **Ekstrak fileHint** — baca stack trace, cari file sumber pertama yang bukan library internal (`UserLib`, `Application`, `emerald`).
3. **Broadcast ke parent (WM)** — kirim IPC `GUI_WINDOW_ERROR` berisi `{ wid, pid, file, error, context, timestamp }` ke parent process; Asteracea menampilkannya sebagai popup.
4. **Print TTY merah** — cetak `\x1b[31m[ERROR]\x1b[0m <message>` ke terminal (stderr style).

Setiap langkah non-fatal — jika salah satu gagal (mis. syslog belum ada), langkah lain tetap jalan.

Batching UPDATE_PROPS

Jangan kirim satu `UPDATE_PROPS` per perubahan properti. **Batch** beberapa perubahan lalu kirim bersamaan — mengurangi jumlah IPC dan latency render. Di Emerald (`src/mirror/lib/emerald.ts`, class `Window`) mekanismenya sudah otomatis:

- `updateProps(targetId, props)` tidak langsung mengirim — perubahan di-merge ke `dirtyProps` (Map), lalu `scheduleFlush()` dijadwalkan.
- `scheduleFlush()` memakai `setTimeout(..., 0)`: semua perubahan dalam satu async tick dikumpulkan dulu, baru di-flush di akhir tick. Jika sudah ada flush terjadwal, tidak ada jadwal ganda.
- `flushNow()` mengirim semua `UPDATE_PROPS` tertunda sekaligus.
- `Screen.update()` / `setText()` / `setVisible()` / `setStyle()` semuanya memakai jalur batch ini.
- `setContent()` sengaja **tidak** di-batch (kirim langsung via `sendImmediate`) — satu operasi atomik `innerHTML=""` lalu `MOUNT_NODE` per child.
- `Screen.on()` memanggil `flush()` otomatis setelah bind — memastikan listener terpasang di browser.

Praktik terbaik: jangan menulis loop `for (...)` await `app.update(id, props)` dengan harapan terkirim per item — Emerald sudah menggabungkannya untuk Anda.

Piagam Antigona (4 aturan GUI)

Piagam Antigonon adalah aturan strict untuk AI agent & developer yang menulis GUI TSIX (sumber: `wiki/PIXELSPACE_DEVELOPER_GUIDE.md`):

- 1. **NO DOM di Userland** — `@tsix/emerald` TIDAK boleh menyentuh `document.*` atau `window.*`. Semua rendering lewat protokol PixelSpace (Worker → Kernel → DOME → Browser).
- 2. **State-Sync** — Jangan kirim `UPDATE_PROPS` dalam tight loop; gunakan batching.
- 3. **Memory Cleanup** — Setiap node yang di-unmount, bersihkan event listener-nya (cegah kebocoran memori).
- 4. **Type Safety** — Semua payload harus sesuai `IGUIPayload`; payload cacat → `SIGKILL`.

[!TIP] Dua praktik berdekatan yang tak kalah penting: pasang listener **mount-time** (`onClickId/onInputId` di props saat build, bukan `app.on()` setelah mount) untuk menghindari bug `cloneNode` (Modul 19 & 20), serta jaga UI agar **state replay** aman setelah F5 (Modul 22).

Alur / Cara Kerja

Langkah menulis app TSIX yang benar (gaya `Program()`):

- 1. **Pilih gaya** — utilitas CLI singkat: `main implements IProgram`. App baru & GUI: `Program()`.
- 2. **Tulis entry point** — `export const main = Program(async (args) => {...})` (atau `export class main implements IProgram`).
- 3. **Deklarasikan mode GUI** (jika app menampilkan window) — `export const appMode = "gui"`; agar `WorkerEntry` menangani GUI.
- 4. **Akses library via proxy** — `std, fs, shell, net, db` — import sekali dari `@tsix/Application`.
- 5. **GUI: pakai Emerald** — `new Screen({...})`, `app.mount(...)`, `app.on(...)`, `app.loopUntilClose()`.
- 6. **Error = window** — panggil `await std.error(msg, context)` saat gagal; jangan biarkan crash diam-diam.
- 7. **Tema** — `await theme.loadCurrent()` + `theme.watch()` agar warna mengikuti prefs Asteracea.
- 8. **Batch update** — serahkan batching ke Emerald; jangan kirim `UPDATE_PROPS` per item.
- 9. **Patuhi Piagam Antigonon** — tanpa DOM langsung, tanpa tight loop, cleanup listener, payload sesuai `IGUIPayload`.

Kode Sumber

File	Isi
<code>src/mirror/lib/IProgram.ts</code>	Kontrak <code>IProgram</code> & <code>OSContext</code>
<code>src/mirror/lib/Application.ts</code>	<code>Program()</code> wrapper + proxy singletons (<code>std, fs, shell, net, db</code>)
<code>src/mirror/lib/UserLib.ts</code>	<code>StdLib</code> (<code>print, log, error</code>), <code>FsLib</code>
<code>src/mirror/lib/emerald.ts</code>	<code>Window/Screen</code> , batching <code>UPDATE_PROPS</code> , <code>bindHandler</code>
<code>src/mirror/lib/theme.ts</code>	<code>ThemeProvider</code> — <code>loadCurrent, watch, switchTo, applyToDome</code>
<code>src/mirror/bin/cat.ts, echo.ts</code>	Contoh gaya <code>IProgram</code>
<code>src/mirror/root/ps-sample2.ts, ps-sample3.ts</code>	Contoh <code>Program()</code> + <code>Emerald</code>
<code>src/mirror/opt/set-theme/set-theme.ts</code>	Contoh tema (switch dark/light)
<code>wiki/PIXELSPACE_DEVELOPER_GUIDE.md</code>	Piagam Antigonon

Snippet (level kode)

App CLI — gaya `IProgram` (readfile)

`execute(os, args)` memakai `os.std / os.fs` (kontrak `OSContext`). String yang dikembalikan akan dicetak `WorkerEntry`; `void` → `shell.exit(0)`:

```
import { IProgram, OSContext } from "../lib/IProgram";

export class main implements IProgram {
  async execute(os: OSContext, args: string[]): Promise<string | void> {
    const { std, fs } = os;
    if (args.length === 0) {
      return "Usage: readFile <path>"; // string return → dicetak WorkerEntry
    }
    const fd = await fs.open(args[0], "r"); // fd < 0 = gagal buka
    if (fd < 0) {
      await std.print(`Error: Cannot open ${args[0]}\n`);
      return;
    }
    const content = await fs.read(fd);
    await std.print(content ?? "");
    await fs.close(fd);
  }
}
```

[!TIP] Untuk baca file sekali jalan, FsLib sudah menyediakan `readFile(path)` yang membungkus `open` → `read` → `close`.

App CLI — gaya `Program()` (hello)

```
import { Program, std } from "@tsix/Application";

export const main = Program(async (args: string[]) => {
  await std.print(`Hello, ${args[0] ?? "world"}!\n`);
});
```

`std.error` — error menjadi window

```
// Log syslog → ekstrak fileHint → broadcast GUI_WINDOW_ERROR → TTY merah
await std.error("Disk full", "myapp");
await std.error("Connection timeout", "net", app.wid); // wid opsional
```

Saat app memanggil ini, WM/Asteracea menampilkan popup error di desktop — bukan crash console.

Tema — import & apply

```
import { theme } from "@tsix/theme";
import { Screen, div } from "@tsix/emerald";

// Muat tema aktif (terang/gelap) sesuai prefs Asteracea
await theme.loadCurrent();
theme.watch(); // ikut perubahan tema saat app berjalan

// Pakai warna terpusat — tanpa hardcode hex
const app = new Screen({ title: "App", width: 400, height: 300 });
await app.mount(
  div({
    id: "root",
    style: { background: theme.colors.bg, color: theme.colors.text },
  }),
);

// Ganti tema global + broadcast THEME_CHANGED ke DOME
await theme.switchTo("theme-light.json");
// Terapkan ke window ini (titlebar, border, shadow)
await theme.applyToDome(domePid, app.wid);
```

File tema berada di `/opt/asteracea/theme-*.json` (`theme-dark.json`, `theme-light.json`). Contoh lengkap: `src/mirror/opt/set-theme/set-theme.ts`.

Latihan / Praktik

1. Tulis app "Hello" dalam dua gaya — bandingkan struktur & import.
 2. Tambahkan `std.error()` di app yang sengaja crash — amati window error di desktop.
 3. Refactor app GUI agar mem-batch `UPDATE_PROPS` — ukur perbedaan responsivitas.
 4. Baca `wiki/DEVELOPER_GUIDE_SCRIPTING-V2.md` — kerjakan contoh script v2.1.
-

Referensi

- `wiki/PIXELSPACE_DEVELOPER_GUIDE.md` — Piagam Antigonon & panduan PixelSpace
 - `wiki/DEVELOPER_GUIDE_SCRIPTING-V2.md`, `wiki/Panduan-Developer.md`
 - `wiki/course/00-overview.md` §9
 - `src/mirror/lib/IProgram.ts`, `src/mirror/lib/Application.ts`, `src/mirror/lib/UserLib.ts`
 - `src/mirror/lib/emerald.ts`, `src/mirror/lib/theme.ts`
 - `src/mirror/root/ps-sample2.ts`, `src/mirror/root/ps-sample3.ts`
 - `src/mirror/opt/set-theme/set-theme.ts`, `src/mirror/opt/test/gui-demo.ts`, `src/mirror/opt/file-cruiser/file-cruiser.ts`
 - `src/mirror/bin/cat.ts`, `src/mirror/bin/echo.ts`
-

Modul 24 — selesai. Kurikulum TSIX lengkap! 🎉 Lanjut menulis? Perbarui `toc.md` dan buat terjemahan `.en.md` (FORMAT §5).